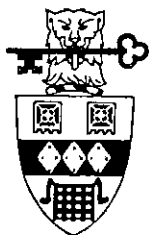


VECTOR

APL-ication: Papers and Panels

- * 16-page Educational Supplement
- * Statgraphics reviewed
- * Prys-Williams on Forward Chaining
- * more Missembling from Allan Gay
- * prize competition



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.5 No.2 November 1988

Contributions

All contributions to VECTOR may be sent to the Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible (DSDD) diskette. APL code should be kept separate from the text, and can be accepted as camera-ready copy, or as APL*PLUS/PC workspaces or IBM .AIO files.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1988-89

Category	Fee	Vectors	Passes
Non-voting Student	£5	1	1
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10	5
Overseas Corporate	£140	10	
Sustaining	£360	50	5

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Treasurer on the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £100 for half-page or less (subject to layout constraints).

Deadlines for bookings and copy are given under the Quick-reference Diary

Advertisements should be booked with, and sent to, Cathy Dargue, whose address is given with the Index to Advertisers.

CONTENTS

EDITORIAL:		Page
APL NEWS	Adrian Smith	3
Quick Reference Diary	Adrian Smith	5
General Correspondence		7
British APL Association News		
Committee Activities	Graham Parkhouse	15
The Education VECTOR	Alan Sykes	17
Report on the 1988 Summer Course	John Clift	18
APL at Mill Hill School	Bruce Dickson	22
A Pilot Course in APL	Bruce Dickson	24
The Latest on I-APL	Anthony Camacho	26
Counting in APL	Alan Sykes	27
News from Sustaining Members	Cathy Dargue	33
 REVIEWS		
APL Product Guide	Cathy Dargue	36
New Product Announcements		46
Product Reviews		
APL as a Tool of Thought(V)	Derek Wilson	47
Statgraphics	Gavin Schwarzenbach	49
APL Booklist(Renaissance Data Systems)	Ed Shaw	54
 RECENT MEETINGS:		
Syracuse Meeting on APL in AI	John Adams	56
APL-ication Report ...		
Conference Highlights	Adrian Smith	61
Notes on Given Papers	Adrian Smith	62
Panel: the APL Market Niche	Donnelly et al	71
Computers in the Boardroom	David Preedy	77
 TECHNICAL SECTION		
Prize Caption Competition		82
Hackers Corner: the TSO session	Adrian Smith	83
Technical Correspondence	Sullivan, Hayward	85
The Limits of Forward Chaining	Allan Prys-Williams	88
Missembling	Allan Gay	105
 British APL Association Software Library		
Short Catalogue	David Ziemann	121
Software Submission Details	David Ziemann	124
Software Order Form	David Ziemann	126
Index to Advertisers		127

MicroAPL for...

APL on the widest range of micros

PCs

MultiAPL - APL.68000 under PCDOS

Sample configurations:

68000/1MB	£ 995
68000/1MB/Maths processor	£1295
68020/1MB/Maths processor	£2995
68020/4MB/Maths processor	£5495

Prices include: Plug-in card, APL.68000, Keyboard stickers, Manual, STSC emulation software

STSC's APL*PLUS

£425

STSC's APL*PLUS II PC

£1395

Single user micros

APL.68000 for the Apple Macintosh	£86.91
APL.68000 for the Apple Macintosh II	£260.82
APL.68000 for the Atari ST	£86.91
APL.68000 for the Commodore Amiga	£86.91

Prices include: APL.68000, Keyboard stickers, Manual, Reference card

Multi-User systems

APL.68000 runs under the following operating systems:

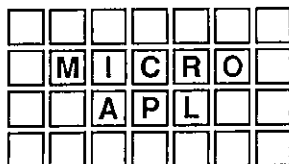
Unix, CP/M68K, Wicat MCS, Mirage

on computers which include:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride

Prices range from £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted

Editorial: OOPS and WIMPS and the Way Ahead

by Adrian Smith

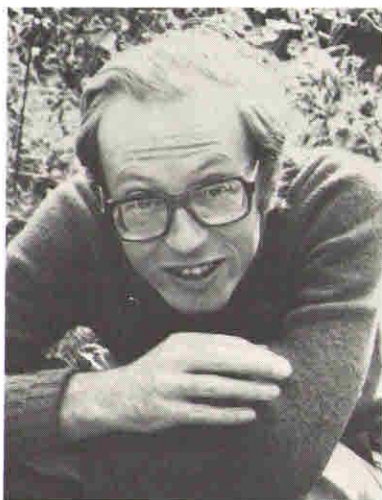
Interviewer: What other projects did you work on at MIT?

Bricklin: I did a wide variety of things. I was a member of the team implementing APL, and by 1973 I was running the project that implemented APL. I was also involved in the LISP project.

In case you don't know, Dan went on to create VisiCalc, and thus to revolutionise the relationship between the business community and the computer. (The above quote is taken from a whole series of interviews in "Programmers at Work", published by Microsoft Press.)

Of course Dan was heavily constrained by the Apple-II, a machine with just 16K of memory and the (very occasional) floppy disk. In the circumstances he did a pretty incredible job, but he had to cut away many of the most powerful APL ideas (operators, function definition) out of his design.

So APL begat VisiCalc, and VisiCalc begat Lotus-123, and Lotus-123 begat Quattro and Excel; and great is the profusion thereof. These days you can run Excel on a 16M Mac-II with fabulous windows and wonderful graphics and you still can't do anything other than plus-reduction!! All those cloners and copiers and extenders of VisiCalc completely missed the point! Instead of taking the chance to add in genuine vector arithmetic (in spite of appearances Lotus-123 is an essentially scalar environment) they let the marketing men take over and concentrated all their effort on flashing up the tiny subset of APL that Dan had managed to squeeze into his tiny Apple-II.



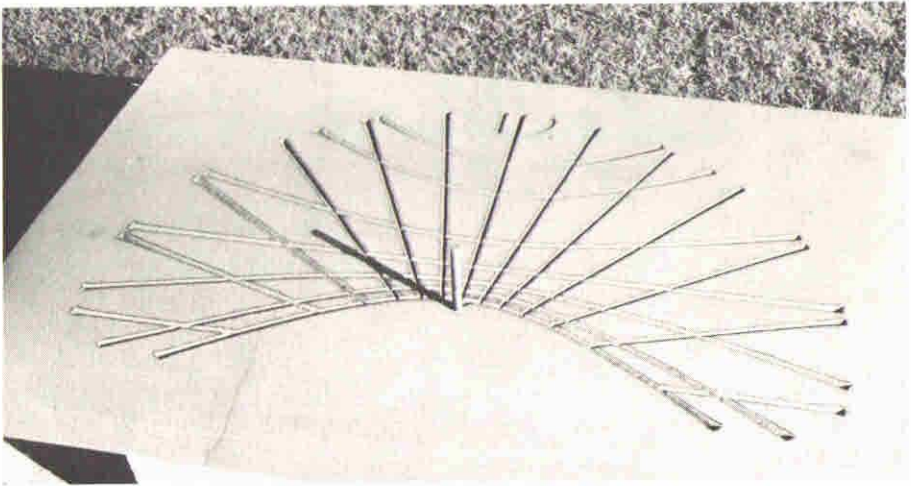
Adrian Smith
(by Richard Smith)

Let me make a suggestion. At APL-ication we got excited about three things: WIMPs, OOPs and SQL. We all of us agreed that APL is a wonderful tool for writing user-interfaces, that it would extend easily to take in object-oriented programming, that it fits naturally around the concepts and data-structures of SQL. It is also one of the very few

languages that can already handle concurrent programming (shared variables), and that will readily pipeline for vector processing.

Do you think there just might be a market for an OS/2 product with all the hooks into Presentation Manager, support for object-oriented programming, built-in access to SQL data, straightforward tools for concurrent programming, ?

I suggest we call this new language Ken, after its author. What do you think?



□TS

Dates for future issues of VECTOR

	Vol.5 No.3	Vol.5 No.4	Vol.6 No.1
Copy date	1st Jan 89	1st March 89	1st June 89
Ad booking	12th Jan 89	10th March 89	12th June 89
Ad Copy	23rd Jan 89	25th March 89	25th June 89
Distribution	January 89	April 89	July 89

Quick Reference Diary

by Jonathan Martin

At the Annual General Meeting in April, I asked for and received a number of suggestions to group together into a series of meetings. The following maps the topics to future BAPLA meeting dates:

Date	Venue	Event
18 November	London	APL & the Outside World
One of the most exciting developments in the APL world is the fusion between APL and modern techniques such as Parallel Processing, Relational Databases, Expert Systems etc...		
13 January 1989	London	Applications in APL
The standard format for APL meetings, four speakers relating their experiences in implementing various APL applications.		
17 February	London	APL - The Next 5 Years
Representatives from the major APL suppliers to tell the world what they are working on now and what they expect to be doing in the future.		
17 March	London	Meeting (to be announced)
21 April	London	Annual General Meeting Find out how your vote influenced the structure of next years committee.

All British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm.

APL CONSULTANTS

LONDON & READING

Account Managers	(6 years+)	to	25K
Senior Consultants	(4-6 years)	to	21K
Consultants	(2-4 years)	to	17K
Junior Consultants	(1-2 years)	to	13K

Are your APL skills and potential being recognised and rewarded?

Cocking & Drury consultants have been implementing successful decision support applications for 10 years, with clients who appreciate the productivity benefits of APL.

In our professional team you will experience a range of APL environments - APL*Plus, VSAPL, APL2 and Unix, on both mainframes and micros. You will also be developing systems which, increasingly, need to interface with non-APL Information Centre products.

Of course as the leading APL consultancy, in a rapidly expanding market, we offer a rewarding career with first class benefits - profit sharing, free health insurance, and a non-contributory pension.



For further details call Ralph Wilson on 0734 588835

COCKING & DRURY LTD.

155 Friar Street, Reading, RG1 1HE

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All letters should be addressed to the Editor, and should indicate whether they are for the general or technical section. (Letters containing APL code will normally be classed as 'technical'.) The editor reserves the right to edit any letter unless the writer states that it is to be published in full or not at all.

APL Conference on CIX

From: Craig Lomas

28th October 1988

Like most APLers, I have felt the need to spread the word about APL (sometimes with a missionary-like zeal!!). In the past this was not easy because of a number of difficulties, particularly with the APL font. However with the common availability of IBM PC's and compatibles with soft font capability, this is no longer a problem. Now is the time if at all that APL can be brought to a wider audience.

To this end I have just started an APL conference on CIX. CIX, or rather the Compulink Information Exchange is the largest computer based conference system in the UK. It currently offers over 200 active conferences on a diverse range of subjects. CIX is quite easy to use, and is available throughout the UK.

I invite the APL community to take an active part in this conference, to help spread the word about APL and to facilitate the exchange of ideas among APLers

CIX can be joined by simply dialling (01) 399-5252 with a V21/V22/V23 modem and then enrolling on-line. For those outside the London area CIX can be contacted via your local Mercury X25 node, you do not have to have a Mercury account.

I can be contacted on (01) 702-0401, or send messages to 'CJL' on CIX

Yours sincerely

Craig Lomas

I-APL and the APL Keyboard

From: Anthony Camacho (a reply to F H D van Batenburg VECTOR 5.1 p14) July 1988

The default I-APL keyboard is very different from any other APL keyboard. How could the project have been so stupid and arrogant as to confuse an already poor situation by introducing yet another non-standard keyboard? Here is an explanation of how the decision came about and permission to anyone who cares enough and has the skill to implement an I-APL with any keyboard they like. After the introduction, the three parts deal with the aim to please the new APL user, some technical considerations and the advantage of having a standard default keyboard.

Introduction: Microcomputer Keyboards

There is no such thing as a standard keyboard. A program designed to run on many types of microcomputer will not be able to hold every character in the same place on every keyboard. Most microcomputers have a way to enter the 7-bit ASCII set of codes; this contains 32 control codes, space, 94 printable characters and the delete character. Many of these can display the ANSI characters, but in other countries there are often accented and special characters which occupy the positions of some of the less used characters in the ANSI set.

The Wasted Scholarship

In 1986 the project listed all the known allocations of keys and codes to APL characters. There are quite a few, even if those using EBCDIC are ignored. Most computer manufacturers have implemented APL and sought a way to allow it to be used from their common ASCII terminals. I-APL collected the lists of keywords, abbreviations or transliterations for VAX, Prime, Data General, Burroughs and ICL as well as variations in the quad-AV between most of the well known microcomputer APLs - APL/PC, Sharp APL, APL*PLUS/PC, PortaAPL, MicroAPL, VIZ::APL, TisAPL, APL90.

All these painstaking lists were ignored when we realised that the nub of the matter was whether we wished to please people who already use APL or those who haven't yet tried it. Those manufacturers were trying to make it possible for people with a requirement for APL to buy and use their machinery, not to encourage the use of a notation which can be executed on a computer. I-APL is not trying to sell computers. More relevant to I-APLs needs was Llewellyn Jones' APL-pip; an ASCII transliteration of APL to allow it to be transmitted and published through ASCII channels while preserving enough of the look of traditional APL to allow an APLer to read and understand it.

I-APL Aims to Attract New People to APL

New users of APL who know a keyboard will prefer to have as many keys as possible in their familiar places. Established APLers may prefer something similar to the golfball typewriter on which APL was first printed or to their current implementation.

Unless a compromise can be found which suits both old and new users I-APL would prefer to please the new users. After all, most established APLers will have no need to use I-APL; they are already equipped with an APL. Not one of them is going to give up APL as a result of I-APL's choice.

The Idea of a Union Keyboard

A key should cause the display of the character marked on it wherever that is possible. There does not seem to be any adequate reason for moving the characters which are the same in ASCII and APL from one place on the keyboard to another. To do so would merely make APL harder to use without any benefit to the learner. This is one of the principles of the union keyboard - that similar symbols should stay on the familiar keys - and I-APL has adopted it. There are minor differences: the APL star has five points, the ASCII one sometimes has six, the stile is unbroken, whereas the ASCII vertical bar has a short gap in it, and the symbol for AND has longer legs than the circumflex but these will not cause anyone difficulties in finding the corresponding keys. So the 22 ANSI symbols correspond to their APL look-alikes. This policy will also reduce the number of keys which need extra markings such as key stickers.

Fitting the APL Symbols into the ASCII/ANSI Set

I-APL hopes to have the interpreter running on over a dozen machines. The interpreter is designed to allow all APL characters to be used on machines which can handle extended character sets. Some machines, such as CP/M machines with dumb terminals may have to manage with the ASCII/ANSI set. There are not enough codes in that set for all the APL characters even when the lower case alphabet is reallocated to be used for APL. Of the 94 displayable characters 58 are allocated so far:

- 10 the decimal digits
- 26 the alphabet
- 22 the symbols common to ANSI and APL
-
- 58 total allocated so far

There are 36 ANSI characters and 41 or more APL characters left.

The Chosen 36 APL Characters

The symbols not yet allocated to keys are these:

left arrow, right arrow, up arrow, down arrow, multiply, divide, high minus, rho, epsilon, iota, up stile, down stile, logarithm, circle, representation, base value, format, execute, jot, transpose, rotate, OR, del, quad, quote-quad, quad divide, lamp, alpha, omega, not equal to, greater than or equal to, less than or equal to, slash bar, backslash bar, comma bar, circle bar, delta, NOR and NAND.

The list begins with the most needed symbols and ends with those which could at a pinch be foregone on ASCII-only keyboards and displays. As the I-APL quad-AV contains all the APL characters, anything can be constructed using quad-AV indexes. On machines with extended keyboards it may be possible for the porter to provide access to all 256 codes. On ASCII-only keyboards there is not room for the last five in the above list. The symbols to leave out were chosen because they are used little and can easily be provided as combinations of other functions and operators. The delta was left out because it is only required in one place - to delete lines during editing with the del editor - and the tilde can easily be used instead. Although alpha and omega are not used in most commercial APL work and might have been omitted, I-APL took the view that direct definition will be of great value in education. We considered using A and W instead of alpha and omega and concluded there were too many disadvantages - we chose to include them rather than NOR and NAND.

Allocation of Selected Symbols to Keys

Where a key has to be used for a character different from the mark on it some kind of mnemonic is called for. This list of allocations begins with the best justified and ends with the arbitrary.

Similar shapes ...

x to be multiply
v to be OR
o to be circle
i to be iota
a to be alpha
w to be omega
e to be epsilon
z to be not equal to
% to be divide
' (grave accent) to be high minus
to be quad

Distinctive characters ...

& to be quote-quad
 \$ to be del (the dollar is commonly a control character)
 @ to be branch
 _ (underline) to be assign

Similar Symmetries ...

n to be encode	) These four fit well
u to be decode (upside-down n)	) together as a mnemonic
h to be format (n plus a bit)	) set. APLers only have
y to be execute (upside-down h)	) to remember the n.
q to be upgrade	)
d to be downgrade	) One symmetry preserved.
p to be upstile	)
b to be downstile	)
(to be less than or equal to	) The points are on the
} to be greater than or equal to	) same sides.
s to be transpose - it has the same diagonal symmetry	

Others ...

r to be rho - r is the equivalent of the greek rho	
f to be rotate - phi looks like rotate and is greek f	
j to be jot	)
m to be matrix divide	) Initial letters may
l to be logarithm	) help with these.
t to be up arrow (take)	)
k to be down arrow (arbitrary)	
" to be lamp - association of ideas	
c to be slash bar (arbitrary)	
g to be backslash bar (arbitrary)	

Overstrikes do not provide extra characters .

Some Technical Considerations

The Porter Controls the Shell

I-APL is written in a language called DE. It stands for Development Environment. The DE code is interpreted by the program the porter writes - nowadays commonly called DEGO. The porter provides a complete shell insulating the interpreter from any machine dependent features. Every character input to the interpreter or output from it passes

through the DEGO program. The porter is in complete control of this channel and so can make any translation of characters he wishes.

The bulk of DEGO is now available to prospective porters for the 6502, 8086 family, 68000 family and Z80. All that a porter needs to add is the links to the native operating system to get and display characters, to switch the printer toggle on and off, to save and load workspaces using the native filing system and to provide the quad-MC interface for machine code routines. The simplest possible port to a new machine would simply send, display and receive the ASCII equivalent characters.

Characters are Sent and Received in Two Forms

To make it as easy as possible to deal with whatever range of characters is available, I-APL passes characters for output as a pair of bytes and receives characters input in one of a pair of bytes. One output byte is the index in quad-AV and allows use of the complete range; the other is the ASCII equivalent according to the tables above. One input byte is the index in quad-AV and the other is ASCII which will be translated according to the tables above. The porter chooses which output byte to use and which input byte to provide and the choice may vary from one byte to another so hybrids between ASCII equivalence and full quad-AV are possible. All ports so far are hybrids.

So a porter can implement keywords or mimic whatever keyboard he chooses. He can also offer alternatives of keyboard mapping much as other interpreters allow a choice of keywords, traditional APL or a union keyboard. These choices might be selected by adding a parameter to the initial call of I-APL. I-APL is happy to give porters permission to do this; all we require is that a proper job is done and that it is thoroughly tested before issue.

Accented and Special Characters

In many countries special characters or accented characters are required and these are often allocated to positions in the ASCII set of infrequently used ANSI characters. When only the ASCII 94 characters can be used, none of these extras is available. On extended keyboards I-APL allows for them as follows:

There is a spare alphabet in the I-APL quad-AV which defaults to lower case underlined. This may be used for any special characters. As the keys marked with the special characters will return ASCII codes the porter will have to trap these and translate them.

The porter will also have to specify a method for entry of the ANSI characters normally returned with the trapped codes (commonly a combination of keys).

Advantages of a Standard Default Keyboard

The key allocations explained above will probably be those most widely used in implementations of I-APL. The project has some influence with porters and can encourage them to follow a particular line. Which line should I-APL favour?

One of the strengths of I-APL is that if the machine-dependent functions quad-MC, quad-TS and quad-DL are not used then an I-APL workspace will run on any I-APL machine and produce the same results. It is possible to send a workspace from a PC clone to a BBC B, a Sinclair Spectrum or a VAX and run it unchanged.

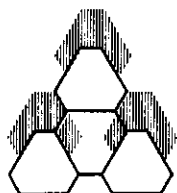
It would be very good to have all machines that use I-APL, compatible in other ways such as the keyboards. This is not possible (as explained above), but it is possible for the ASCII character set to be treated similarly on all machines.

Many schools have a mixture of computers and have difficulties using them all in a single class because of the variations between machines. If the variations are only in the positions of the keys while the association between key labels and APL characters stays constant, most of the difficulties are overcome. Whereas it would cause confusion if the quad were sometimes on the key marked # and sometimes on the SHIFT-L, little trouble is caused by the varying position of the asterisk or plus sign; so long as an APL character is always on the key with the corresponding ASCII mark there is little difficulty holding a class with mixed machines.

A teacher faced with the need to equip a class with a mixture of PC clones, CBM 64s and Apples would probably prefer to have one machine per pupil or per pair of pupils, all equipped with an APL which uses the same key allocations rather than manage with the PC clones, shared one machine between several pupils. The same instructions to the pupils about how to enter a line of APL can be given to all users of I-APL. The extra speed and capacity of the commercial APLs is less important in the classroom than having fewer pupils per machine and being able to say that the quad is on the key marked with a hash or that execute is SHIFT-Y. Furthermore the commercial APLs are expensive.

I-APL is asking all porters to provide the ASCII to APL key associations shown above as the default, while encouraging anyone who has the time and effort to offer alternatives which may suit established APLers and advanced pupils better.

Anthony Camacho
2, Blenheim Road
St. Albans



APL PEOPLE

THE APL EMPLOYMENT AGENCY

Among our **many** current vacancies, we are **urgently** seeking people for the following:

- ★ **DEALER-SUPPORT IN THE CITY**
to £50K + large bonus + profit-share + car
highly experienced APLers with energy and
enthusiasm. No previous financial knowledge
required.
- ★ **OPERATIONAL RESEARCH ANALYSTS**
Various locations. Interesting work and good
career prospects. Salary negotiable
- ★ **INFORMATION CENTRE ANALYST – Herts.**
2 years' APL experience required. Opportunity to
learn Dyalog APL. Salary up to £16,500

These are just a selection of our current vacancies. We are always pleased to hear from people seeking either a permanent position or contract work in APL.

Contact Jill Moss on 0225 462602 during office hours
or 0225 333618 eves & w/ends

or write to her at The Old Malthouse, Clarence St., Bath BA1 5NS

British APL Association News

by Graham Parkhouse

Meetings

Those present enjoyed an excellent special educational meeting at the Royal Over-Seas League on the evening of September 2nd, when Anthony Camacho, Alan Sykes and Frank Evans gave talks levelled at newcomers to APL. The intention had been to attract teachers and others on the I-APL mailing list, but unfortunately the attendance was very small. Due to administrative fumbling the I-APLers were sent details of APL-ication instead of details of this meeting. This was rectified, but only at the eleventh hour. Only three teachers attended, which was very disappointing. Apologies to the speakers and to those who would otherwise have benefitted from being at an excellent meeting.

A post-mortem at the September committee meeting did not prevent disaster overtaking the October Friday afternoon meeting which had to be cancelled at the last minute because the labels for posting the flyers kept not materialising. Should we consider a scheme of answerphone owner accreditation? The cheery offer to return your call can become so familiar as to breed contempt.

BAA Administration

A solution to our problems may come in the pleasant form of Alison Chatterton, our new paid part-time administrator. Alison has two young children, the youngest at playschool, and it suits her to take on some BAA administration to do at home. She is being equipped with a PC clone and software to be able to set up articles for VECTOR, manage the software library, keep our records up to date, and print off labels. We are not giving her an answerphone!

Valerie Lusmore wants to re-recruit as many experienced APLers as she can, who may have enjoyed corporate membership of the BAA in the past through previous employment. If you know of any such people please give Valerie a ring on 0225 62601 in the daytime.

Outstanding Achievement Award

Please may we have nominations for 1989 awards. There are two categories of award, as mentioned at the 1987 AGM, for recent work in APL and for spreading awareness about APL. The award for 1987/88 went to John Scholes for his work in developing Dyalog APL, for which he received a glass apple, which he holds until the next award, and a cheque for £500. This year the prize will again be £500.

Nominations should consist of the name and address of the candidate and two signed and dated statements, one from the proposer and one from the seconder, explaining what the outstanding achievement is and why the candidate merits the award. If, in the adjudicating committee's view, none of the achievements are sufficiently outstanding, no award will be made.

Please send your nominations to the Hon. Secretary of the BAA, Graham Parkhouse, Dept. of Mechanical Engineering, University of Surrey, GUILDFORD, Surrey, GU2 5XH, to arrive by 28th February 1989.



APL CONSULTING

**QUALITY APL CONSULTANCY SERVICE
SPECIALIZING IN:**

- ★ Project Management
- ★ Prototyping and Database Design
- ★ Marketing Information Systems
for Management
- ★ Design and implementation of
large financial systems

We can solve your APL problems, large or small,
and will supply a single consultant or a supervised
team according to your needs.

Contact Jill Moss or Tim Perry on 0225 462602
APL Consulting Ltd, The Old Malthouse, Clarence St., Bath BA1 5NS

**THE
EDUCATION
VECTOR**

November 1988

Edited by Alan Sykes

Reprinted from VECTOR Vol.5 No.2. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Graham Parkhouse, Dept. of Mech. Eng., University of Surrey, GUILDFORD.

Contents

Editorial	Alan Sykes	2
Report on the 1988 Summer Course	John Clift	2
APL at Mill Hill School	Bruce Dickson	4
A Pilot Course in APL	Bruce Dickson	4
The Latest I-APL News	Anthony Camacho	7
Counting in APL	Alan Sykes	11

Dr Alan Sykes,
Dept of Management Science and Statistics,
University College of Swansea,
Singleton Park, SWANSEA SA2 8PP

The Education Vector

by Alan Sykes

Welcome to Education Vector once more. The good news is that Education Vector is to be a regular feature of each issue of Vector. The bad news is that Norman Thomson is no longer Education officer, and as the new incumbent, I must take this opportunity to thank him for all his hard work. I am sure that Norman will continue to play a vital role in the future developments of APL in Britain, and that future pages of Vector will be blessed by his erudite contributions.

The use of I-APL in schools continues to grow. During the summer, two courses were held, one masterminded by Norman in Winchester, and another one in Swansea. As a result, I-APL has been introduced to a number of comprehensive schools and colleges. It is important that Vector recognises this growth of application. Hence the new committee decided that Education Vector would be designed in the form of a supplement that can be mailed directly to the growing numbers of people on the I-APL mailing list in Britain.

The nature of this supplement in the future is largely up to you, the reader. In this first issue I hope newcomers to APL will find the first of a series of articles of benefit in explaining some elementary aspects of APL, and at the same time presenting some useful functions. Also there are reports from John Clift who attended the course at Winchester, and from Bruce Dickson at Mill Hill School. In addition, Anthony Camacho brings us up to date with I-APL news.

I extend an invitation to all readers of Education Vector and Vector to submit articles, letters, and comments for future editions of Education Vector, so that it can truly be a successful forum for the exchange of ideas on the use of APL in all spheres of education.

A Report on the Second Winchester Summer Course in APL

by John Clift

Promoting APL as a suitable language for use in secondary schools seems at first sight an almost impossible task. If the average computing teacher has heard anything at all about APL it is that the language is unreadable and written in a bizarre character set. It goes without saying that the organisers of the APL course for teachers relish the challenge and to add further spice the course is arranged at the end of the summer term when teachers' energies are at their lowest ebb.

A rather apprehensive group of ten teachers assembled at Winchester for the weekend of 8-11 July. We found that we represented a wide range of educational establishments,

comprehensives, independent schools and further education colleges. Even one county advisor managed to spare some time from his heavy round of administrative duties. Our experience of computing was equally varied as about half our number were principally teachers of mathematics.

Little time was wasted by our energetic and enthusiastic instructors, Norman Thomson, Anthony Camacho, John Durran and Sylvia Camacho. The computer centre at King Alfred's College was reassuringly equipped with BBC computers, but as these were running the I-APL interpreter we soon found ourselves unlearning familiar concepts: * no longer meant multiply, $\div$ was used for division instead of / and arithmetic now had to be done starting from the right.

Adapting to the APL symbols proved less of a problem than might have been expected. We were soon able to solve elementary problems and practise in the "voluntary" evening sessions which only terminated on the arrival of the caretaker at 9 o'clock. The main impression this work left me with was the ease with which APL handles operations on vectors and matrices.

Norman Thomson led the course at a fast pace and, looking back, I am astonished at the number of topics we covered in three days. Guest speaker Michael Kingston helped to place this activity in context by stressing the features of APL which made it attractive in commercial installations.

On the final morning time was found for a discussion on the future of APL in education. No definite conclusions were reached and this may have been because APLers wanted to talk about general principles underlying teaching while the teachers tried to move the discussion to particular applications of APL in the school situation.

I certainly plan to introduce APL into my teaching next year, but I am fortunate in having one class which is not working to any set syllabus, leaving me free to experiment with new ideas. It is much more difficult to see how APL could fit in with GCSE courses or with the developing National Curriculum.

Was the course a success? On the personal level, certainly. The teachers were encouraged to join in the discussion by the informal approach adopted by our tutors, not hesitating to interrupt each other to amplify a point of interest or correct an obscurity. Will significant numbers of schoolchildren have an opportunity to experiment with APL in the foreseeable future? I think this is likely, but without the efforts of I-APL there would be none at all. I am sure all the participants would join me in thanking the organisers and tutors, the British APL Association and those working for I-APL. If we gained nothing else from the course we have seen something of the power of APL and can understand why the APL community feels that the language deserves a wider audience.

APL at Mill Hill School

by Bruce Dickson (Mill Hill School, London NW7)

Computer Education at Mill Hill is split into Information Technology (wordprocessing, spreadsheets, comms, DTP, etc.) which is taught on an MS-DOS system, and "Computing" in which we are free to write our own syllabus, so we are including this year a couple of lessons introducing APL to illustrate its potential.

We have just installed a class set of Archimedes 310 computers and were given a pre-release version of I-APL to try out. The pupils are in our first three years, 14 to 16 years old, doing courses of about 26 lessons p.a. Only one attempt has been made as yet to use APL for project purposes - the teacher involved is not an APL user, and is unwilling to launch pupils into an area where he cannot easily guide them.

Following this note is the worksheet we are about to use. It was trialled briefly with the computer club and a couple of end-of-term classes last year. It is submitted for publication as a stimulus rather than an end-product, and criticism, suggested amendments and extensions would be welcome. You may, gentle reader, feel inspired to redo the whole thing; if so please send the writer of these notes a copy!

An APL Pilot Course

Put in the APL disc and call up APL with the command *APL. The message CLEAR WS tells us that there is an empty workspace that can be filled with functions and variables.

Check that the Caps Lock light is on. Then hold SHIFT and touch 'I'; the squiggle is called iota; it affects the single number that follows it. Enter 1100 and 11000 and any other values you care to try.

Holding SHIFT and pressing 'R' produces the greek letter ρ . It can be used to find how many numbers there are in a list of numbers like the ones you have just created. Try this with: $\rho 110$ $\rho 1000$ $2 \rho 1100$ $\rho 1\ 4\ 8$ $\rho 100\ 100$ $\rho 153$.

These APL commands give quick results that are printed on the next line. Some results do not fit onto one line: try

```
2 5\r10      (THE 2 5 RESHAPE OF 10)
2 5\r110     (THE 2 5 RESHAPE OF IOTA 10)
20 1\r120    (THE 20 1 RESHAPE OF IOTA 20)
```

To save typing time, parts of commands can be copied with the COPY and arrow keys; the whole of the last command can be repeated by using the right bracket ')'.

Now try 2 5 8p150 4 5 8p1100

Note that if reshape runs out of information to use, it just starts at the beginning again as often as necessary to fill the pattern.

Try 2 3 4 20p1480

You must be sick of p by now.

The left arrow + is called "gets". Print it with SHIFT 'P' Now enter ...

PAGE2+3 4 20p1480

Nothing happens until you enter PAGE again. Try typing PAGE and the following commands:

PAGE+2
PAGE+100
PAGE-100

(Note the high minus used to display negative numbers.)

PAGE1+PAGE+5
PAGE2+PAGE-5
PAGE1+PAGE2
pPAGE
20 18pPAGE
p20 18pPAGE (THE SHAPE OF THE 20 18 RESHAPE OF PAGE).

Exercise

Experiment with different numbers all through this course. If you have stuck with us so far, you will make a good APL programmer!. All that it takes is the familiarity that comes with practice.

A First Database

Make a list of five surnames and enter them as:

LIST1+'SMITH JACKSON DICKSON DUCKWORTH GRAHAM'

Check with LIST1 that the spelling is O.K. Improve it to LIST2 by putting enough spaces in between the names so that there are 12 characters for each name, i.e. 'SMITH' must have 7 spaces before 'JACKSON' starts, which is then followed by 5 spaces and so on.

Now try

```
5 12pLIST2 (THE 5 12 RESHAPE OF LIST2)
```

If all is well, you should get

```
SMITH
JACKSON
DICKSON
DUCKWORTH
GRAHAM
```

against the left margin. If not, make LIST2 again counting spaces carefully.

Make this into LIST3 with

```
LIST3+5 12pLIST2
```

The expression 5 1p15 makes a column of numbers from 1 to 5. Before we can make a numbered list, we have to change this column of numbers to a column of characters. This is done with the 'format' symbol '7' obtained using SHIFT and 'H'. Type in

```
(75 1p15),LIST3
```

To put spaces between the numbers and names, enter

```
LIST4+(75 1p15),' ',LIST3
```

and check that all is well again by typing LIST4.

Finally try ,LIST4. The comma unravels the table (matrix) of characters.

Exercises

1. Change SMITH in LIST4 to your surname. Hint: use ,LIST4, change it, call it LIST5 and then reshape it with LIST6+5 12pLIST5
2. Make a new numbered list with at least three of your friends' names.
3. (More difficult) Can you make a numbered list of your friends' surnames and first names in three columns?

N.B. To see what lists we have made so far in this workspace, enter)VARS.

I-APL NEWSLETTER No 7 October 1988

by Anthony Camacho, Old World Chairman of the I-APL project

Thank you Vector and the APL Association

The Education Vector including this newsletter is being mailed to everyone on the I-APL mailing list as well as being included in Vector. We are very grateful to Vector to Alan Sykes and to the British APL Association for doing this, as it enables I-APL to keep people informed.

The I-APL mailing list is over 1400 names and addresses and about 500 of these live outside the UK. The cost of mailing to all these people is too high to do it regularly whether there are new products or not, just to keep everyone informed. There are new things coming. We don't have them yet but we hope they will be in stock by the end of November. If you send an order it will be acknowledged at once but the goods may take until Christmas. We are hoping that the APL Association will include I-APL news regularly in Vector so that instead of starting an I-APL users magazine we can encourage all APLers to take an interest in the other uses of APL. If this can be agreed then I-APL will cease to issue newsletters and ask everyone to join the APL Association to keep in touch.

Delays!

We were hoping to be able to issue the BBC B version of I-APL in May and then we revised this to September. We still haven't issued it and now we are hoping for November 88. The problem has been (is) that we feel it would be very foolish to give people who haven't yet met APL, an I-APL product which does not give them access to the colour, graphics and sound which are such attractive features of the BBC machine. Any reviewer would find such a product unattractive. We want APL to be as attractive as it can be to those meeting it for the first time.

Those of you who have seen the Beta test version will know that the access given through the operating system to colour graphics is very slow to use because it makes one or more operating system calls per pixel. Alan Mayer is now writing a program which takes a complete array as the right argument and does a PLOT with each row of it. I am glad to give Beta test versions of I-APL/BBC to anyone with a BBC who feels able to do some machine code work and produce a workspace giving access to any BBC facility (even the joysticks!).

Version 1.1 of IAPL.DEB and I-APL/PC

Paul Chapman has finished version 1.1 of I-APL. Paul has also done a new version of the PC port. This has some extra facilities and a new workspace of machine code functions called FSCREEN. This workspace gives full screen control.

Another facility is to use the operating system to write the screen (invoke I-APL with I-APL/D). Doing this makes displays slower but allows near-compatibles or software-only compatibles to run I-APL without locking up, losing or permuting the display. Version 1.1 maintains a character buffer of the current screen and so WGET and WPUT (get or put characters into a window) are really very quick. People had written to us about many problems with their CGAs and EGAs and this option allows even the Nimbus PC to run I-APL. The new version also drives an EGA properly.

All Known Bugs Cured

Porters are encouraged to ask me for the IAPL.DEB file for version 1.1; it cures all the known bugs in version 1.0. There were only a few, including these: the result from a scalar left argument to encode was reversed; there was a fault with)PCOPY; the results from tangents of over ninety degrees sometimes had the wrong sign; and the result from domino of negative one was negated.

Availability

Version 1.1 of I-APL/PC will be available (disk only) in December from the BAA software library. You cannot get the books from them. The software library doesn't handle any paper. The address to write to is:

The BAA Software Library,
c/o Alison Chatterton,
9 Oak Grove,
Hertford, SG13 8AT.

Ask for I-APL version 1.1 (and while you are about it why not order Morten Kromberg's GRAF, the FinnAPL statistics and Zdenek V Jizbas tutorial disks at the same time). The price is £2 a disk for APL Association members and £3 for non-members plus £1 for post and packing. I-APL Ltd will continue to sell the books on their own, and disks when a manual is ordered at the same time.

We have sold out of manuals at present and anyone who ordered recently should have had an apology and a promise to send the goods as soon as a reprint is ready. If by any chance you haven't it is probably my fault. Please forgive me and bully me as gently as you can manage.

The address for orders is:

I-APL,
56 The Crescent, Milton,
Weston-Super-Mare,
Avon BS22 8DU.

My address is: 2 Blenheim Road, St Albans, Herts AL1 4NR. The reprint of the manual will also cover Tony Cheal's BBC version (even if we cannot yet issue it).

Ports to Other Machines

Many porters are getting on with the work. We are promised news of the version for the Sinclair Spectrum (48K and up) by Christmas. Monsieur Emmanuel Roche is doing a port to the Amstrad word processor CP/M machines. The lack of progress on the Apple II and Commodore 64 ports is worrying. Paul says it would take a competent programmer, who knows both the operating system and machine code thoroughly, between a week and a fortnight to do a good port.

Porting Help is Badly Needed

Such abilities are like gold dust (at least if they are combined with the time and the sort of altruism needed to work on this project) and if anyone knows where they are to be found we would be delighted to hear. A port started now and done by (or over) Christmas for the Apple II or Commodore 64 or Atari ST or Macintosh or any other common machine could bring the author kudos, a warm feeling and the thanks of thousands. We will pay out of pocket expenses. If you know of a potential porter and think you could keep them encouraged to a successful conclusion - ring me up. I have a stack of ready prepared porter's kits.

Workspaces

Apart from the workspaces on the original issue disk and Paul's FSCREEN mentioned above, we now have utilities from FinnAPL, and on the same disk is a nice collection of statistics workspaces which Tauno Ylinen tells me were contributed by Professor Smillie of the University of Alberta. There are also workspaces for graph drawing from Morten Kromberg and a tutorial from Zdenek V. Jizba. These are available in the BAA software library.

We would like other workspaces, especially if they are of value to teachers in their teaching or their administrative work; we are promised one from APL $\diamond$ 385 which does schoolmaster's ranking and many other pretty things with registers of marks. It is on Beta test in a Yorkshire school at the moment.

Books

Sales so far have been excellent. Norman Thomson and Linda Alvord's "Tutorial" has led the way with sales of not far from a thousand so far. The manual has sold out - the print run was 600. The Encyclopedia of APL has sold between six and seven hundred. The I-APL board decided that we would pay royalties on any material we publish from now on. We have been offered several things already. "APL for the social sciences" by Tama Traberman, Linda Alvord's next book. Something perhaps from Howard Peelle (I believe Ed Cherlin has a draft).

The second edition of the Tutorial is in camera copy form waiting for the first edition to run out - it will not be long now, because the Tutorial has been a great success and ordered world-wide. Garry Helzer is working on the second edition of "An APL encyclopedia".

We think there is a demand for pamphlets making APL attractive to workers in particular disciplines - about the size of "APL for Teachers" or "APL for Scientists and Engineers" by K. E. Iverson. Could you write such a pamphlet? Do please let us know.

International Efforts

There are groups supporting the I-APL initiative all over the world. Scandinavia holds the most successful groups. The Danish group produced and issued Kim Andreassen's port for the Piccoline - the first for a school machine - and now we have Morten Kromberg's GRAF workspace. The Swedish and Finnish groups have also done great work.

The Germans and German-speaking Swiss are collaborating with Dr Andreas Geyer-Schulz on translation of the material into German. They are disappointed by the delays to the Commodore 64 version which is the commonest school machine there. In Holland Herman van Loon and his group have distributed many copies.

The Australians have Ian Shannon's port to the microbee and Prof Douglas has sent a disk of statistical functions which is being prepared for issue by Jake Ansell. Both Alain Delmotte and Gerard Langlet have translated the Tutorial into French. I knew about Alain's work but was surprised by receiving the Langlet version which may be published by the Belgian APL Association (BACUS). We have educational distributors in Kenya (John Stevens) and South Africa - the republic - (John Corrie).

Counting in APL

by Alan Sykes

Introduction

Since APL is an array-oriented language, not surprisingly it is very good at counting. I don't mean by that producing the natural numbers 1 2 3 and so on, though of course the expression (for example) 16 produces a list of the numbers 1 to 6. No, I mean counting incidences of occurrence of numbers or names in a list. In this first article of the new form of Education Vector, I shall try to introduce newcomers to APL to some of the techniques of counting that are really useful when it comes to applications. At the same time, I hope to include one or two 'goodies' namely useful functions whose coding may be a little complicated to the APL beginner, but nevertheless are easy to use effectively.

Throughout what follows, lines indented by five spaces correspond to APL statements that can be executed at the keyboard, with the following line(s) denoting the response.

Simple Counting Techniques

Two lists (vectors) will be useful in this and following sections.

```
NOS←2 1 2 3 2 4 1 5 6 4
STR←'ACDBCDEABBA'
```

First we review the idea of comparisons and Boolean values (0 or 1). In APL the statement $2=3$ has a numerical value of 0, and the statement $3=3$ has a numerical value of 1, corresponding to the first statement being false, and the second true. Similarly the statement $X=2$ is not an assignment statement as it would be in BASIC or FORTRAN, but results in one or more values of 1 or 0 depending on whether the individual numbers in X are equal to 2 or not. For example, try $NOS=2$ or equivalently

```
2=NOS
1 0 1 0 1 0 0 0 0 0
'A'=STR
1 0 0 0 0 0 0 1 0 0 1
```

Hence if we want to know how many 2's there are in NOS we simply have to add up the elements of the result of $2=NOS$, which we do using the expression $+/ \dots$

```
+/2=NOS
3
+/'A'=STR
3
```

Now, being ambitious, we wish to count not only the number of 2's in NOS, but also the number of 1's, 3's and so on. Obviously this is possible simply by using five similar statements, with 2 replaced by 1 3 4 5 and 6. But, as you might suspect, there is an easier solution. The so-called 'jot-dot' construction using the two symbols 'o' and '.' together allows the arithmetic operation that follows to be applied to all possible pairs of numbers, one from the left argument and one from the right:

```

      1 2 o . = 1 2 1
1 0 1
0 1 0

```

(Notice how the resulting array has as many rows as the left argument and as many columns as the right.) Hence ...

```

      1 2 3 4 5 6 o . = NOS
0 1 0 0 0 0 1 0 0 0
1 0 1 0 1 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 1 0 0 0 1
0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 1 0

```

This is rather a large matrix (by 'pencil and paper' standards) but its row sums give the desired counts or frequencies. Recalling that 1 2 3 4 5 6 can be generated using l6 and remembering the right to left rule, the complete set of frequencies is given by the expression:

```
+/(l6)o . = NOS 2 3 1 2 1 1
```

Producing a General Purpose Frequency Table

In Statistics, we often want to count not individual values but all values within a certain interval. For example, with NOS we may require to know how many values there are between 0 and including 2, between 2 and including 4 and so on. We can convert NOS into appropriate numbers by first subtracting 0, dividing by 2 and then rounding up to the nearest integer, converting ...

```

      2 1 2 3 2 4 1 5 6 4
TO 1 1 1 1 2 1 2 1 3 3 2

```

This is achieved easily using $\lceil (NOS-0) \div 2 \rceil$. Now the ordinary counting techniques of section 2 can be applied:

```
+/(l f / T) o . = T + f (NOS-0) ÷ 2 6 3 2
```

Two points are worth noting about this. The right-most occurrence of $\lceil$ is executed first and rounds values of its argument up to the nearest integer. The next use of $\lceil$ is in the

expression $\lceil T$ which calculates the maximum value of the values in the list T. Also we have used a temporary variable T, assigned to it and then operated with T later (to the left!). This is a frequent nicety that experienced programmers in APL slip into, but there is nothing wrong in first assigning to T, and then executing the rest of the line up to the second occurrence of T. Indeed as a general rule always approach a new APL statement by taking each meaningful bit of it, working from right to left, and seeing what the result is - only then is it sometimes possible to understand what's really going on.

Back to our frequency table. It would be nice to output the frequencies 6 3 2 together with the mid-interval values associated with those frequencies. These values are calculated using:

```
(0-2÷2)+2×⌈T 1 3 5
```

These two results can be formed into a column matrix with the first column equal to the mid-interval values, and the second equal to the frequencies using:

```
1 3 5 , [1.5] 6 3 2
1 6
3 3
5 2
```

So now we aren't too far from piecing bits of APL together to make a really useful function. Before we do so, we have to decide on the SYNTAX of that function, i.e. its name and what its arguments are. We will call it FREQTAB. This function needs two arguments, one to specify the name of the list of numbers it operates on - the right argument, and a second one telling us where to start our intervals, and how wide they are to be. So I propose that the above result of investigating NOS should be executed using the expression

```
0 2 FREQTAB NOS
1 6
3 3
5 2
```

Users of I-APL have two methods of constructing this function. The direct definition method uses right argument ω and left argument α and care must be taken to construct the rather long statement:

```
FREQTAB:(( $\alpha[1]-\alpha[2]\div 2$ )+ $\alpha[2]\times M$ ),[1.5]+/(M+⌈T/ω)∘.=ω*⌈(ω- $\alpha[1]$ )÷ $\alpha[2]$ 
```

The alternative to this mouthful, is to use the ordinary APL function editor and type short meaningful lines, as follows:

```

      VTAB←A FREQTAB W;F;V;T;M
[1] T←(W-A[1])÷A[2]
[2] M←1/T
[3] F←+/M°. =T
[4] V←(A[1]-A[2]÷2)+A[2]×M
[5] TAB←V,[1.5]F
      ▽

```

(N.B., the variables F,V,T, and M are only required internally and so are made LOCAL variables to the function FREQTAB by inserting them, separated by semi-colons after the function name and right argument.)

As an example of its use, create 25 numbers at random between 0 and 1 and count how many there are in intervals of width .25. First, to construct the numbers ?25p10000 produces 25 numbers chosen at random from 1 to 10000, so dividing the result by 10000 should suffice.

```

      R←(?25p10000)÷10000
      □←F←0 .25 FREQTAB R
0.125  5
0.375 10
0.625  4
0.875  6

```

Displaying FREQTAB using a BAR CHART - hence HISTOGRAMS!

Producing a bar chart from a frequency table such as the above is not an easy task for a newcomer to APL, but when explained clearly, the APL solution is quite straightforward - and again the power of thinking in terms of arrays is emphasized.

To illustrate the argument, consider the frequency column of F, consisting of the numbers 5 10 4 and 6, with maximum frequency 10. The bars corresponding to these frequencies must be of length 5,10,4,6 - or they could all be of length 10 as long as the last 5,10,6 and 4 characters respectively in each bar consist of blank characters! That's the first clue. Secondly, the expression

```

      5 10 4 6°.≥110
1 1 1 1 1 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
1 1 1 1 0 0 0 0 0 0
1 1 1 1 1 1 0 0 0 0

```

yields a pattern of 1's and 0's corresponding to that required. All we have to do is to replace the 1's by a character such as '□', the so-called 'quad', and the 0's by a blank. Normally we work in index origin 1 (the default setting) and so we can index the two element vector '□' using the boolean array plus 1.

```

      ' [1+5 10 4 6°.≥110]
00000
0000000000
0000
000000

```

Now all that is required is to glue to the left-hand side of this array, a separator, say '|', preceded by the first column of the frequency table as an array of characters. The resulting function is called BAR and takes as its right argument a frequency table.

```
BAR:({Fω[; ,1]),'|',' [1+ω[;2]°.≥1[F/ω[;2]]
```

or equivalently

```

VB+BAR F
[1] B+({F[; ,1]),'|',' [1+F[;2]°.≥1[F/F[;2]]
V

```

Let's try the results on some simulated dice values.

```

THROW1←?50p6
THROW2←?50p6
THROW3←?50p6
AV←(THROW1+THROW2+THROW3)÷3

```

```
.5 1 FREQTAB THROW1
```

```

1 10
2 7
3 6
4 8
5 10
6 9

```

```
BAR .5 1 FREQTAB THROW2
```

```

1|0000000000
2|00000000
3|0000000
4|00000000
5|0000000000
6|000000000

```

```
BAR 1.5 1 FREQTAB THROW1+THROW2
```

```

2|0000
3|0000
4|000000
5|0000
6|0000
7|000000
8|0000000000
9|000
10|0000
11|00
12|000

```

```

          BAR .5 1 FREQTAB AV
1 | 
2 | 
3 | 
4 | 
5 | 
6 | 

```

As these simple examples illustrate, FREQTAB and BAR are powerful tools for investigating data. In fact pursuing examples like the ones above are quite sufficient to convince newcomers to Statistics of the validity of one of the most important results of Statistics - the so-called Central Limit Theorem, which for example, justifies the empirical observation that the values of AV (the average of three separate tosses) tend to be more frequent near the mean value of 3.5 than at the extremes. (This is not true of course for just one throw.)

In practice, FREQTAB and BAR are used so often together that it is worth combining them to give HIST, as follows:

```
HIST: BAR α FREQTAB ω
```

or ...

```

      VH+L HIST DATA
[1] H+BAR L FREQTAB DATA
      V

```

so that statements such as 0.5 1 HIST THROW1 are now meaningful.

Conclusion

Throughout the presentation here, I have made use of the essential counting technique of summing the rows of a boolean matrix constructed by using 'o . ='. This is by far the most elegant way of achieving the results required, and is particularly handy for small-scale problems. However, for larger sets of data, the method does become rather inefficient. In the next issue of Education Vector we will discuss alternative methods, and present an interesting case study where its application is most effective.

News from Sustaining Members

Compiled by Cathy Dargue

APL People Ltd

The agency business continues to flourish, with an increase in both consultancy work and placements. Things are certainly looking very healthy at the moment for those seeking a job move, with new vacancies coming up all the time and not enough people to fill them! The range varies tremendously, from openings for junior staff to prestigious new opportunities in the city for highly experienced APLers (offering city rates too!).

The new office premises are working out very well and we have had to take on another member of staff to cope with the ever-increasing workload.

In addition, we have set up a new division called 'IN-FOCUS', to handle consultancy work in the fourth generation language, FOCUS, and already have a number of FOCUS contract programmers available.

With 1992 fast approaching, our next aim is to concentrate on Europe and the States. We have of course, already done a certain amount of business abroad, both finding people jobs overseas and now with our first contract outside the UK, and we intend to build on this over the forthcoming year or so.

Dyadic

Dyadic is pleased to announce a new screen manager for Dyalog APL. The new screen manager is designed to allow developers of APL applications to provide a state-of-the-art interface for their users. It provides facilities for pop-up/pull-down menus, context-sensitive help, spreadsheets, validation, scrolling and even animation.

The screen manager is implemented using a new system variable `□SM` (Screen Map) and a system function `□SR` (Screen Read). `□SM` is a nested matrix which defines a form: each row in `□SM` represents a field; each column an attribute such as field contents, position, window size, colour and so on. Field contents can be character or numeric arrays (or even another form) which scroll vertically and horizontally in the window defined for them. When the contents of a field are scrolled all other fields in the same vertical and/or horizontal scrolling group are scrolled in parallel.

Whenever you assign a value to `□SM`, the change is immediately reflected on the screen. Thus you can dynamically change contents, position, colour and appearance of fields to produce animated effects. The system function `□SR` allows the user to interact with the

screen, and edit the contents of the fields. Changes in the data are directly reflected in QSM.

Pop-up menus and help screens can be implemented simply by localising QSM in a suitable function. The local definition of QSM is overlaid on top of whatever is on the screen at the time, and automatically disappears when the function exits.

Dyadic believes that the use of a system variable to represent the screen is a powerful and natural extension to APL. The new screen manager is currently on Beta Test and is expected to be on general release in January 1989.

HMW Computing Ltd

HMW Computing Limited have been working on "4XTRA" our front end Foreign Exchange Dealing and Position Keeping System adding facilities to enable overall management analysis across dealing positions as well improving our graphical displays of trading data.

HMW Computing Limited have also launched a new employment service for the APL community 'HMW EMPLOYMENT LTD.' This complements the services we currently offer.

The directors of the new company, Valerie Lusmore and Stan Wilkinson have both been involved with APL since the early 70's and can offer an unequalled service to people looking for APL personnel as well as APL technicians looking for a career change or even contract work.

In line with HMW's philosophy we will also be using the employment business to offer a service for other areas in computing such as Oracle, Focus, JSP Cobol etc.

To find out what we can offer, please call Stan or Val on 01-353-4212 or write to us at the address on the back cover. If you would prefer to phone out of office hours Stan's home number is 01-286-7068 and Val's is 07618-727.

Cocking and Drury Ltd

Several new consultants have joined us in recent months, bringing with them more experience of APL systems in practical business situations. Rossy Norman, Peter Merritt and Christian Jensen have joined as Consultants, and Clive Hibberd as a Senior Consultant.

Interest in the APL*PLUS Mainframe interpreter and Compiler software continues. The advanced features, efficiency and compatibility with the PC, Unix and Dec VAX counterparts is encouraging its use as the strategic mainframe APL for the future.

The User Command Processor feature of release 6 is proving popular with customers. It allows libraries of programs to be accessed as if they were always part of the workspace, giving productivity and efficiency benefits when utilities are standardised and centralised.

Performance trials of the new APL*PLUS II system for 80386 machines (pronounced 'two-system') are proving very successful; APL*PLUS PC features show speedups of between 1.5 and 30 times, with intensive mainframe applications being matched and even outperformed by the II system.

The II system imposes no artificial limits on array or workspace sizes, so you can go as high as the installed memory; 15 Megabyte workspaces are now a reality. The second-generation productivity features such as nested arrays, generalised operators and the multi-object ring-editor all proved very popular at the recent APL-ication conference.

Only Cocking and Drury include one year's free upgrades and support in the APL*PLUS II system purchase price.

Demand for training courses has increased recently, with in-house APL2 and System Design courses being especially popular.

APL Product Guide

Compiled by Cathy Dargue

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to:

Cathy Dargue,
60 Downhall Ley,
BUNTINGFORD,
Herts SG9 9TL.
Tel: 0895-31313 x3038

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	APL Coprocessor	3,500+	32-bit coprocessor board for IBM PC. NS32000 cpu with FPP, up to 4Mb RAM, 16Mb virtual memory. Software includes Unix V.2, Dyalog APL, graphics support, DOS interface. Provides multi-user Unix/DOS environment.
	IBM 6150	15,000+	Multi-user Dyalog APL system with Fast 32-bit RISC processor, FPP, up to 8Mb RAM, 210Mb Disk, 16 users. Interface to SQL, graphics and APL support for standard IBM peripherals.
	Altos 3068	25,000+	Multi-user Dyalog APL system with MC68020 cpu & MC68881 FPP. Also features a LAN which supports IBM PCs as Dyalog APL terminals.
	Sun 3	15,000+	Multi-user Dyalog APL systems which can be configured as a network of workstations and or a traditional time-sharing cpu. With its 25MHZ 68020 cpu, the Sun 3/200 is the fastest APL microcomputer on the market.
Inner Product	IBM PC	2,000-6,000	IBM PCs supplied for turnkey applications
M.B.T.	MBT Series 10	poa	UNIX/68010 based multi-user APL system
	TORCH	poa	68000/Z80 multiprocessor
MetaTechnics	-	poa	Details on application -- IBM PC compatible
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/35 Mb disc, 12 ports.
	Spectrum/386	2,000+	80386-based PC to run APL*PLUS II. Fully expandable.
	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	Dyalog APL	1,000-8,000	See Dyadic Systems entry
Cocking/Drury	APL*PLUS/PC Rel 7	475	STSC's full featured APL for IBM PC, PC/AT and PS/2. Now with virtual workspace, quad-MF, GSS graphics, Network and VGA support.
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS UNX	poa	STSC's 2nd gen hi-performance APL for IBM PC/RT, Sun, DEC, HP and major Unix computers and workstations.
	APL*PLUS VMS	poa	STSC's 2nd generation APL for DEC VAX computers running under VMS. Comparable in features to APL*PLUS II system, APL*PLUS Unx, and APL*PLUS mainframe.
	APL*PLUS mainframe	poa	STSC's 2nd gen APL interpreter. Enhances VS APL with component files, nested arrays, formatting, user command processor, fastfns and other productivity features. For VM/CMS, MVS/TSO: offers simple upgrade from VS APL.
Dyadic	Dyalog APL	795-10,000	2nd gen. APL for UNIX systems, e.g. IBM 6150, Sun, Vax, NCR, HP9000, AT&T, Altos, Apollo, Whitechapel, Sperry, etc.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.

IBM UK	IBM PC APL	poa	Event-handling & APs for full-screen I/O disks, diskettes, asynch. comms.
IBM USA	IBM PC APL2	500	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. USA distribution only - call your IBM representative for more details.
Inner Product	VIZ::APL	250-350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyalog Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
Mercia	APL*PLUS/PC Rel 7	475	STSC's full-feature APL for IBM PC, and compatibles. No 64K object size limit.
	Upgrades 5 to 7	325	
	Upgrades 6 to 7	145	
	APL*PLUS/UNIX	poa	Interpreter for UNIX systems: WICAT, CADMUS, CALLAN, FORTUNE 32:16, HP, 9000/500, OLIVETTI 382, SUN etc.
	APL*PLUS/MAC	350	APL for the Macintosh. Big workspaces, big objects, mouse, icons etc
MetaTechnics	APL*PLUS Rel 6	475	Discount on quantity.
MicroAPL	APL68000	1,000-2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL68000 Apple Macintosh	87	
	APL68000 Apple MacII	260	
	APL68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL68000 for Atari ST	87	
	APL68000 for IBM-PC	995	
	APL*PLUS/PC - REL 7	425	
Portable	PortAPL	\$195	IBM PC Software
		\$275	Macintosh
		\$2,995	DEC VAX
I.P. Sharp	APL-LAB/PC + Q pack	480	STSC APL plus object packaging for the PC
	Upgrade V6 to V7	120	
	Upgrade any to V7	200	
	RTS	70	Closed version of APL-LAB/PC
	Sharp APL/PCX V1	1,650	For IBM XT/AT
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	" diskettes only	10	
	" manuals only	30	
		1,000+	For IBM mainframes
	Sharp APL/PC	325	For IBM PC or PC/XT
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers.
	PortaAPL	280	PORTABLE SOFTWARE's APL for APPLE MACINTOSH.
		2,995	PORTABLE SOFTWARE's APL for the DEC VAX.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyalog	Lynwood j300	1,560	Monochrome ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	Lynwood j500	2,295	Colour ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.

	IBM 3163	791	Low-cost Monochrome APL vdu. Supports downloaded Dyalog APL font.
	IBM 3164	1,093	Low-cost Colour APL vdu. Supports downloaded Dyalog APL font.
Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 15/8if5 anti-reex screen, low profile keyboard
	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata	400	Second-hand Elite 3045A
M.B.T.	various		Contact MBT for details
Meta Technics	IBM EGA compatible	299	Emulates EGA & Hercules, Half Card
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS2010	1,215	ANSI3.64 DEC VT52/100/220 compatible. 15" tilt/swivel screen, low profile keyboard 8 page memory, windows, viewports, 80/132 columns, full overstrike, 2 or 3 comms, ports, 55 PF keys, NVM storage.
	HDS2010G/GX	1495+	As above plus Tektronix 4014, Retrographics VT540/D0640 and Visual 500 compatible. 1024 x 390 or 1024 x 780 resolution.
Tektronix	4114B	13,500+	19" D.V.S.T.:Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024); Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax interface).
Dyadic	IBM 4201 Proprinter	poa	100, 200, 40(nlq) cps, matrix printer, with graphics. Supports downloaded Dyalog APL font.
	Toshiba P351	poa	24 pin high-quality matrix printer 100 cps letter quality, 192 cps draft.
Inner Product	Epson FX80	500	Soft char. set, 180 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	APL-385	50(PC), 125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities

APL Software Ltd (mainframe)	AFM/AP	11,035	Interprocess Software for VM/CMS & MVS/TSO.
	- Keyed Access	2,650	Component File Management System (VSAPL/APL2)
	- Interactive Link	1,325	
	- Mail Exchange	2,650	
	CALL/AP	4,030	Non-APL program execution (VSAPL/APL2)
	APLPRINT	2,205	Output to high speed line printer or 328x devices (VSAPL/APL2)
	ENHANCED FORMAT	2,205	Extends Format operator to full "Quad-FMT" status (VSAPL/APL2)
	ISP	750	Input and Output Stack Processors for manipulating terminal I/O
	OSP	2,205	with facilities for Error Trapping (VSAPL)
	DISPLAY CAPTURE	poa	Allows terminal output to be collected and held for retrieval by an APL function (APL2)
	UCF	poa	User Communication Facility for data transfer between users (APL2)
	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL*PLUS/PC
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Convert non-APL files to APL files
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRSII, LOTUS123, VISICALC, MULTIPLAN, DIF files
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(microcomputer)	STATGRAPHICS Ref 2.6	645	Integrated Statistics and graphics on IBM PC, AT, PS/2 and compatibles
	Update 2 to 2.6	140	
	Update 1 to 2.6	330	
	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, file documentor, exception handling and games.
	IRMA Module	120	327x IRMA support.

	Fin & Stat. Library	350	Financial & statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
E&S	PROTOPAK		Packages for prototyping management information systems - consisting of: PC & mainframe modules
	RMS		Relational databases.
	AMS		Multi-dimensional arrays
	RAMS		Combined RMS & AMS.
	BMS		Dynamic nancial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP124-based)
	CMS		Communications package.
	SOS	poa	Scheduled ordering and stock control.
FASTCODE	FASTCODE, MONITOR	\$239	Assembler written monitor for APL applications in APL*PLUS/PC
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Ofsu3suc System
	HASLEMERE	poa	Hotel Accounting System
Mercia	STATGRAPHICS 2.6	635	Integrated stat. graphic system for PCs.
	Upgrade 1.0 to 2.6	295	
	Upgrade 2.0 to 2.6	145	
	EXEC*U*STAT	395	Easy to use Statistics for management.
	APL*PLUS tools		
	- VOL 1	225	IBM PC Utilities:IRMA3270 comms, full screen, RAM Disk report generator
	- VOL 2	125	File documentation, screen editing. Exception handling.
	FINANCIAL AND STATISTICAL LIB.	325	Financial and Statistical analysis
	APL Spreadsheet	195	APL spreadsheet - links to popular spreadsheet software.
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC, Forecasting, Inventory Control, Scheduling, Distribution, etc
MetaTechnics	MetaScreen	99	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes: MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor.
	APL-IEEE488	99	Controls IEEE488/GPIB Bus from APL*PLUS/PC.
	PLOT/PC	99	2D & 3D Graphics package. Includes interactive diagram Editors.
	Browse	99	Scrolling of DOS files, large APL variables.
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.

MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product
	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
	STATGRAPHICS Rel 2	535	
Parallax	ExecuCalc	\$5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.
	ExecuPlot	\$5,000	Mainframe-based colour graphics with micro links.
I.P. Sharp	ACT	poa	Actuarial system
	APS	poa	Financial Modelling
	BOXJENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	Record maintenance/reporting
	MAGIC	poa	Time series analysis/reporting
	MAGICSTORE	poa	N-dimensional database system
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	65	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	MICROCOM	poa	Mainframe to micro link
	Q PACK V.7	45	Object packaging for APL-LAB/PC
	Q PACK upgrade	10	Upgrade v.6 to v.7
	Q PACK RTS	10	Run time version
	SAGA	poa	General graphics, most devices
	SIFT	poa	Forecasting system
	SNAP	poa	Project management
	SUPERPLOT	poa	Business graphics
	VIEWPOINT	poa	4GL - Info centre product
	XTABS	poa	Survey Analysis
Sugar Mill	Stat 1	\$129.95	Statistical toolbox, menu driven
Uniware (mainframe)	STSC's ENHANCEMENTS	10,715	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	10,715	Component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	5,715	
	FILEPRINT	5,715	
	FILESORT	5,715	
	FILECONVERT	5,715	
	FILEMANAGER (EMMA)	5,715	STSC's database package.
	APL*PLUS COMPILER	21,430	First APL compiler. Complements APL*PLUS enhancements and Sharefile under VM/CMS and MVS/TSO.

(microcomputer)	EXECUCALC	3,895	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL (VM or TSO).
	STATGRAPHICS	725	Statistics & Graphics for PCs.
	STATGRAPHICS FCA	140	An add-on module to STATGRAPHICS: Factorial Correspondence Analysis.
	APL*PLUS/PC TOOLS		
	- VOL1	325	Incl. 327 x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
	- VOL2	125	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	250	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
	APL*PLUS/PC FIN. & STAT.LIBRARY	350	Collection of financial and statistical utilities.
	POCKET APL	140	Smaller version of APL*PLUS/PC.
	UNIASM	275	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	240	APL*PLUS/PC spreadsheet-like data entry and validation system.
	The APL DEBUGGER	105	First released APL*PLUS/PC debugger.
	OVERLAYS	250	Fast assembler routines to handle overlays in APL*PLUS/PC.
	R:BRIDGE	380	Interface between APL*PLUS/PC & R:BASE 5000.
	DMA	380	A version of EMMA (APL database manager) for APL*PLUS/PC users.
Warwick University	APL2C	295	Interface between APL*PLUS/PC and DATALIGHT C language.
	ADAPTA/DLS	33,333	Production & purchasing scheduling for process manufacturing.
	BATS	500	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to 35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Consulting	Consultancy	180-550	Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation.
APL People	Consultancy	from 120	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL68000, C/Unix, TSO/MVS, VM/CMS, graphics. Additional skills available: OR, statistics, financial, commerce, retail, engineering.
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry. Designing, programming and implementing applications of O.R. and statistics.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	120-150	Junior consultant
		140-200	Consultant
		185-300	Senior consultant
		275-400	Managing consultant
Peter Cyriax	Consultancy	100-150	Junior Consultant
		120-200	Consultant

		160-300	Senior Consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Inner Product	Consultancy	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy	poa	
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MetaTechnics	Consultancy	poa	Management Information & Production. Engineering, APL - C/Assembler custom programming
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Uniware	Consultancy	call	Junior to managing consultancy in APL.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
Mercia	ALLCARD	495+	Memory management unit, allowing 952K under DOS - extra 312K APL*PLUS/PC workspace.
	MULTI-APL	195+	Multi-task / Multi-user / Network APL*PLUS/PC with file locking, etc.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Consulting	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
	Tim Perry	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT. Tel: 01-577 3541
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT Tel: 01-577 3541
APL Software Technology	John Hagger	14 Rosewood Avenue, Alveston, Bristol BS12 2PP Tel: 0454415737
	Per Hultin	46 Vicarage Road, South Benfleet, Essex SS7 1PB Tel: 0374550501
Beta-plan APS	Kim Andreasen	Stengrade 75, DK-3000 Helsingør, Denmark. Tel: 45 2 21 48 48

Buckland Management Systems	John Buckland	Westwood, 9 Grove Road, Camberley, Surrey, GU15 2DN Tel: 0276 684327
Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ Tel: 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 60130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE16 Tel: 01436 9481
		155 Friar Street Reading RG1 1HE. Tel: 0734-588835
Datatrade Ltd.	Tony Checkseid	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. Tel: 0937-61961
	Roger Attard	Davenport House, Bowers Way, Harpenden, Herts. Tel: 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. Tel:(914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. Tel: 01-864 9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 01-353 4212
IBM UK Ltd	Chris Sell	PO Box 32, Alencon Link, Basingstoke, Hants. RG21 1EJ. Tel: 0256-56144
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. Tel: 01-873 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge. Middx. UB8 2UD.Tel: 0895-59826
Mercia Software Ltd.	Gareth Brantnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MetaTechnics Systems	John Stenbridge	Unit 216, 62 Triton Road, London, SE21 8DE. Tel: 01-670 7959
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. Tel: 212-475-4001
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0860-377963 (Mobile)
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138. Tel: 617-547-2918
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0276-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd., Rochester NY 14620 Tel:716-461-3169
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. Tel: 02407-2027
I.P.S.A. Ltd.	David Weatherby	10 Dean Farrar Street, London SW1. Tel: 01-222 7033
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kike Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. Tel: 06284-6000
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France Tel: (1) 45-27-20-61 Telex: 648348F UNIWARE
Warwick Univ.	Dr Andy Pole	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel:0203-523066 x2158

New Product Announcements

IBM APL2/PC (including 32-bit APL and free runtime)

This is an almost complete APL2 interpreter for the PC. The product was announced in the UK on 14th November 1988, and will be available early in 1989.

It includes a 32-bit version of the interpreter for use on 386-based machines, and the 16-bit version has the option of a 'packager' which produces a .EXE file from your APL workspace which you may freely distribute.

Auxiliary processors are included for full-screen management (AP124 style), file handling, communications and so on.

The interpreter was demonstrated at APL-ication, both in 16-bit and 32-bit versions. The price is £348.

STSC APL*PLUS II

A full 32-bit rewrite of APL*PLUS/PC with nested arrays as per the mainframe product. The cost is approximately £2,000 ... see the entry for Cocking and Drury in the Product Guide.

Adelphi APL to Oracle Interface for APL*PLUS/PC

Tibor Horvath (Arlington Associates) announces Adelphi to provide PC users access to an SQL database from APL. It connects APL*PLUS/PC to the Oracle relational database. The product consists of a compiled routine which links APL to Oracle, and APL functions for interacting with the database. SQL statements are passed as arguments to these functions, and data is transferred between the database and the workspace using one APL variable for each field.

Adelphi requires APL*PLUS/PC version 3.0 or higher, Oracle version 5.1 or 5.1A, IBM PC with 640K, plus 896K extended memory, and a fixed disk.

It is available from August 31st 1988, at a cost of \$195. Contact:

Arlington Associates
3700 Mount Diablo Blvd., Suite 300
Lafayette, CA 94549
(415) 283-5613

**"APL as a Tool of Thought V"
Proceedings of the 5th Annual
Professional Development Seminar 24/10/1987**

reviewed by Derek Wilson

This is published by NY/SIGAPL. It is available from Renaissance Data Systems, 660 Amsterdam Avenue - Suite 124, New York 10023, USA. The price is 15 US dollars; overseas orders should add 10% to cover postage.

The papers cover a wide variety of topics:

Programming the Connection Machine - Michael J.A. Berry
APL is a Tool of Actuarial Thought - Joe Heise
A Fractal Experimenter's Toolkit - David M. Law
I-APL Tutorial on Mathematical Tools for Interactive Use - Dr Linda Alvord
APL2 and Deductive Logic - David Macklin
The Yorktown Logic Language - Norman Brenner
Rule-Based Inferencing Functions - Jonathon Bard
Statistics with APL - Arthur Castel
Database Structures in APL - Christopher Lett
A Bit of Accounting - Ed Shaw

The theme of the conference was how APL thinking can lend understanding to a variety of fields, and browsing through the papers I found three which are of particular interest. This is not to denigrate the others, I am sure that any other APLer will find a number of articles of interest.

The first paper gives a particularly optimistic foretaste of the future of APL. It deals with the appropriateness of APL to the new generation of highly parallel computers, epitomised by the Connection Machine. This has tens of thousands of processors which all perform the same instructions on differential data elements, i.e. it is a data-parallel machine. To utilise this processing power, new language constructs have been developed for languages such as Fortran and C, whereas APL lends itself quite naturally to this hardware architecture, and indeed the designers of the Connection Machine languages have borrowed heavily from APL. To quote the conclusion:

"The marketplace has shown a distinct preference for languages which fit hardware architecture well over languages which describe algorithms well. There is no reason we should not be able to have both."

The other two papers I found specially interesting were those of Heise and Shaw which illustrate how APL can be used to express concepts in Actuarial Science and Accounting respectively. The conciseness and legibility of the APL expressions for concepts such as

STATGRAPHICS

reviewed (for an APLer by an APLer)
by Gavin Schwarzenbach

As an Operational Research Scientist using APL to build solutions to business problems, I often find myself needing statistical algorithms and fancy graphs. My need is for a set of relevant functions, either to use as a toolkit, or to build into a system for a user. With that in mind I came to review Statgraphics to see how well it could meet this need, rather than to analyse the statistical validity of the algorithms. The size of the package means that I have only looked at certain areas of it in detail, and cannot comment on the usefulness of all of the functions.

Introduction

Statgraphics is a statistical and statistical graphics package, written in APL for a PC. The areas covered are best shown by looking at the main options menu.

STATGRAPHICS Statistical Graphics System

DATA MANAGEMENT AND SYSTEM UTILITIES

- A. Data Management
- B. System Environment
- C. Report Writer and Graphics Replay
- D. Graphics Attributes

TIME SERIES PROCEDURES

- L. Forecasting
- M. Quality Control
- N. Smoothing
- O. Time Series Analysis

PLOTTING AND DESCRIPTIVE STATISTICS

- E. Plotting Functions
- F. Descriptive Methods
- G. Estimation and Testing
- H. Distribution Functions
- I. Exploratory Data Analysis

ADVANCED PROCEDURES

- P. Categorical Data Analysis
- Q. Multivariate Methods
- R. Nonparametric Methods
- S. Sampling
- T. Experimental Design

ANOVA AND REGRESSION ANALYSIS

- J. Analysis of Variance
- K. Regression Analysis

MATHEMATICAL AND USER PROCEDURES

- U. Mathematical Functions
- V. Supplementary Operations

Use cursor keys to highlight desired section. Then press ENTER.

```
1Help  2Edit  3Savscr 4Prtsr 5      6Go    7Vars  8Cmd   9Device 10Quit
INPUT  11/ 4/88  17:00 STATGRAPHICS Vers. 2.6      Display
```

Fig1: screen copy of Statgraphics main menu

Most users would be unaware of its being written in APL, apart from having to read mathematical expressions from right to left, which is perhaps natural only to those acquainted with APL.

Installation was fairly straightforward, with the user being clearly guided through the joys of setting up drivers and editing config.sys files. There are two methods of running Statgraphics: either straight from DOS or, more useful to an APLer, as a workspace within APL*PLUS. Running Statgraphics from APL*PLUS on a Toshiba portable caused problems on exiting from the graphics, which was due to the use of the soft CGA character set (E=1024) for APL*PLUS. It was however easy to change the colours of the Statgraphics menus and highlighting, in order to improve readability on a plasma display.

The user-interface is good, with easy and quick access to familiar routines, but also with good help facilities for the less familiar. Graphs are easily produced, labelled and then printed or saved, as shown below.

Surface Plotting Options

Top title:

(2 lines)

X-Axis Title: X

Y-Axis Title: Y

Z-Axis Title: Z

Plot type: Surface

For surface plots:

X-axis resolution: 100

X-axis skip factor: 1

Y-axis resolution: 100

Y-axis skip factor: 1

Lines parallel to: Both axes

Hidden line removal: None

For contour plot:

Type: Lines

Contour levels:

or

Number of Divisions: 10 Minimum: 1.10241

Maximum: 5

Complete input fields and press F6.

1Help	2Edit	3Savscr	4Prtscr	5	6Go	7Vars	8Cmd	9Device	10Qui
INPUT	11/ 4/88	17:48	STATGRAPHICS Vers. 2.6			Display		SURFAC	

Fig2: screen copy of graph label

Response Surface Plotting

X-axis minimum: -0.5

Y-axis minimum: -0.5

X-axis maximum: 0.5

Y-axis maximum: 0.5

Number of x-axis intervals: 20

Number of y-axis intervals: 20

Function type: User function

Function: $1/0.2 + (\text{SQRT}((X*X)+(Y*Y)))$

Complete input fields and press F6.
 1Help 2Edit 3Savscr 4Prtsr 5 6Go 7Vars 8Cmd 9Device 10Quit
 INPUT 11/ 4/88 17:58 STATGRAPHICS Vers. 2.6 Display SURFACE

Fig3: setup options

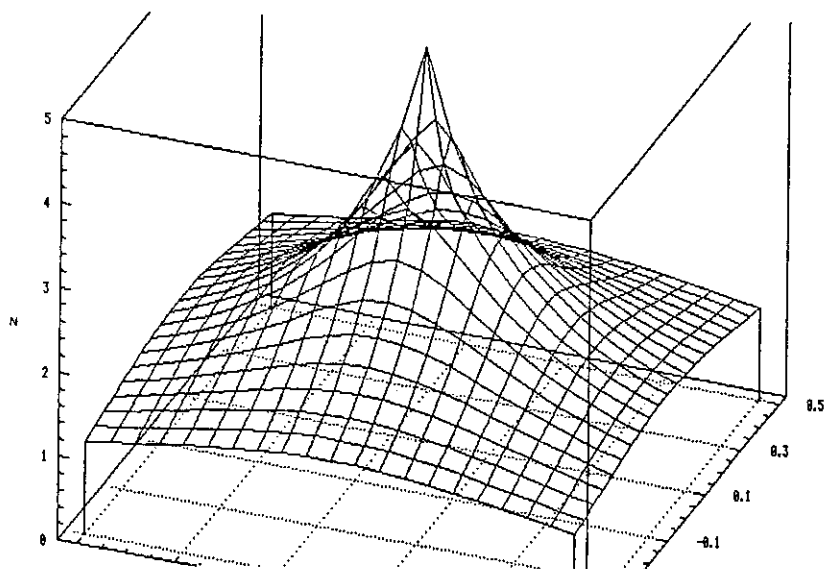


Fig4: Failed attempt at a Mexican Hat!

I didn't find the need to use the documentation very often, mainly because I didn't get lost. The documentation is organised into sections according to the main menu screen, making it relatively quick to look up problems.

Analysis of Experimental Data

Having accumulated a fairly large amount of data from a 'Taguchi' experimental design, this seemed an excellent opportunity to put the package through its paces. At this point, certain annoying features came to light which would probably only be noticed by APLers. The first is that when inputting an expression within the package, one has to type English equivalents for common or garden APL expressions such as TAKE, COUNT for iota, COMPRESS etc. The other more important fault is the lack of active memory when running from APL*PLUS rather than from DOS. I found that it was very easy to crash the system trying to perform fairly normal tasks like trying to edit 6 variables that were 216 long, or to perform multiple regression on three of the above variables.

It was quick and easy to explore the data and summarise, through simple and 'box and whisker' plots, as one would expect from a statistical package. However, fitting a multiple regression model to the data was nowhere near as good as the University of Kent package "UREG" which I was also using. UREG points out potential problems with the data, suggests transformations that could improve the fit, and generally is more informative than just producing a model as Statgraphics does. Anyone buying Statgraphics to perform a lot of regression analysis, or more complex analysis, could be disappointed.

Statgraphics was also useful for quickly producing response surface plots of the results, although it would have been useful to superimpose the actual data points against the surface.

Of more Interest to an APLer

I set out to investigate the possibilities of building a toolkit of statistical and graphical functions, that could be used by others as a toolkit, or merged together to form part of a larger system. Being written in APL gives Statgraphics advantages in this respect, but more importantly the system design enhances these possibilities. When running from APL*PLUS one can 'drop out' into APL, and then restart the package, thus enabling data manipulation in a familiar environment.

There are however more significant advantages to running Statgraphics from APL*PLUS than just being able to manipulate data. One can write functions using the "building blocks" that Statgraphics uses, which is an excellent way to build fairly complex routines quickly. Most of the functions used by Statgraphics are documented,

making them very easy to use. Once a function has been written, it can then be incorporated into Statgraphics and accessed from the menus, just like any other function

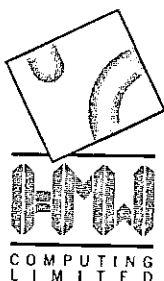
If the environment provided by Statgraphics is not suitable for any reason, the Statgraphics functions can be used in another workspace, enabling users to build systems tailored to their needs. Of course you do need a Statgraphics runtime license before you give such a system to anyone else!

Summary

Statgraphics provides a fairly broad, but shallow approach to Statistics, which should meet most people's needs. To an APLer there is quite a lot of scope for enhancing it, and for using selected functions as part of a customised statistical package.

Availability

Statgraphics is available from Cocking and Drury, or from Mercia Software. Please check the VECTOR Product Guide for the latest prices. Both the above vendors run regular Statgraphics courses, again you will find the dates in the VECTOR Course Diary.



SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:- deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212

RENAISSANCE DATA SYSTEMS
BOOK DIVISION

*** Enlightenment Thru Information Processing ***
1989 Catalog Excerpts (as of January 1, 1989)
+++ All APL Books in Print +++

	Quantity	Total
Important for the Professional APL Programmer:	Price	Ordered Cost
=====		
THE APL IDIOM LIST, Perlis and Rugaber - Funded by Mobil. Written at Yale. Very terse, but includes nuggets of gold. 1977, 50 p.	\$ 6.50.	.
APL - A DESIGN HANDBOOK FOR COMMERCIAL SYSTEMS, Smith - Readable, technical, thorough. Includes a small collection of good idioms. 1982, 180 p.	\$44.00.	.
APL - ADVANCED TECHNIQUES AND UTILITIES, Bergquist. Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques, financial utilities. Assumes knowledge of the basics. Many idioms. 1986, 450 p.	\$44.95.	.
Diskette for above. Contains all functions described in text. Format is for STSC APL*PLUS/PC but is uploadable to Sharp APL, VS/APL, APL2.	\$15.00.	.
APL HANDBOOK OF TECHNIQUES, IBM - Utilities. Many comments. 1978, 109 p.	\$18.00.	.
A Bit of APL History:		.
=====		
~~~~~ New Printing! ~~~~~		
A PROGRAMMING LANGUAGE, Iverson - The book that started it all. A computer science classic. APL's roots. Reprinted by RDS on acid free paper in a high quality, soft cover edition. Only 25 copies available! 1962, 286 p.	\$39.95.	.
A SOURCE BOOK IN APL, Falkoff & Iverson - By the famous duo. A must for serious APLers. The whys and the wherefores. Includes "Notation as a Tool of Thought," Iverson's 1979 ACM Turing Award Lecture. 1981, 139 p.	\$15.00.	.
So you want to learn APL?		.
=====		
APL2 AT A GLANCE, Brown, Pakin, Polivka. Solid, unimimidating, introduction to APL2. Clear illustrations, modern exercises in each chapter. Gets you started. Three sample applications. 1988, 444 p.	\$26.00.	.
APL - AN INTRODUCTION, Peelle - Especially for the computer-shy. 1986, 459 p.	\$29.95.	.
----- Proceedings from APL Conferences -----		
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL. They cover a wide range of topics in business and education.		.
1987 - Logic, insurance, statistics, A.I., accounting, fractals, linked lists.		.
... 1984, 1985, 1986, 1987, 120 to 150 pages each	each \$15.00.	.
1983-1987 as a package.	\$55.00.	.

Sub-total: \$

Postage and handling - U.S. and Canada (\$2.50 each item over \$25. \$2.50 minimum.):

Postage and handling - International (please add 15% surface mail, 50% airmail):

NYC Residents please add 8 1/4% sales tax. NY State, 4%:

Total: \$

Please send complete catalog: _____

Date: _____

Name: _____

Street: _____

City, etc.: _____

Please make check or money order payable in \$US and send with this form to:

Renaissance Data Systems  
P.O. Box 20023  
Park West Finance Station  
New York, New York 10025-1510  
U.S.A.  
(212) 864-3078

Postal

Code: _____

Country: _____

**RECENT MEETINGS:**

**Workshop on APL in  
Artificial Intelligence**

**Syracuse University, New York  
August 16th-20th 1988**

**APL-ication: the Practical State-of-the-Art  
APL Conference**

**28th-30th September 1988 at the University of Kent**

Copies of the conference proceedings are available from the British APL Association at the following address:

British APL Association  
c/o Special Interest Groups Liaison,  
The British Computer Society  
13 Mansfield Street  
LONDON W1M 0BP

Price to British APL Association Members: £12.00  
Price to non Members: £15.00

Please send your cheque with the order (payable to "British APL Association")  
or quote your Mastercharge or Visa number.

## **Appropriate use of APL for Artificial Intelligence**

### **Notes on APL in AI Workshop at Syracuse University**

by John Adams

I have been working on Expert Systems projects in British Airways for the last 2 years using a mixture of PROLOG, C, and PC-based shells. Since my previous background was as an APL programmer I have kept a close watch on whether APL has anything to offer the AI world. This article is intended as a summary of my opinions in the light of the recent conference workshop "APL Techniques in Expert Systems".

The amount of AI work taking place in the APL community is now pretty large - the conference material overflows a large lever-arch file such that the total library of APL/AI papers now covers 3 volumes. However most of this work is still research with only 5 systems that I would call successful. Nevertheless the conference has further convinced me that APL could be a good choice for working in the following areas:

- Uncertainty applications such as risk analysis can benefit from APL's arithmetic capabilities. Most research has concentrated on how to produce a variety of fuzzy logic functions [8][9]. An interesting alternative is Hagamen's approach in MEDCAT where 'conclusions' are only fired when the combined effect of the 'conditions' rises above a certain threshold [2][3].
- Natural language. Hagamen has demonstrated that APL can produce effective natural language input and output provided the discourse is kept within a limited specialism [3].
- Neural networks. These learn to recognise patterns in complex data and may have applications in computer vision, speech recognition, and forecasting. APL seems a natural choice since these models work by manipulating numeric arrays thousands of times over. Parallel processors are eagerly awaited by neural net researchers - a suitable interpreter would be able to use APL's inherent parallelism without any need to change the source code.
- Embedded intelligence. APL has proven itself as an excellent tool for MIS, DSS, and OR building. The use of APL could eliminate the need for data extractions, file downloads, or complex memory linking in applications where the 'intelligent' part must be well integrated into a larger whole. The benefits could be particularly marked in an application requiring complex data manipulations or hefty number crunching. This is IBM's reason for using APL in the LMS system at Burlington [4].

### Rule-based systems

For embedding intelligence into new systems or adding intelligence to existing systems the simplest technique, if appropriate, is to represent expert knowledge as rules. Rule-based systems are fairly straightforward to build in APL. Rules would be typed-in via a full-screen interface using an English-like syntax and converted into an internal format. The two most promising internal formats are: (a) a list of rules each of which would be an executable APL statement or (b) a set of Boolean tables relating conclusions to rules, rules to conditions, conditions to objects etc. The first approach is easier to program but the second should be more efficient [5].

### Success Stories

RESEDA.....Interestingly this system dates back to at least 1979 and was written in old-fashioned VS APL. It was developed under Zarri at the French Centre National de la Recherche Scientifique and is used by historians for querying a database of biographical information. The query obtains not only information which was directly introduced into the database but also implicit information found using inferences. The historian's inferencing rules take the form:

<general observation> BECAUSE <possible explanation>

For example (in natural language): "One might take a particular attitude in an argument because one has close links with one of the parties in a conflicting situation." For further details on the (rather complicated) methodology behind RESEDA please see the reference [1].

MEDCAT.....Bill Hagamen is a neuro-physiologist at Cornell University Medical College with some particularly clever innovations for computer aided teaching systems. MEDCAT is a MYCIN-like system aimed at sharpening the diagnostic skills of medical students. It has 4 modes of working: diagnosis, consultation, teaching, and learning.

The diagnostic mode uses forward chaining adding up numbers received from stimulus nodes and firing conclusions when they rise above a certain threshold. The consultation mode uses backward chaining to explain conclusions. The teaching mode runs in a stepwise fashion through the forward chainer but instead of showing conclusions it asks the student for their diagnosis: if the student's logic is inconsistent or incomplete it asks them some probing questions until any misunderstandings are resolved. In learn mode MEDCAT is able to learn from experts, incorporating knowledge about a new or wrong diagnosis. To make all this even better the dialogue is via natural language with responses given in a matter of seconds [2][3].

CATS..... is another Hagamen teaching system but this time for anatomy. It shares many features with MEDCAT but looks more like a 'semantic net'; there are no threshold values to the inferencing mechanism. The natural language interface is quite clever. Synonyms and spelling mistakes are accepted as input but responses are always given using the strict anatomical terminology. If the question contains an ambiguous word the system tries to look at the context before asking for clarification. Questions are not isolated but depend on what has gone before; for example pronouns refer to the last mentioned object of an appropriate type to fit the verb. The question "Why?" does not restate the facts but examines the knowledge base to infer 'rules of thumb' which might explain things in a more general sense [3].

?..... developed by Stephen Jaffe at Mobil Research to help geologists determine the existence of Submarine Fans and hence oil. It has the same feel as a diagnostic expert system built using a PC-based shell. Rules are held internally as executable lines of APL such as 35 IF (NOT P 33) AND (OR/P 30 31 32) AND (10 < D 6)

LMS..... Logistics Management System developed by IBM at the Burlington manufacturing facility and reckoned to save IBM several million dollars every year. LMS is described as "an implementation of continuous flow manufacturing principles using the power of knowledge based expert systems and drawing on the fields of industrial and manufacturing systems engineering, computer networking, window interfaces, data engineering, operations research, statistics, and rule, object, and function based programming paradigms. ...LMS is now a critical component in running major areas of the manufacturing facility, and critical for future successful plant operation"[4]. Implementation details of LMS are not given but the chief architect for LMS has written several papers on coding the traditional AI paradigms in APL [5][6][7].

## Conclusion

Products purpose-built for AI work will always be better than APL at the tasks they were designed for. Neuron Data's 'Nexpert Object' will beat APL for frame-based object hierarchies, Intelligent Environment's 'Crystal' will beat it for rule-based programming, and PROLOG will beat it hands-down for logic programming. APL has something to offer the AI purist where the purpose-built tools are at an early stage of development. Uncertain systems, natural language, and neural nets are examples of these. Reproducing the functionality of existing AI tools in APL will sometimes be justified where a hybrid MIS/DSS/OR system requires a relatively small amount of embedded intelligence.

## References

1. Using APL in an AI Environment: a Knowledge Engineering System Dealing with Biographical Data. G Lee, R LeLouche, V Meissonnier, G Zarri  
APL 82 conference proceedings
2. Medcat: an APL Program for Medical Diagnosis, Consultation, and Teaching  
W Hagamen, M Gardy, G Bell, E Rekosh, S Zatz  
APL 85 conference proceedings
3. Associative Semantic Processing. W Hagamen, P Berry, K Iverson, J Weber  
APL Techniques in Expert Systems (SigAPL 1988)
4. Logistics Management System: Continuous Flow Manufacturing using Knowledge Based Expert System (KBES). G Sullivan, K Fordyce  
APL Techniques in Expert Systems (SigAPL 1988)
5. An Efficient Algorithm for Rule Based Production Systems using Boolean Arrays and Functions as an Alternative to Tree Structured Networks  
K Fordyce, G Sullivan  
APL Techniques in Expert Systems (SigAPL 1988)
6. Object Programming: a Brief Review of What it is and Why it is Useful  
K Fordyce, G Sullivan  
APL Techniques in Expert Systems (SigAPL 1988)
7. APL and Expert Systems. J Brown, E Eusebi, K Fordyce, G Sullivan  
APL Techniques in Expert Systems (SigAPL 1988)
8. Mapless and Uncertainty. D Bonyun, G Jones  
APL Techniques in Expert Systems (SigAPL 1988)
9. Fuzzy Rule Based Expert Systems. A Geyer-Schulz  
APL Techniques in Expert Systems (SigAPL 1988)

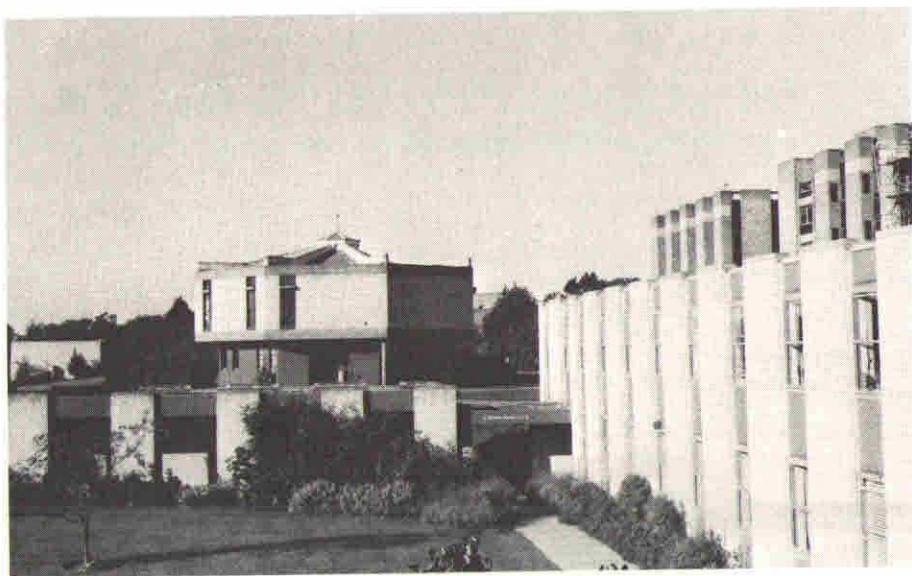
## APL-ication: the Practical State-of-the-Art APL Conference

notes mainly by Adrian Smith

### Acknowledgements

I would like to thank the committee of the British APL Association for paying my conference fee. Having been lucky enough to visit Sydney for APL88, I was in no position to ask Rowntrees (or is it Nestles?!) for another outing at company expense.

The notes on the papers are (as always) highly selective. Please write us a letter if you feel misrepresented, or send us a summary if you have been missed out altogether.



The Campus at Kent University  
on the first morning of APL-ication

Just after I returned home, I had a call from my father to say they were on their way back from the continent, and were staying over in Canterbury for the night. On hearing of my recent sojourn in the University, his first question was "how long did it take you to find your room?" Some years ago, he too had the experience of the "E2/N7" geography of Rutherford Hall, and the sight of several hundred bewildered clerics wandering the stairways late at night had clearly remained with him!

## **Conference Highlights**

by Adrian Smith

### **Michael Wheatley on the History of the APL Environment**

A lot of really good hard 'inside information' on the early development of APL, and how it fitted in to its environment. The rationale behind the development of DNA and the shared variable processors.

### **John Scholes on the Dyalog Session Manager**

A really good idea, well implemented and well demonstrated. Thoroughly enjoyable.

### **The APL*PLUS II Product Forum**

Again, lots of good hard information from the man who knows more about it than most. Invaluable for anyone thinking of moving to 32-bit APL.

### **Hands on IBM's 32-bit APL2/PC**

A product destined to put the wind up STSC. A real joy to play with; I can't wait to get my hands on a review copy.

## Extending the Domain of APL

by Michael Wheatley (IBM Santa Teresa)

Michael spoke from a long experience of APL development, his main interest having been the linking of APL to the external world. In the beginning, APL was its own operating system, and even in OS:

"It was a characteristic of those early systems that OS would gradually grind down, while APL kept running. We'd poke around in the control blocks for a while, and kick one of the users off, and the system would come back up again!"

I suppose this was the sort of thing that gave APL a bad name, but at least it made it popular with the system programmers!

Michael commented that the 'user-interface' of early APL had developed far more incrementally than the 'core' of the language. It was an ad hoc mix of )cmds, 'magic' locked functions (SETIO), and dyadic I-beams, which did things like shutting down APL!

By 1969, file systems had crept in, again via locked functions. APL now ran on DOS/360 and then on OS/360. It still considered itself in control of the machine; APL got all the interrupts first, then passed the unwanted ones to DOS!! The remainder of the history is better known:

- 1971..... APLSV (IBM internal use). System variables and shared variables.  
Formalized access to the environment.
- 1974ish..... APL360 handed over the machine to CMS! More functions were  
included to access the environment .... quad-TS
- 1982..... APL2. Various implementations had been running since 1976.
- 1985..... APL2 with quad-NA. Greatly broadens the range of facilities; a  
'coming of age'.
- 1988..... now you can call APL things from outside APL, e.g. PL/1 can access  
arrays created within the APL environment.
- ???..... co-operative processing. "I think that we have all the necessary  
linguistic constructs now!"

Michael continued with a detailed tabulation of all the ways APL had ever interfaced to the outside world; what these were originally designed for, what they were actually used for, what they proved good for, and what they were bad for! I shall attempt to reproduce those points which stuck most firmly in my notes/brain ....

Feature	Designed for	Used for	Good for	Bad for
)CMDS	People! No results returned!!	)COPY under applcn control	People	Interfacing to the envt.
I-beam	ad hoc facility	Exec assembler code outside APL	Sys-level debugging	
quad-IB	undoc. locked functions	e.g. turning vector on/off		
Lkd fns	providing 'side-effects', without really understanding them!	'magic functions'		Need copying; name conflicts - you can't rename them.
Sys fns Sys vars	Iverson & Falkoff wouldn't give us any more symbols! "Wow, now I can have as many as I want"  "We went much overboard .... APL2:41 APL*PLUS:120	Giving control to the outside envt. Proper return when complete.  APL developer's tool	No name conflicts!  m/c dep'dt things like quad-ARBIN.	Name pollution. No consistency across APLs  Didn't allow users or installations to add their own facilities.
Shared Variables you	Asynch. interface to outside world.  Very much simplified with the use of nested arrays.	File systems  Synchronous processes, with full quad-SVC	Formal descr of APL\360  Extremely useful general comms facility. Asynch access to SQL data "Has that dumbo over there finished my query yet?"	"The management of multiple pairs of variables drives nuts, & is a wonderful source of error"  Costly: high path-length, much data movement etc. Over-engineered for the commonest use in synch processes.
Quad-NA	"I'm sure this one is right, because I thought of it!"  ---- Synchronous interface ----		Resolves name conflicts	High performance, not high useability.

In summary ....

Horses .....Courses ....

Primitives.....should be totally machine-independent. OK for use when there is one well-defined way of doing something. □ is a classic example of a bad primitive!

□fns/vars.....ad hoc linguistic facilities, like □IO. A path to other functions, like □SVO. Bad for system-dependent things like files.

- NA ..... magic locked functions, but user-definable. Handy for implementors who want to experiment with new ideas without polluting the language. Give access to existing subroutine libraries in other languages.
- SVO ..... asynchronous comms. and distributed processing. These should work between machines as well as from user-to-user on the same machine.

## A Better Mouse-trap

by Richard Nabavi (MicroAPL)

Richard has more experience than most of implementing APL interpreters in the Mac environment. His demonstration of APL.68000 on a 4Mbyte Mac2 was most impressive: some of his comments were ...

"The APL application has to rise to the challenge of using these interfaces. The APL interpreter needs to use them also."

"A programming language has to be able to control the environment as well as run under it. This is a problem for APL, because the application has to run simultaneously with the 'compiler and editor' that created it."

The freedom that the Mac environment gives is often a problem for applications ... you might need to stop the user resizing the active window if you had just set up a screen panel within it!

Normally you would expect the screen-handling to be a function of the application. In Mac-APL it all works the other way round: APL is called as a subroutine of the operating system event-loop, which is continually checking for mouse actions and so on. The odd thing is how little help it gives to the application (in this case the APL interpreter). Tricky operations like writing output to half-hidden windows must all be dealt with by the interpreter. Of course all your application does is to output text to the 'screen' as normal .... Richard and his pals have already done all the hard bit.

## The Dyalog APL Screen Manager

by John Scholes and Peter Donnelly

This was ostensibly a Product Forum, but turned into a substantial tutorial on screen management with nested arrays, and included some exceedingly silly ideas, such as screen panels which 'bounced' around the screen (and off each other) as you attempted to type your name. It culminated in a very workable spreadsheet, implemented almost entirely in a single nested screen definition.

Some long time ago I wrote a book about APL design, and those of you lucky enough to own a copy can look up Chapter-9 (page 90), where you will find:

"For too long, APL has been a multidimensional notation constrained by a one-dimensional view of the world. Even now, life is not perfect: why can't we just treat quad as the screen?"

```
□[2+13;10+160]+HEADING
DATA+□[10+110;10+120]
```

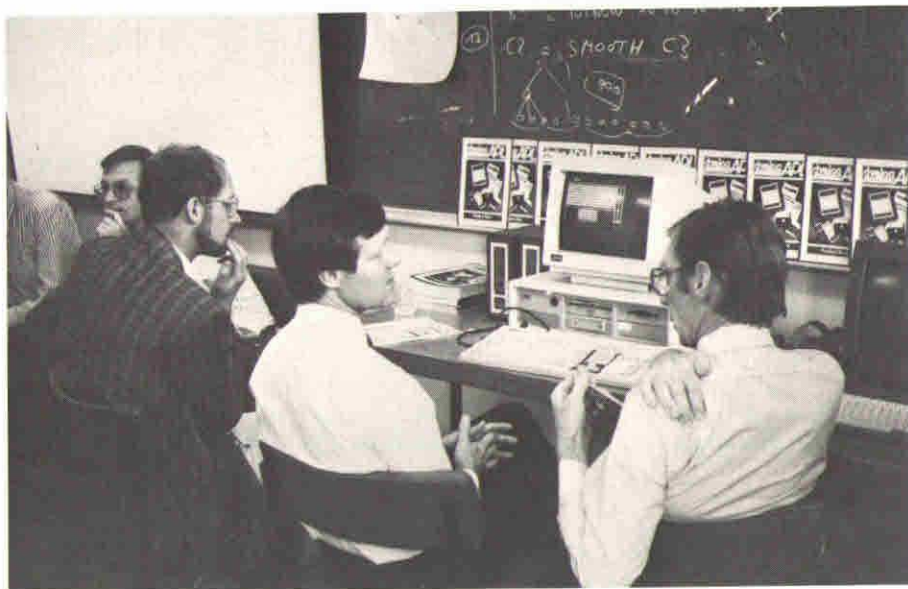
... the damn' thing looks like a screen after all! One day ...?"

That was written in 1981; at last, Dyalog have come up with a screen-management system which gives the user this kind of power and simplicity!

Essentially, the screen is reflected in a single nested array called `□SM`. This can be assigned to, referenced and (very important) localised just like any other system variable. Each row in `□SM` represents a rectangular field on the screen (a bit like the GDDM panel definition in AP126, or `□WIN` in APL*PLUS/PC); each column (of course most of these have defaults) defines a different attribute. The great advantage of the nested representation is that column-1 can be a text matrix of field contents for text fields, a numeric matrix for numeric fields, and so on.

When you assign to the array (or catenate a new field to the end) the content of `□SM` is immediately reflected on the screen. The user can then scribble all over the input bits, and when he exits, the auxiliary variable `□SR` holds the usual information about the exit key, modified fields, cursor location (again as per GDDM).

The great value of localising `□SM` is that you can do basic windowing, pop-up help and so on trivially easily. As long as your help function mentions `□SM` in the header it can define part of the screen as a new panel, manipulate it as it wishes, and as soon as it exits the original screen is fully restored with no action at all from the programmer! Wonderful!!



Peter Donnelly expounding the virtues of the  
Dyalog Screen Manager to Tim Perry

Only one quibble, which I took up with John Scholes afterwards, and which I still feel is wrong. Because the change to  $\square$ SM is reflected instantly on the physical screen, if you want to build up a screen over several lines of code (say repeated catenates to the end) you need to do this to a local work variable, only assigning to  $\square$ SM at the last minute. It might have been better to reflect the changes to the physical screen only on the call to  $\square$ SR (as per GDDM when you don't see anything until you issue an ASREAD with CTLS+101). However I accept that the fancy animation effects (making a panel scoot round the screen) would be less effective.

For me, the highlight of the conference. Every home should have one.

## MicroAPL: the next 5 years

by Richard Nabavi

"The Mac is a window into the future. Design for the Mac in the knowledge that the IBM world will eventually catch up!"

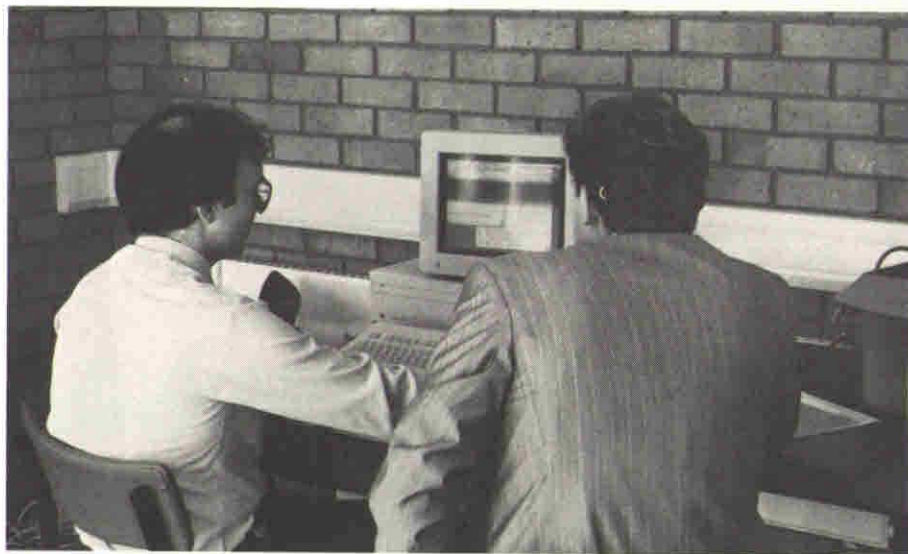
Richard saw three main strands of development ...

- the IBM PS/2 with OS/2 (Presentation Manager) in its true 386 guise (end 89)
- the Mac 6820/68881 (here and now)
- Unix-based systems (with windows)

Richard's theme was very straightforward and to the point (as Richard's themes always are). If you want to learn how to program for windows and mice, get a Mac and start now. OS/2 is a dead duck on 286 hardware:

"Microsoft have done something to OS/2 which is about the nastiest thing they could have done. You are stuck with 64K segments as OS/2 doesn't save the 386 registers. The Phar-lap extender runs 386 protected mode: Microsoft thought this might be useful .... so they stopped it!"

While we wait for the REAL OS/2 (the 386 version) we can buy a Mac2, or we can use the Phar-lap extender with APL*PLUS II.



David Eastwood Demonstrating APL.68000

**Mr HRH Prince, Charles Buckingham,  
Palace Road, The Mall, LONDON**

by David Piper (Cocking & Drury)

... or to be more accurate, the generation of decent looking name and address output from truly yukky input!

This had every right to be called an Expert System! The problem was to take free-format, upper-case name and address data (full of nasty little keying errors and incorrectspacing), analyse it for content errors, and correct automatically as many keying errors as possible.

The output was required to look smart, get the case correct on the McGuffogg's and O'Hara's and sort out little details like "The Revd. Canon C Smith MA".

Oh ... and it needed to run in a finite time on files with several millions of records! The benefits to the customer were obvious: reduced cost in eliminating dud mailshots; reduced customer irritation at incorrect salutations; better take up.

### Implementation

The system was essentially rule-driven, with user-maintained tables of standard titles 'Group Captain' and possible abbreviations for these, such as 'GRP CAPT', 'GRP CPT' etc. In this particular case there were nine different ways of abbreviating this one title; 'Squadron Leader' had 21!

Simple QSAM files were used, with fixed length blocked records. The processing order of battle ...

1. Keying errors corrected. 5 --> S etc with some degree of intelligence.
2. Tokenize data, retaining boolean pattern to record position in original data.
3. Find standard words and divide into clauses where possible
4. Search early in the clause for known lists of titles.
5. Search late in the clause for 'gongs'. Some hairy cases such as "A McTavish MC" cause problems!
6. Use a  $\Phi$  algorithm to turn tokens into token codes. At this point the variation in spellings of 'SQDR LDR' is eliminated. Note the decision to avoid dyadic 1 on a nested vector.

The resulting output could be verified as having both a title and surname, initials in a sensible place, and as not containing a specific list of 'reject' words like 'deceased'!

### Efficiency Considerations

Very large files .... hence blocked I/O on fixed length records. The absolute avoidance of .. at the record level ... processing is some 15 times faster if the looping is by clause rather than by record.

Lots of 'partition vector' operations using plain old VS APL techniques; the word and phrase search uses these in preference to a nested approach.

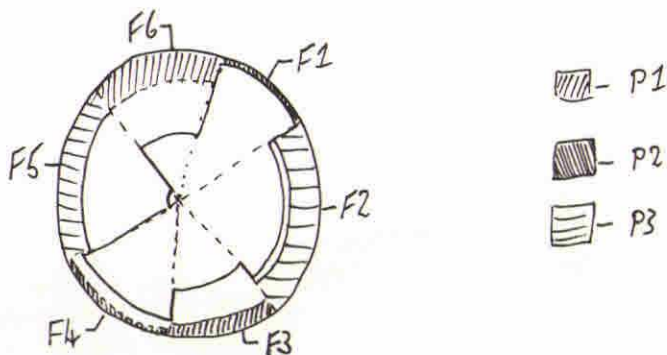
Runs in batch from a TSO CLIST, using the AP101 stack to set a return code on completion. On an IBM 3090/200 (MVS/XA) it will process 1000 records in 4.1 to 4.6 CPU sec, using a 2Mbyte workspace. Multi-million record files are processed, with a hit rate of 94 - 96%, most of the errors being untitled names.

### All Comparisons are ODIIOUS

by Rishad Talyarkhan (Product Area Manager, IBM Europe)

How do you compare a bunch of products like AS and IC/1? As all comparisons are odious, that was the name chosen for this system. It was essentially a decision analysis model, where all the attributes for the different systems code be arranged in a hierarchy and weighted to produce an overall 'strength' in a particular category.

The results were typically presented in the form of a 'radar pie' which might look a bit like this:



... apologies for the lousy drawing! The weightings determine the size of each segment, the pie being ordered by segment size. Only the outermost band was shaded in each sector, this representing the dominance of that particular product in that sector. If they were all much the same, then very little shading would show, on the other hand a product with a really strong 'unique selling proposition' would jump off the page.

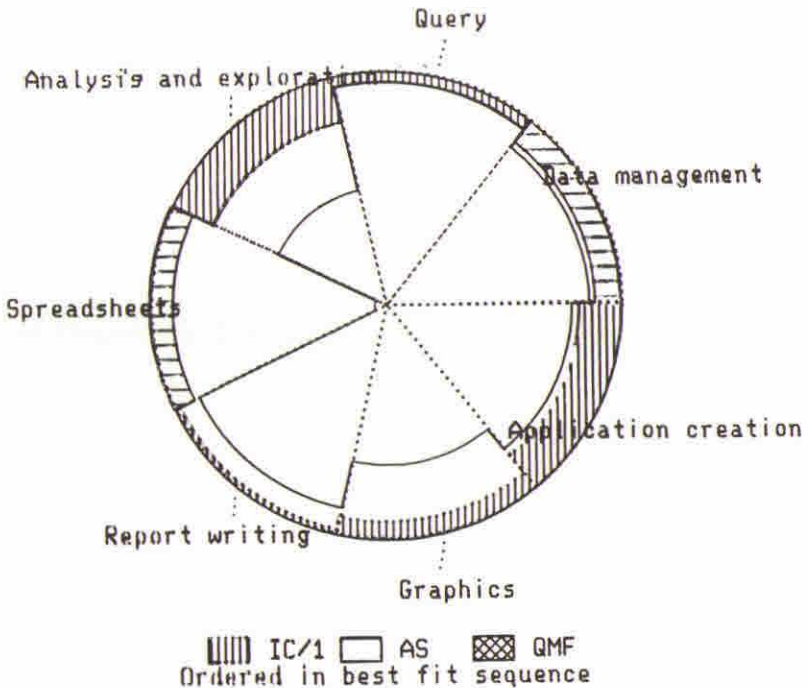
It was written in 3 - 4 man-months using Graphpak and APE.

Q..... could we have written this in AS?

RT..... No. Users will try to do it in AS, and will fail. On the other hand, the initial APL hump is too big, so people just won't try at all. AS sits somewhere in the middle; it is nowhere near as flexible as APL, and by the way, neither is IC/1

Q..... Who makes the choice between systems?

RT..... The answer is changing. 5 years ago it was DP ... the answer was QMF. Since the users couldn't get what they wanted from DP they went to PCs and applied pressure to get access to the data.



## Panel Discussion: a Market Niche for APL

Chaired by Peter Donnelly,  
with Brian Drury, Tim Perry, and Richard Nabavi

### Brian Drury

APL has done itself a great dis-service; it has singularly failed to standardise the links to the outside world. APLers tend to feel they are superior, because they are dealing with a superior product, and interpreter writers have shown a lack of commercial awareness.

There are three important question marks over APL's future ...

1. what is IBM's commitment?
2. if the APL people are so good, and if APL is so good, then why are we so few??
3. why don't successful software houses use APL???

In the future, most APL use will be concentrated on the superminis and micros. It will be successful only if it is more competitively priced. In particular the price of runtime APL currently precludes the hi-volume, lo-price market.

"I would trade many of the enhancements to APL over the last few years for one ☐ORACLE or ☐FOCUS. We are limited only by the imagination and commercial awareness of the APL developers"

### Tim Perry

Of the areas where APL makes sense, the interpreter is not a critical part of the price. What do we do about writing our APL code to make it easy to change over to C? What utilities are there to help APL to interface to data maintained by other languages?

Where is the glue to take data and get it into the format we want?

### Richard Nabavi

I would like to go back in history a few years. Most of what I'm going to say is based on my own experience of the smaller micro.

There's an implication that APL is in competition with something else ... are we saying that the competition is one thing (spreadsheets, Basic, Pascal) or a whole lot of application-specific tools? The truth is that APL has lost out in relation to 100's of alternatives!

APL is a very general-purpose tool. Which are the market niches where it really should have done well ....

- Management information. Yes
- Financial planning and budgeting. Yes
- Insurance. Yes.
- Forecasting and modelling. Yes.
- Market Research bureaux. Here is a niche perfectly matched to APL's strengths, yet no success at all. I recently met a forecast of telecoms apparatus, based on tables of installed base, growth rates etc. This is what APL is for, and they used FORTRAN!!
- Engineering and scientific. They still use HP calculators and Fortran subroutines.

What is the competition to APL? In the areas where APL has succeeded in the past, its patch is being encroached upon by spreadsheets or applications packages. The strong areas are being squeezed from both sides. All that is left to us is prototyping.

So ... I'm fairly pessimistic if we regard APL as the product we sell today. The real market opportunity was 1982-3 when the competition was MS Basic. The competition now is Quattro and Paradox and Excel! We can't just stand still ... if we were wrong in 1983 we are even more wrong now.

The competition has been putting all its effort into the things that sell packages; links and interfaces. So ... what have they got, and what have we got that they haven't?

BD ..... Lotus was a single company with a single commercial goal. Oracle has had massive marketing and promotion. We cannot hope to match that.

RN ..... The work done by ISO was extremely good, but only covered about 10% of what needed to be done. No standard covers the file systems, screen handling, printers and so on.

RN ..... Writing the package is a very small part of what a software house does - say 10% The rest is documentation, marketing and maintenance.

TP ..... I've been using a rather pathetic system called SAS which has a sort of matrix manipulation language.

- BD.....If you have 18 maintenance programmers and lose one you have 17 left. Thus you have both high visibility, and a degree of security. If you have 1 APL programmer, and lose 1 that leaves none. This could explain a lot about the average DP manager's attitude to APL.
- BD.....I would like to see APL swimming with the tide. Let's build hybrid systems, not rewrite Oracle in APL every time.
- TP.....we need a common set of tools we can use on m/c a, m/c b and so on. With these we will be able to build systems that will last.
- RN.....A lot of emphasis at this conference has been on the user interface. This must fit with the other systems on the same machine - APL must look right to the user.

## APL*PLUS II

by James Wheeler (STSC)

"Like any APL system, I am at my best when used interactively"

APL*PLUS II is aimed to fill the gap between desktop convenience and mainframe capacity. I find conventional PC APL very satisfactory for the majority of things I do; APL*PLUS II is for the serious APL user who wants a seamless all-in-one APL environment at their fingertips all the time.



Aelred Tobin explaining some of the finer points of APL*PLUS II

### Architecture

STSC have followed a two-fold thrust in APL technology ...

- the 'PC' road to a ver complete PC-oriented APL
- a portable APL*PLUS system. A very advanced interpreter for 32-bit computers. Designed for a 10-year life; currently available on Unix systems and will be used on the IBM mainframe.

They have long had the dream of bringing together the two activities into a single product. The 80386 is an ideal machine for APL ...

- 'real mode' is a very fast PC.
- 'protected mode' is a true 32-bit machine.

Hence APL*PLUS II is a two-part syatem:

- the Session Manager runs in real mode, and has the look and feel of a PC product. It controls the screen, sound, keyboard etc.

- the APL interpreter runs in protected mode using everything above the 1Mb line as free workspace. It co-operates with the session manager ... each "enter" passes a command to APL for execution. The chip can switch very rapidly between the two modes, so there is no perceptible delay.

### Features of the Interpreter

The data is held mainframe style (bitty booleans, big integers), but in the APL*PLUS/PC  $\square$ AV. The nested array implementation is the same as release-6 of the mainframe product.

The  $\square$  operator is provided to manage looping, and the interpreter was designed from the outset to support nested arrays.

There are 160 system commands and functions ...

"Adding lots of system functions is the right way to do it. We are convinced that integration direct into the interpreter is what our users want."

### Features of the Session Manager

Function editing is integrated with the APL session. The editor gives a ring of sessions, which may include several functions, DOS native files, as well as the interactive APL session. Cut and paste is allowed from anywhere to anywhere in the ring.

The session can be driven from a menu tree, or via keystrokes; the keyboard is configurable, but from a configuration file outside APL, rather than with  $\square$ POKE.

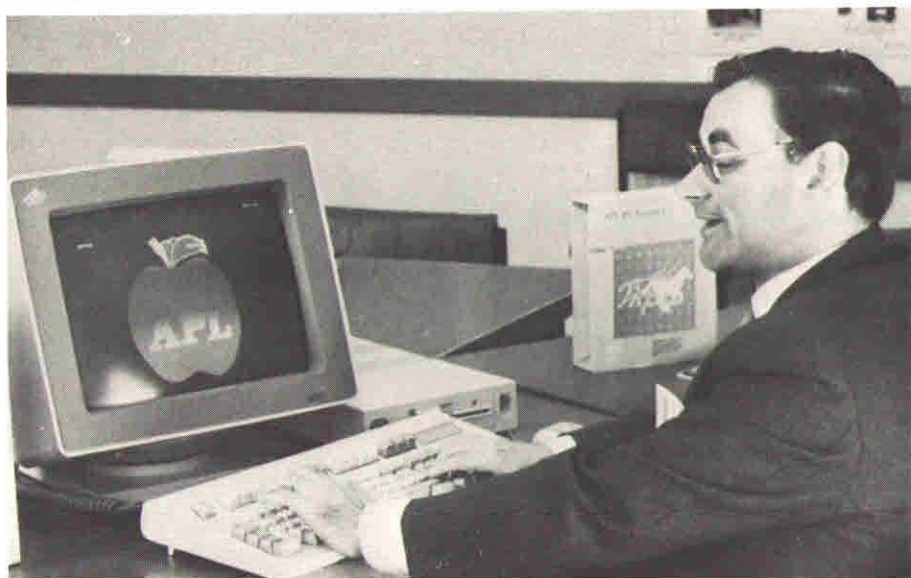
### Summary

This is a true 32-bit APL in the MS DOS environment. It uses all the available memory; it is the serious APL for professional APL users.

## IBM's APL2/PC

notes by Adrian Smith

One of the most visited rooms in the exhibition contained a solitary PS/2 model-80 running the 8514-A large monitor, and the US version of APL2/PC.



APL2/PC and the famous Apple.

I managed a mere half-hour on this very exciting APL. The 32-bit version responded happily with `⎕WA` as 9797104 (i.e. lots) and proved rather fast and extremely robust. There are APs to do pretty well everything under the sun, and the whole lot is now available for £348 (or at least it would be if IBM would actually sell you one!)

The 16-bit version includes a packager, which really does hold the promise of free runtime!

P.S. If you can get your hands on the 8514-A display, it is really lovely.

## Computers in the Boardroom

notes by David Preedy

The final session of the conference was given by David Preedy, Development Director of Metapraxis - the developers of the RESOLVE Executive Information System. David described the background to the development of RESOLVE, which is currently being used by directors of about 60 major organisations both in the U.K. and America.

In the introduction, the talk explained some of the unique challenges presented by work directly for senior executives. These included the need to design an easy-to-use system, the problems of working under little direct guidance from the ultimate user, and the over-riding importance of focusing the project on the current business needs of the directors rather than being side-tracked by the technical issues, however interesting they might be.

The talk then described some of the main issues related to the range of data coverage most appropriate to senior executives and of Metapraxis' experience that their users are looking primarily for a "what-is" rather than a "what-if" system.

The RESOLVE system itself was then shown in action, including a pre-recorded "management presentation" describing a typical monthly financial report, followed by live access to a corporate database controlled by a member of the audience using a special infra-red control system designed for use in corporate Boardrooms.

Following this presentation of an artificial corporation, the versatility of the RESOLVE approach was demonstrated by switching to the output from "Mandate" - a joint project between Metapraxis and the London Business School. This has combined certain key economic trends from the government's CSO database with the LBS econometric forecasts, presented using RESOLVE graphics. The result was a brief but enlightening summary of many of the key economic trends currently making the national headlines.

# dyalog APL

## C INTERFACE

```
#include "apl.h"
#include "support.h"

main()
{
    ARRAY *rslt, *larg, *rarg;
    *getarray(), *fre
    *idn d(), *d_idn()
    int fncode;

    startup(2);
    define("IDN_TO_DATE", RSI
    define("DATE_TO_IDN", RSI

    larg = rarg = rslt = 0;
    setjmp(errbuff);

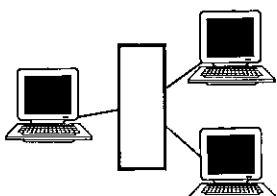
    while(1)
    {
        larg = freearray()
        rarg = freearray()
```

## UNIX INTERFACE

## CHOICE OF HARDWARE

IBM Ps/2 Model 80  
Compaq 386  
Altos 3068  
Apollo  
AT&T 382  
Bull SP57, SP59  
Dec Vax  
Hewlett-Packard HP9000  
IBM 6150  
Masscomp  
NCR Tower 32  
Pyramid  
Sun  
Unisys Series 5000

## MULTI-USER



## FILE SYSTEMS

## SQL INTERFACE

## EDITORS

```
[0] Z←DISPLAY Z;
[1] DIO←0 ⋄ SZ←x
[2] Z←' '(⊖FMT
[3] NSS:→(1 ⋄DZ
[4] NOR:Z←(1⌈pZ)⌈p
[5] →(1=DZ)/SMP
[6] Z←' ',(QTV⌈1
[7] SMP:QB←(2⌈0 '
[8] FMT:Z←⊖FMT Z
[9] BOX:R C←⌈1⌈pZ
[10] QB←⌈1⌈pZ
[11] QB←⌈1⌈pZ
```

## FULL-SCREEN INPUT

Share Name

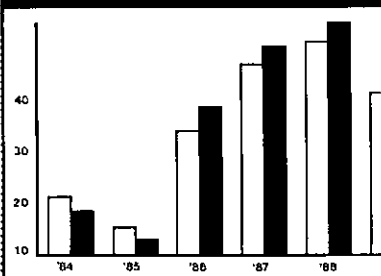
Number of Shares

Unit Cost

Buyer

EMPNO	ENAME	JOB	MGR
7329	SMITH	CLERK	7902
7499	ALLEN	SALESMAN	7698
7521	WARD	SALESMAN	7698
7566	JONES	MANAGER	7839
7654	MARTIN	SALESMAN	7839
7698	BLAKE	MANAGER	7830
7782	CLARK	MANAGER	7839
7788	SCOTT	ANALYST	7566
7839	KING	PRESIDENT	
7844	TURNER	SALESMAN	7698
7876	ADAMS	CLERK	7788

## GRAPHICS INTERFACE



## DEVICE SUPPORT

## NESTED ARRAYS

### DISPLAY REPORT

	North	South	East
Cakes	14	76	46
Buns	22	5	68

**...gives you all this – and APL too!**

For further information, contact  
**Dyadic Systems Limited**  
 Park House, The High Street, Alton, Hampshire, GU34 1EN  
 Telephone: (0420) 87024. Telex: 858811

## **TECHNICAL SECTION**

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

# A single source for APL*PLUS

---

APL*PLUS  
PC



for PCs  
and the PS2

APL*PLUS  
UNIX



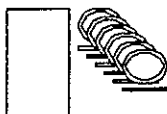
for UNIX  
systems

APL*PLUS  
PRO



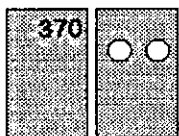
for the PS2  
model 80

APL*PLUS  
VMS

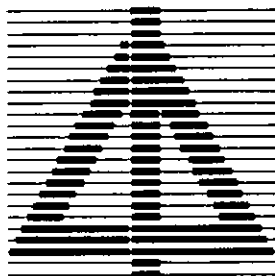


for the  
VAX range

APL*PLUS  
VM & MVS



for IBM  
mainframes



---

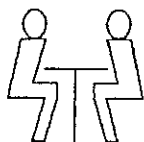
## COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road,  
London W1P 9LE  
Tel: 01 - 436 9481

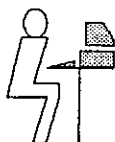
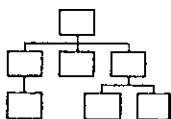
# A single source for APL

---



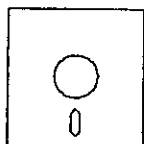
Advice

Design



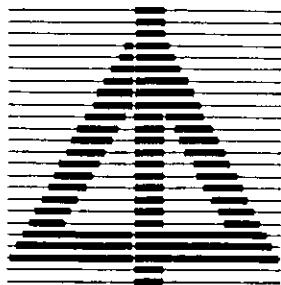
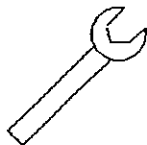
Application  
Development

Education



APL software

Support



---

**COCKING & DRURY LTD.**

THE APL PROFESSIONALS

180 Tottenham Court Road,  
London W1P 9LE

Tel: 01 - 436 9481

**Prize Caption Competition**

Entries are invited to caption the above pair of photographs. The usual rules and prizes apply (£100 split as the judges see fit amongst the three best entries), with the proviso that "Beam me up Scotty" is ruled to be a NONCE ERROR by the editor.

Please submit your entry to the editor, or to Dave Ziemann, by 1st April 1989.

## Hacker's Corner: Reading the TSO Session Screen

by Adrian Smith

Hacking the TSO session manager has always had some fun potential (such as the reverse-video pink and white stripes illustrated in an earlier VECTOR). However I have to admit that the function below is actually rather useful!

R+SNAP PAGES;XX;CTLZAP;DATZAP;SS;CTLISM;IN;FLDS

```
[1]      AGET TSO SESSION SCREEN(S) INTO RESULT VARIABLE.
[2]      A
[3]      A XX+SNAP 5  GETS LAST 5 SCREENS (LESS TRAILING NULL LINES)
[4]      A
[5]      A** Saved on 17/10/88 at 14:20 (by 2001)

[6]      XX+126 DSVO 2 6  P'CTLZAPDATZAP' # XX+120 DSVO 'CTLISM'
[7]      PAGES+1 P1+,PAGES
[8]      R+ 0 79 P' '

[9]      A SCROLL BACK TO THE FIRST REQUIRED PAGE ...

[10]     CTLISM+'PAGE -',PAGES-1 # XX+CTLISM

[11]     AFIND OUT HOW BIG THE SCREEN IS ....

[12]     CTLZAP+ 110 14 # SS+CTLZAP[8]

[13]     A SELECT GDDM PAGE 1001 ....

[14]     CTLZAP+ 305 1001 # XX+CTLZAP

[15]     AREAD ALL FIELDS .....

[16]     NEXT:FLDS=(1008+;SS-2)
[17]     CTLZAP+,422,FLDS,[1.5] 79 # XX+CTLZAP
[18]     IN+((SS-2),79)P DATZAP

[19]     A STOP IF WE HIT A SCREEN FULL OF NULLS ...

[20]     ->(DAV[256]A.,IN)+EXIT
[21]     R+R,[1] IN
[22]     ->(0/PAGES+PAGES-1)+EXIT
[23]     CTLISM+'PAGE +1' # XX+CTLISM # ->NEXT
```

```

[24]  RFINALLY DESELECT THIS RATHER DANGEROUS PAGE ....
[25]  EXIT:CTLZAP+ 305 1 # XX+CTLZAP
[26]  XX+QSVR 2 6 ρ'CTLZAPDATZAP'
[27]  R LOSE SPARE LINES AND THE LINE WITH 'SNAP' ON IT ...
[28]  R+((~1++/V\QRV.≠QAV[256]),79)†R
[29]  R TRANSLATE NULLS TO BLANKS ...
[30]  IN+,R # IN[(IN=QAV[256])/ιρIN]+ ' ' # R+(ρR)ρIN
  ▽

```

The motivation was as follows. We had just acquired a reasonable APL font for our Xerox 4045 laser. Of course this had the characters in very peculiar places, so a quick QAV lookup was in order. It also needed a few control sequences to select the APL font, set sensible margins and so on.

Now this was fine for function listings, where you already had hold of the data in an APL matrix. Of course it was quite useless for screen hardcopy (of things like traces) unless you could somehow nab the screen contents into a variable. Hence the above listed function.

It will go back as many screens as you ask for (SNAP 999 will end up at the beginning of scrolling memory) and then page forward a screen at a time taking snapshots as it goes. As soon as it hits a completely null screen it assumes (as a safety measure) that it has arrived back at your current screen, and it therefore gives up.

The other interesting use was in creating the basics of a user guide from genuine screen hardcopies. In this case the snapped images were wrapped around with some text and shipped down to the PC for final editing. Have fun.

## Technical Correspondence

### Security? What Security?

From: John Sullivan

20th July 1988

I am becoming more than a little concerned about the way APL implementers seem to imply that passwords in filing systems and function locking give security to an APL system. The truth of the matter is that there is no such thing as APL security, and any impression that there is gives a dangerous false sense of security to those who decide to rely on it. The only way to ensure that your APL programs and data are safe from prying eyes is to use the security systems that you use for your other programs and data, and if you haven't got any other security systems you ought to start looking for one.

If security is non-existent, what is the point of function locking? This question is even more relevant when there are well-known (but undocumented) 'unofficial' ways of unlocking locked functions which range from the exceedingly handy (APL*PLUS/PC) to the inconvenient (mainframe APL2). Even more, if you don't have a function unlocker, but you have the knowledge of the workings of APL that can be gained from talking to a few technically-minded people or from reading a few technical reports, you can write one in a reasonable time that will work well, even if it is not the best way of doing the job. For example I have recently written a 17-line function in MicroAPL's APL68000 for the Atari ST that will unlock locked functions; the research required to write this (which also involved deciphering the APL component filing system) took me about 24 hours of non-continuous work, and also gave several side-effects which may well prove profitable in further research.

So, function locking is a waste of time when it comes to security. What is function locking good for, then? Well, you don't want your users to get confused when your functions crash because some condition has occurred that you didn't program for (sooner or later this will happen to you!), so locking functions and allowing the error to be dealt with further up the stack seems a reasonable idea (this idea is already incorporated into IBM's dyadic quad-FX, with a feature to allow unlocked functions to be non-suspendable). To help with this, why don't the implementers give you a facility for easy unlocking of locked functions? I can think of two ways of doing this, those implementations that use quad-lock could also use quad-unlock, and of course, what is wrong with using del-tilde for opening function editing for a locked function?

I have heard some people say that functions are locked to protect the secrets of their algorithms. Having unlocked a few functions written by other people in my time I am not at all surprised. The only APL functions that you don't want people to look at are the ones

that you are ashamed of. If you genuinely have a super new algorithm that you don't want people to copy, why don't you supply the function to the users in machine code? Most APL systems have facilities for calling machine code, and machine code usually runs faster than APL, so use it.

### **Sharing GDDM pages between APL2 and Fortran**

From: J. Sullivan

19th August 1988

In my department, it is usual to provide the users with a full-screen panel showing them the progress of the program currently being executed. This panel contains 20 lines of automatically scrolling text surrounded by a static header and trailer, so that they can see the latest progress message together with the previous ones. This means that they can leave a program running and go away to do something else, returning periodically to see how things are getting on.

In order to speed processing up, certain parts of most applications have been or are being rewritten in FORTRAN, and are being called using the Name Association facility. Of course it would be nice to have FORTRAN put its progress messages to the same screen as the APL messages. Looking through the manuals (APL and FORTRAN as well as GDDM), there is no documentation on how to do this seemingly simple thing. So, through our official company channels, I asked IBM to do it. In the interests of those other members of the BAA who may be thinking along the same lines as I am, I have decided to circulate their answer to a wider audience.

You can't do it!

Yours sincerely

John Sullivan  
IT Systems Support  
National Westminster Bank plc  
London EC3V 3NN

### **Gizz a Font**

From: Iain Hayward

3rd November 1988

I am running APL.68000 on my Amiga at home, but those meanies at MicroAPL Ltd didn't give me a downloadable font for the printer. In fact printer support for what is otherwise an excellent product is almost nil (more on that later).

If recent claims by MicroAPL are anything to go by then there must be a lot of other users out there with the same setup. Has anyone got a downloadable font for a 24-pin Epson LQ-compatible printer? I'm thinking of making one and it should be quite interesting at first, writing a font editor (in APL of course!) and then designing the first few characters, but after a while it will become tedious and very time-consuming. I know because I have already done one for the Epson FX80. On that occasion the font editor was knocked up for me by Maurice Jordan in what seemed like a few minutes (including full-screen interface and automatic validation) in a remarkable demonstration of what a serious APLer can do with this language. Even with that program it still took a long time to design the whole font and I am therefore reluctant to do it all again unless it's necessary.

Why should we just give him a font that took us so long to make? I hear you ask. Well, if you've got the same setup as I have then you may still be having problems with printing. This is where a C programmer comes in handy (like me). Amiga APL.68000's printer support only goes as far as giving you a copy of what appears on the screen, including the input lines unfortunately. The manual says that if you want to do more than this (i.e. print reports, workspace documentation, etc., like most people) then you will need to use the 'Arbitrary Input/Output' functions. However, this only works with the serial port, fine for comms, but most people have their printer connected to the parallel port.

My solution to this problem so far, is to write output into a temporary DOS file first, and then (via a DOS window) run a C program which does a character translation and sends the output to the printer. I am thinking of writing a special printer driver so that I can send APL output directly to the printer. This won't be easy, but it should be worth it in return for a nice font.

By the way, did anyone see David Eastwood's article on idioms in the latest edition of MicroAPL News? Neat example of the use of not-equals-scan, wasn't it?

Yours sincerely,

Iain Hayward  
9 rue de Clairefontaine  
1341 Luxembourg  
Grand Duchy of Luxembourg

## The Limits of Forward Chaining

Allan G. Prys Williams  
Department of Management Science and Statistics  
University of Wales, Swansea

### Abstract

Forward chaining is the method of logical inference most 'natural' in APL. Naive implementations on real machines can easily prove impractical: they fail to terminate or they outgrow the workspace. Code is provided to restrict this growth and provide controlled behaviour in various circumstances. It turns out that there are two basic situations and the methods that work for one will not work for the other. Rules are given for both.

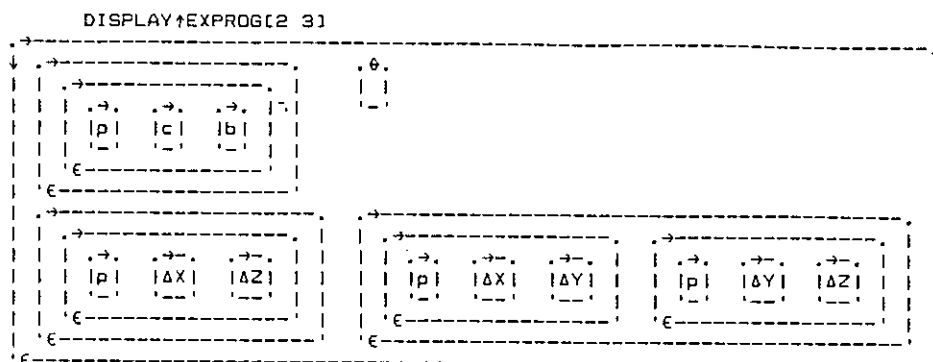
### Logical Structures in APL: Data Structures

Standard APL logic structures are explained in [1], and [2] gives the associated functions as they appear in Dyalog. The data structures used are nested character vectors of considerable depth. In IBM's terminology the highest level is the database, which is a vector of clauses each of which is supposed to be true.

A typical database is EXPROG whose contents in logical notation are

```
CONSTRUE EXPROG
p(a, b)
p(c, b)
(∧X)(∧Z)(p(X, Z) ← (∨Y)(p(X, Y) ∧ p(Y, Z)))
(∧X)(∧Y)(p(X, Y) ← p(Y, X))
```

while the actual APL structure of the middle two clauses is given below. Note that the factual statement has a null right hand side, and that the variables are flagged by their initial deltas, as well as their capital letters. The general rule in the display (rule 3) can be interpreted as 'for all X and Z, X is near Z if there exists a Y such that X is near Y and Y is near Z'. Rule 4, which is not expanded, can be read as 'for all X and Y, X is near Y if Y is near X'.



A clause is a two element vector of lists of predicates. A predicate is a statement which is essentially indivisible, such as  $p(a,b)$ , the general form being:

relation term1 term2 ....

where terms may be variable or fixed. Predicates are the building blocks of logic statements. The left hand side of a clause has usually one element, which is taken to be true if all the right hand side elements are true (unconditionally if there are no such elements). If there is more than one left hand predicate, then the clause means that at least one of them follows from the right hand side. There can be any number of right hand predicates, or none. All will have to be true to justify the left hand side.

Terms can be complex structures, as they may each be of the form

function term11 term12....

so that logical functions tend to be highly recursive.

### Combining Statements

In the basic system, predicates are compared by the recursive UNIFYA function which reports to UNIFY. UNIFY says whether two predicates are the same after appropriate substitutions and, if so, also returns the substitutions. This function is used by the RESOLVANT function to compare clauses in the database.

```

    V Z←AR RESOLVANT BR;T
[1]  AUNIFY AR with BR, Z is 0 or the implied resolution. A & B are globa.
[2]  Z←0 0 →(2←AR UNIFY BR)↓0
      Assume failure: return with 0 if no UNIFY
[3]  Z←((1←A)~cAR),1←B)((2←A),(2←B)~cBR)
[4]  Z←(EVAL DEPTH1 Z)(2←T)

```

This function is used inside functions such as:

```

    V Z←A RESOLVE1 B      A find all rules generated from A, B
[1]  Z←,(1←A)~.RESOLVANT(2←B)  A vector of results
[2]  B A←A B              A reverse order and ...
[3]  Z←Z,,(1←A)~.RESOLVANT(2←B) A ... repeat and catenate.
[4]  Z←Z~0                B eliminate where there wasn't a result.

```

which are designed to see if a new clause can be found from the combination of two old ones. Here RTEST2 is a two element database from [1]

```

    CONSTRUE A B←RTEST2
    (∧X)(p(a) ∨ p(X) ∨ q)
    r ← p(a)

```

and use of the function indicates that two new rules can be found by referring the p(a) in the second clause to either the p(a) or the p(X) [remember X is a variable] in the first clause:

```

    +C←A RESOLVE1 B
    p a X q r      p a q r      ΔX←(,'a')

```

the result being one complex and one simple rule (together with the necessary substitution):

```

    CONSTRUE⇒C
    (∧X)(p(X) ∨ q ∨ r)
    p(a) ∨ q ∨ r

```

## Forward Chaining

In many ways, the natural approach to inference in APL is a breadth-first one. It uses the 'dot-dot' technique. A function generates all immediate consequences of comparing all the rules in the database with each other, then compares the database with all the generated consequences, and so on until either the object of the search is found or no new rules appear.

This technique is very robust. Apparently simple rules, such as EXPROG[4], are able to throw sophisticated depth-first searching algorithms into confusion. Textbooks on Prolog (e.g. [3] Section 7.2) spend a lot of energy preaching the avoidance of such commutative or 'left recursive' rules. While a breadth-first search may still find an infinite path, it will not lock onto it to the exclusion of simpler ones.

It is called forward chaining because it works forward from the database rather than backwards from the object to be proved.

```

V PZ=GOAL FORWARD11 DB;NEW;NEW2;I;M A VCOUNT external
[1] PZ←1 ∅ VCOUNT←0           A assume goal will be found.
[2] NEW←DB                     A DB against itself first time
[3] L1:→(∅(GOAL)(UNIFY UNTIL 1)NEW)/O A done if goal is found
[4] NEW2←NEW. RESOLVE1 DB      A resolve everything
[5] DB←VDB,NEW                 A add last set of results to DB
[6] NEW←V∅,/,NEW2              A bring out new inferences
[7] NEW←(∼OBVIOUS NEW)/NEW     A attempt to eliminate tautologies
[8] O←NEW+VRENAME NEW          A rename variables
[9] →(O=NEW)/FAIL ∅ →L1       A fail if nothing new, else redo
[10] FAIL:PZ←O

```

Line 3 checks whether the goal matches, or is an instance of, one of the rules that are generated. The 'dot-dot' in line 4 is the mark of a breadth-first approach. The intrinsic problem of this approach is that a lot of redundant rules are generated on the way. Line 7 is an early attempt to cut things down by rejecting anything that is of the form 'X implies X'. This is done by the function:

```

V Z=OBVIOUS X A Checks to see if a predicate occurs both sides of X
[1] →(Z≠X)/Z←O           A Precaution: is X a clause?
[2] →(1=V/(C'')≡X)/Z←O   A Precaution: is X a fact (pos or neg)
[3] Z←V/,∅((.≡)/X)       A Matches each of X[1] to each of X[2]

```

For an example, we take the simple target GO and the database DATABASE:

```

CONSTRUE GO
by(box, window)
CONSTRUE DATABASE
by(table, window)
on(box, table)
(∧Z)(∧Y)(by(Z, Y) ← (UX)(by(X, Y) ∧ on(Z, X)))

```

What happens is displayed by line 8 of FORWARD11 which outputs the new, derived rules.

```

GO FORWARD11 DATABASE
by ΔZ_1 window      on ΔZ_1 table
by box ΔY_2          by table ΔY_2
by box window
by ΔZ_4 window      on ΔX_4 table      on ΔZ_4 ΔX_4
by ΔZ_5 ΔY_5        by table ΔY_5      on ΔZ_5 box
by box ΔY_2_6        by ΔX_6 ΔY_2_6    on table ΔX_6
1

```

First two intermediate rules are generated, and then a batch of four, the first of which is the intended result. The limitations of this intuitively appealing approach become apparent as soon as we try to prove something using the database EXPROG defined above.

```

CONSTRUE EXPOS
p(a, c)

```

The function FORWARD11 generates 692 intermediate rules in the course of producing a proof at two removes.

```

EXPOS FORWARD11 EXPROG
p a ΔZ_1          p b ΔZ_1
p ΔX_2 b          p ΔX_2 a
p b a
p c ΔZ_4          p b ΔZ_4
p ΔX_5 b          p ΔX_5 c
p b c
p ΔX_7 ΔZ_7       p ΔY_7 ΔX_7 p ΔY_7 ΔZ_7
p a ΔZ_8          p b ΔY_8 p ΔY_8 ΔZ_8
p ΔX_9 ΔZ_9       p b ΔZ_9 p ΔX_9 a
p a ΔZ_1_10       p b ΔY_10 p ΔY_10 ΔZ_1_10
p ΔX_11 a         p b ΔX_11
p a ΔZ_1_12       p ΔZ_1_12 b
....
p ΔX_38 ΔZ_38     p ΔY_7_38 ΔX_38
p ΔY_7_38 ΔY_38   p ΔY_38 ΔZ_38
p ΔX_39 ΔZ_39     p ΔY_7_39 ΔY_39
p ΔY_7_39 ΔZ_39   p ΔX_39 ΔY_39
p ΔX_7_40 ΔZ_7_40 p ΔY_7_40 ΔY_40
p ΔY_40 ΔX_7_40   p ΔY_7_40 ΔZ_7_40
....

```

Exhibit 1. By-Products of a Successful Proof

The result is even worse if we try to prove something that is in fact false using the DATABASE rules. In theory, it should inspect all possible deductions and return a zero when the goal doesn't match any of them. By the time the WS FULL message arrives, 1090 rules have been printed out with another 998 on the way. Some appear in Exhibit 2.

```

CONSTRUE FALSE
by(box, table)
  FALSE FORWARD11 DATABASE
  by ΔZ_1 window      on ΔZ_1 table
  by box ΔY_2          by table ΔY_2
  by box window
  by ΔZ_4 window      on ΔX_4 table      on ΔZ_4 ΔX_4
  by ΔZ_5 ΔY_5        by table ΔY_5      on ΔZ_5 box
  by box ΔY_2_6        by ΔX_6 ΔY_2_6    on table ΔX_6
  by ΔZ_7 window      on ΔZ_7 box
  by ΔZ_4_8 window    on ΔZ_4_8 box
  by box window      on table table
....
  by box window      on ΔX_4_10_32_172 table
  on ΔX_10_32_172 ΔX_4_10_32_172 on table ΔX_10_32_172
  on ΔX_6_172 box    on table ΔX_6_172
  by ΔZ_13_173 window on ΔX_4_10_32_173 table
  on ΔX_10_32_173 ΔX_4_10_32_173 on table ΔX_10_32_173
  on table box      on ΔX_13_173 box    on ΔZ_13_173 ΔX_13_173
....

```

Exhibit 2. Attempting to Prove a Negative

It is the purpose of the rest of this paper to give the idiom that will reduce these problems, and to discuss the extent to which they are in fact insoluble.

### Limiting Resolution: Suitable Filters

Looking at the seventh derived rule in the proof of EXPOS we find that the two predicates on the right hand side are identical. This sort of inefficiency is common as the number of cycles increases, so one way of improving the system is to ban over-inflated results. The simple idiom for this is in line 5 of

∇ Z←A RESOLVE2 B	A find all rules generated from A, B
[1] Z←,(1A)•.RESOLVANT(2B)	A vector of results
[2] B A←A B	A reverse order and ...
[3] Z←Z,,(1A)•.RESOLVANT(2B)	A ... repeat and catenate.
[4] Z←Z~0	A eliminate where there wasn't a result.
[5] Z←((Z~Z)≡U~"Z)/Z	A reject results if redundant predicates

and when we rerun using a new function FORWARD21 which is FORWARD11 with RESOLVE2 replacing RESOLVE1 we get:

```

....
p  b  c
p  a  ΔZ_7
p  ΔX_8  ΔZ_8
p  a  ΔZ_1_9
p  ΔX_10  a
p  a  ΔZ_1_11
....

p  b  ΔY_7
p  b  ΔZ_8
p  b  ΔY_9
p  b  ΔX_10
p  ΔZ_1_11  b

p  ΔY_7  ΔZ_7
p  ΔX_8  a
p  ΔY_9  ΔZ_1_9

```

Overall the number of rules derived during the proof is reduced from 692 to 317 and the time from 1081 to 515 cpu seconds [on an unimproved IBM 6151] which is a vast improvement.

However, there are still a lot of results that seem far more complicated than the original rules. These are, as are all the results that we are disallowing, logically valid. But are they useful, as well as true? The need for some sort of further improvement is shown by the fact that an attempt to prove the negative result for FALSE against the 3-element DATABASE again fails to converge within the available space and time.

One way to detect redundancy is to say that there should be an improvement in specialisation from the original general rules to the derived rules. This would be a strong heuristic principle, expressed by the exact rule that the number of variables should be reduced at each stage. We can do this by invoking existing utility functions, which were explained in [2] and are given in the appendix:

```

V Z←A RESOLVE3 B      # find all rules generated from A, B
[1] Z←,(1A)•.RESOLVANT(2B)  A vector of results
[2] B A←A B           A reverse order and ...
[3] Z←Z,,(1A)•.RESOLVANT(2B) A ... repeat and catenate.
[4] Z←Z~O             A eliminate where there wasn't a result.
[5] Z←((Z)≡U"~Z)/Z     A reject results if redundant predicates
[6] Z←((Z)~DELDREDGE"~Z)<1f/ /~DELDREDGE"A B)/Z A results must
[7] A reduce the number of free variables compared with originals

```

When this is plugged into FORWARD to get FORWARD31 it is at last possible to terminate the negative search of the DATABASE rules. After generating 141 rules in 27449.8 cpu seconds the algorithm decides that no more can be found. However, this approach still gives rise to some ungainly rules:

```

....
by  ΔZ_85 window      on ΔX_4_10_22_47_85 table
   on table ΔX_4_10_22_47_85 on table box      on ΔZ_85 box
by  ΔZ_86 window      on table table
   on ΔX_24_49_86 box on table ΔX_24_49_86 on ΔZ_86 box
by  box window        on table table
   on ΔX_24_49_87 box on table ΔX_24_49_87 on table box
....

```

The improvement in the positive search from EXPROG is less marked, giving a change to 217 from 317 in the number of derived rules and a reduction to 859 in the number of cpu seconds. This is worth having, but could be sacrificed if we were worried about disallowing some derived rules that might be useful as well as valid.

It will be seen from the derived rules just displayed that many of them are still very cumbersome. The presence of nonsensical predicates such as 'on(table, table)' is logically perfectly proper. It appears in a rule that says 'if the table is on the table (and other conditions) then Z86 is by the window'. A human logician would say that this was true but pointless. It is impossible for a machine to have a general concept of pointlessness, the nearest approach being heuristic principles that certain lines should not be pursued. We have begun to try this in RESOLVE3, and a stronger approach would be to say that any derived rule should be no more complicated than the longest of its parents.

We already have this principle in terms of the number of variables, so we add a filter based on the number of predicates on either side of the derived rule:

```

V Z+A RESOLVE4 B      A find all rules generated from A, B
[11] Z+,{(1A)}.RESOLVANT(2B) A vector of results
[2] B A+A B           A reverse order and ...
[3] Z+Z,{(1A)}.RESOLVANT(2B) A ... repeat and catenate.
[4] Z+Z~O             A eliminate where there wasn't a result.
[5] Z+((C"Z)≡U"C"Z)/Z A reject results if redundant predicates
[6] Z+((C"Λ"/(P"C"Z)S"/P"A B)/Z A do not allow increased length
[7] Z+((C"P"DELDREDGE"C"Z)<1f"/C"P"DELDREDGE"A B)/Z A results must
[8] A reduce the number of free variables compared with originals

```

The positive search now takes rather longer and there is no saving in the number of derived rules. However, the negative search terminates with only 27 rules in 735 seconds, as compared with 141 in 27450 seconds.

```

....
by box window          on table table      on table box
by box window          on table box
by box window          on ΔX_16_23 box      on table ΔX_16_23
by ΔZ_24 window        on table box        on ΔZ_24 box
by box window          on table table      on table box
by ΔZ_26 window        on table box        on ΔZ_26 box

```

0

## Opportunity Costs

Increasing the selectivity of the resolution process brings practical disadvantages. There are some contexts in which the database only contains rules, and the factual element is picked up from the goal. The function of the forward chaining is to generate rules of gradually increasing complexity until they match the goal. An example is the 2-element MEMDB database, shown together with three possible goals.

```

CONSTRUE MEMDB, M1 M2 M3
(ΔX)(ΔXS)member(X, X(XS))
(ΔX)(ΔYS)((ΔY)member(X, Y(YS)) ← member(X, YS))
member(1, [1,4,5,1])
member(1, [4,5,1])
member(1, [2,4,5,1])

```

Here a list is represented, using a structure discussed in detail in [4], as a head-tail pair. The first rule says that X is a member of a list if it is the element that is the head of the list. The second rule says that X is a member of a list if it is a member of the tail of that list. The first goal is an instance of the first rule.

```

M1 FORWARD41 MEMDB
1

```

But the second goal cannot be matched as no new rules are generated under the restrictions we have just been imposing:

```

M2 FORWARD41 MEMDB
0
M2 FORWARD31 MEMDB
0

```

Only when we go back two stages is the function allowed to generate complex sub-rules:

```

M2 FORWARD21 MEMDB
member  ΔX_1  ΔY_1  ΔX_1  ΔXS_1
1 member  ΔX_2  ΔY_2  ΔY_1_2  ΔX_2  ΔXS_1_2

```

M2 is an instance of the new rule generated on the second cycle. M3 needs three cycles:

```

M3 FORWARD21 MEMDB
member  ΔX_1  ΔY_1  ΔX_1  ΔXS_1
member  ΔX_2  ΔY_2  ΔY_1_2  ΔX_2  ΔXS_1_2
1 member  ΔX_3  ΔY_3  ΔY_2_3  ΔY_1_2_3  ΔX_3  ΔXS_1_2_3

```

Clearly, it is impossible to prove a negative using this approach. The database will go on hopefully generating longer and longer lists until the workspace limits are reached.

### Editing Out Redundant Rules

One of the first things generated by DATABASE is the fact 'by(box, window)', but many later derived rules are of the form 'by(box, window) if ...'. These are redundant since something that is true unconditionally is always true conditionally. This suggests that, if we detect absolute rules, any clauses that give conditional forms of them can be discarded. Line 8 in the revised function below finds clauses with null right hand sides in the old rules, and line 9 throws away any new rules that have these as their left hand side.

```

V PZ←GOAL FORWARD12 DB;NEW;NEW2;I;M A VCOUNT external
[1] PZ←1 0 VCOUNT←0 A assume goal will be found.
[2] NEW←DB A DB against itself first time
[3] L1:→(←(CGOAL)(UNIFY UNTIL 1)NEW)/0 A done if goal is found
[4] NEW2←NEW+.RESOLVE1 DB A resolve everything
[5] DB←VDB,NEW A add last set of results to DB
[6] NEW←VDB,NEW2 A bring out new inferences
[7] NEW←(←OBVIOUS(NEW))/NEW A attempt to eliminate tautologies
[8] M←(←(←(←DB)/DB) A find facts in old results
[9] (←(←(←NEW)/NEW) A delete rules proving facts
[10] DB←NEW←VRENAME(NEW) A rename variables
[11] →(←(←NEW)/FAIL 0 →L1 A fail if nothing new, else redo
[12] FAIL:PZ←0

```

Analogous functions FORWARD22, FORWARD32 and FORWARD42 use the more restrictive RESOLVE functions.

The negative search is much shorter in terms of both rules and time, and the difference is even more marked if RESOLVE3 is used rather than RESOLVE4.

## FALSE FORWARD42 DATABASE

```

by ΔZ_1 window          on ΔZ_1 table
by box ΔY_2              by table ΔY_2
by box window
by ΔZ_4 window          on ΔX_4 table          on ΔZ_4 ΔX_4
by ΔZ_5 ΔY_5            by table ΔY_5          on ΔZ_5 box
by box ΔY_2_6           by ΔX_6 ΔY_2_6        on table ΔX_6
by ΔZ_7 window          on ΔZ_7 box
by ΔZ_4_8 window        on ΔZ_4_8 box
by ΔZ_5_9 window        on ΔZ_5_9 box
by ΔZ_5_10 window       on table table        on ΔZ_5_10 box
by box ΔY_2_11          by table ΔY_2_11        on table box
by box ΔY_2_6_12        by table ΔY_2_6_12        on table box
by ΔZ_13 window         on ΔX_13 box          on ΔZ_13 ΔX_13
by ΔZ_5_14 window       on table box          on ΔZ_5_14 box

```

0

The saving is consistent for the positive search also, but the process is still ungainly. FORWARD42 generates some 175 intermediate rules in proving EXPOS.

Looking at the derived rules from FALSE FORWARD42 DATABASE above, it should be apparent that several say the same thing. There is a block of three reading 'by(Z,window) if on(Z,box)' and two reading 'by(box,Y) if on(table,Y) and on(table,box)'. The uniqueness function does not recognise these because of the suffixes on the variables. The next potential improvement is to check for this:

```

V PZ←GOAL FORWARD13 DB;NEW;NEW2;I;M A VCOUNT external
[1] PZ←1 ∅ VCOUNT←0 A assume goal will be found.
[2] NEW←DB A DB against itself first time
[3] L1:→(∅(CGDAL)(UNIFY UNTIL 1)NEW)/O A done if goal is found
[4] NEW2←NEW ∅ RESOLVE1 DB A resolve everything
[5] DB←UDB,NEW A add last set of results to DB
[6] NEW←UDB,NEW2 A bring out new inferences
[7] NEW←(∅OBVIOUS NEW)/NEW A attempt to eliminate tautologies
[8] M←((C'')≡"DB)/DB A find facts in old results
[9] Δ(O≠PNEW)/NEW←(Δ(∅M) ∅ Δ NEW)/NEW A delete rules proving facts
[10] →(O=PNEW)/FAIL A fail if nothing new, else begin ...
[11] I←1 ∅ M←O P O A loop to eliminate effective duplicate
[12] loop:M,1←(C)NEW (FRAUD UNTIL O)I←NEW,DB A generate mask M for...
[13] →((PNEW)≥I←1)/loop A genuinely new results.
[14] Q←NEW←VRENAME M/NEW A rename variables
[15] →(O=PNEW)/FAIL ∅ →L1 A fail if nothing new, else redo
[16] FAIL:PZ←O

```

Lines 11 to 13 generate a mask to filter out rules that are effectively identical to those already generated. The function FRAUD was introduced in [2]

```

V Z←X FRAUD Y;T A Determines if X is the same as Y modulo labelling
[1] Z←0 ⇐ (X=Y)/O A 0 means fraud. Exit if identical
[2] T←DEX 'Δ' QNL 2 ⇐ T←X UNIFYA Y ⇐ Δ(T=0)/'Z←1 ⇐ →0'
[3] A If X and Y do not unify, no fraud. Else look for substitutions
[4] T←↓ 'Δ' QNL 2 ⇐ Δ(O=PT)/'Z←1 ⇐ →0' A Precaution against null case
[5] T←Δ T ⇐ →(Δ/'Δ'≡"Δ" T)/O A If all substitutions are simple variables
[6] Z←1 A then fraud, else pass it as having some content.

```

We have now reached a minimal set of derived rules for DATABASE

```

FALSE FORWARD43 DATABASE
by ΔZ_1 window on ΔZ_1 table
by box ΔY_2 by table ΔY_2
by box window
by ΔZ_4 window on ΔX_4 table on ΔZ_4 ΔX_4
by ΔZ_5 ΔY_5 by table ΔY_5 on ΔZ_5 box
by box ΔY_2_6 by ΔX_6 ΔY_2_6 on table ΔX_6
by ΔZ_5_7 window on ΔZ_5_7 box
by ΔZ_5_8 window on table table on ΔZ_5_8 box
by box ΔY_2_6_9 by table ΔY_2_6_9 on table box
by ΔZ_10 window on ΔX_10 box on ΔZ_10 ΔX_10
by ΔZ_5_11 window on table box on ΔZ_5_11 box

```

0

and a similar improvement is found for EXPROG. The pattern in the number of derived rules can be set out as a table:

#### Number of Derived Rules for EXPOS FORWARD.. EXPROG

	RESOLVE4	RESOLVE3	RESOLVE2	RESOLVE1
FORWARD3	91	91	118	118
FORWARD2	175	175	275	646
FORWARD1	217	217	317	692

and for those versions for which the negative search converges:

#### Number of Derived Rules for FALSE FORWARD.. DATABASE

	RESOLVE4	RESOLVE3
FORWARD3	11	17
FORWARD2	14	26
FORWARD1	27	141

The relationship between choice of functions and time shows that the final improvement carries a considerable cost:

#### CPU Seconds for EXPOS FORWARD.. EXPROG

	RESOLVE4	RESOLVE3	RESOLVE2	RESOLVE1
	-----	-----	-----	-----
FORWARD3	11387	11222	23641	23590
FORWARD2	839	828	497	1066
FORWARD1	869	859	515	1081

though the effect is far less marked, or even absent, in the negative case:

#### CPU Seconds for FALSE FORWARD.. DATABASE

	RESOLVE4	RESOLVE3
FORWARD3	305	755
FORWARD2	262	935
FORWARD1	735	27450

This discrepancy is based in the fundamental difference between the two types of inference. The positive case requires the system to have a minimum time per cycle. Since halting is certain, redundancy is only worth eliminating if it is slowing things down. The negative case will not halt until it reaches a closed set of inferences, and thus the extra checking may well pay off in terms of time as well as elegance.

This type of checking for duplication could be strengthened to include cases where the lists contain the same predicates in a different order. These are logically equivalent, although the order of the predicates can make a big difference in many practical systems of inference (e.g. Prolog). However, this would require the unification of every permutation of one with every permutation of the other, with a frightening effect on overheads. Already, the FRAUD check often takes far longer than the original resolutions.

None of these precautions is sufficient to provide a closure for EXPROG. The set of possible deduced rules of no greater complexity than the originals is vast. For instance, in a single cycle FORWARD33 has to reduce NEW from 1271 items in line 9 to 277 in line 14. This prevents the size from exploding in the next cycle, but involves more than eight hundred thousand comparisons and takes an incredible length of time.

The extra overheads in terms of time seem to correspond to the use of the functions DELDREDGE and FRAUD. Thus FORWARD22 is fast and uses, apart from the resolution function, only raw APL. Since it also permits use of expanding systems such as MEMDB, it can be recommended for use in any situation where positive inference is expected, though it would be necessary in practice to trap the WS FULL error.

## Forcible Closure

It appears that, to get a system strong enough to terminate a negative search of a general set of rules, it is necessary to be downright brutal. It is also advisable to use raw APL as far as possible. We thus posit a form of resolution so strong that the extra checking in the third variant of FORWARD should be largely unnecessary:

```

V Z←A RESOLVES B      A find all rules generated from A, B
[1]  Z←,(1⊃A)←.RESOLVANT(2⊃B)  A vector of results
[2]  B A←A B           A reverse order and ...
[3]  Z←Z, (1⊃A)←.RESOLVANT(2⊃B) A ... repeat and catenate.
[4]  Z←Z~0             A eliminate where there wasn't a result.
[5]  Z←((⊃Z)≡0)~Z/Z     A reject results if redundant predicates
[6]  Z←(⊃Z)~/(P~Z)<~(C2 1)⌈⌈/P~A B)/Z
      A do not allow increased length
[7]  Z←((+/~'Δ'='~Z)<1⌈⌈+/~'Δ'='~A B)/Z A must reduce variables

```

This requires that the resolvants should be both simpler and positively shorter than the fuller of the original rules. It is powerfully reductive:

```

FALSE FORWARD52 DATABASE
by ΔZ_1 window      on ΔZ_1 table
by box ΔY_2         by table ΔY_2
by box window
by ΔZ_4 window      on ΔZ_4 box
0

```

and even EXPROG can only give rise to 19 rules.

	EXNEG	FORWARD52	EXPROG
p	a	$\Delta Z_1$	p b $\Delta Z_1$
p	$\Delta X_2$	b	p $\Delta X_2$ a
p	b	a	
p	c	$\Delta Z_4$	p b $\Delta Z_4$
p	$\Delta X_5$	b	p $\Delta X_5$ c
p	b	c	
p	b	$\Delta Z_7$	p a $\Delta Z_7$
p	$\Delta X_8$	a	p $\Delta X_8$ b
p	b	$\Delta Z_9$	p c $\Delta Z_9$
p	$\Delta X_{10}$	c	p $\Delta X_{10}$ b
p	b	b	
p	a	a	
p	c	a	
p	a	c	
p	c	c	
p	c	$\Delta Z_{16}$	p a $\Delta Z_{16}$
p	$\Delta X_{17}$	a	p $\Delta X_{17}$ c
p	a	$\Delta Z_{18}$	p c $\Delta Z_{18}$
p	$\Delta X_{19}$	c	p $\Delta X_{19}$ a

0

Exhibit 3. The Closure of EXPROG

EXPOS is proved after 15 cycles, and the times are 104.4 cpu secs for EXPOS and 232.7 secs for EXNEG, using FORWARD52 in both cases. FORWARD53 takes about twice as long, but has no effect on the outcome. FORWARD51 is not recommended. It takes longer than FORWARD52 and EXNEG does not converge.

## Conclusions

It is clear that there is no single best approach to forward chaining. The two quite different situations are:

- a. one where it is virtually certain that a rule will eventually be generated to match the goal. This is best matched by the limited restrictions in RESOLVE2 and FORWARD22. In practice it would be necessary to trap the WS FULL error.
- b. the more plausible one where a negative result is highly possible. This requires a set of rules that stops growing reasonably quickly, and there is no general principle to do this that will also allow complex rules to be generated. In this case we recommend RESOLVE5 used within FORWARD52.

At the theoretical level the precautionary technique in the FORWARD3 group of functions is extremely effective, but the cost in cpu time is unacceptable for most applications.

## References

- [1] Brown et al., 'Algorithms for Artificial Intelligence in APL2' in APL86 Tutorials, ed Camacho, London 1986, pp 87 - 150.
- [2] Prys Williams, A. G., 'Extending IBM's Logic Algorithms in Dyalog' in Vector Vol 4.1 (July 1987) pp104-122
- [3] Sterling, L. and Shapiro, E, The Art of Prolog, M.I.T., 1986
- [4] Guerreiro, R., 'APL2 and LISP: A Comparison' in APL86 Tutorials (London 1986) pp151-174

## Appendix: Other Functions That Have Been Cited.

The function DELDREDGE returns the set of variables cited within a logical statement:

```

      V R+DELDREDGE A      A Look for variables in A
[1]   R+''                  A Initialise list for variables
[2]   DREDGE1 A            A Search A. Variables are passed to R
[3]   R+UR~C''             A Delete duplicates and header

```

using the subsidiary function

```

      V DREDGE1 A
[1]   →((12|≡A)^( 'Δ'≡A)^3=QNC(A))/O      A Pass variables back to R in DREDGE
[2]   *(12|≡A)/'R, +C('Δ'≡A)/A φ →O'      A Stop on meeting logic function
[3]   DREDGE1'A                             A If A simple & variable, keep it
                                           A If complex, look one stage lower

```

The operator UNTIL is essentially that given in [1] changed to compare THIS with the first element in the result:

```

      V Z←(L)←(F UNTIL THIS)R;I      REACH THAT QUIT ON TRUE/FALSE
[1]   →(2≠QNC'L')/MONADIC
[2]   L←(O#PPL)/L1                    Abranch L not scalar
[3]   L←(PR)PL                        Aextend scalar left
[4]   L1←(O#PPR)/L2                   Abranch R not scalar
[5]   R←(PL)PR                        Aextend scalar right
[6]   L2:Z←I←O                        Ainitialize result and counter
[7]   LP:→(I2PR)/O                    Aexit when counter exceeds length
[8]   →(THIS≡Z←(CI↓L)F(CI↓R))/O      Aexit when result THIS
[9]   I←I+1 φ →LP                     Acontinue
[10]  MONADIC:Z←I←O                    Ainitialize result and counter
[11]  LP1:→(I2PR)/O                   Aexit when counter exceeds length
[12]  →(THIS≡Z←F(CI↓R))/O             Aexit when result THIS
[13]  I←I+1 φ →LP1                    Acontinue

```

## Missembling

by Allan Gay

The discerning reader will have noticed a couple of howling errors in my previous "Dissembling" article which I feel obliged to correct.

The first was, of course, the absolutely unforgiveable misspelling "wanged", which should, as all lovers of the English language appreciate, have been "whanged". My sincere apologies go to all those who were shocked into catatonia by this affront to their sensibilities - if they can still hear me.

The other error was a trivium which merely rendered the Assembler code completely unusable. I refer, of course, to the first Divide instruction which, operating on 52 significant bits (after shifting), produced a quotient requiring more than 31 bits for its expression and so yielded a fixed-point divide exception.

It's interesting to note that APL, which holds large values in exponential format and migrates freely between formats as the data varies, is unaffected by such narrow limitations. Because I neglected to take this fact into account, the error was not revealed by the simulation which was the subject of the article.

It's also interesting to note that the fixed point divide exception is the classic trap which lies in wait for the unwary writer of a TOD-clock decoding routine - and I fell headlong into it.

....

Examining the smouldering remains of the program, I confirmed that the first Divide instruction was the critical one. Subsequent divisions would produce results of manageable size once I was over that first hurdle. The question was: how to perform the division?

Eleven year old readers who have passed their mathematics exams will see the answer immediately. Those who, like me, re-sat the same exam at the age of thirty-five - and failed - may find the following description of interest. If not, they can always sneak off to the pub during the heavy bits.

The trick of it is to split the number into two manageable parts, divide them separately, note their quotients and remainders, and then combine them. Since we are working with 32-bit registers, of which the high order bit is reserved for a sign, it seems reasonable to split off the low-order 31 bits.

One snag which we have to watch out for is that the high-order fragment's remainder and quotient after division are  $2 \times 31$  times greater than they seem. Consequently, they must be multiplied by shifting, after which the remainder must be divided again. This gives us three quotients and two remainders for respective summing.

The other snag is that, when we sum the two remainders, we may get a result which exceeds the divisor. In this case, we must subtract the divisor from it and increment the sum of the quotients by 1. Since we are summing only two remainders, our result cannot but be smaller than twice the divisor, so a single incrementation is all we'll ever need.

The following APL function demonstrates the principles involved.

```

V R←DIV ALGORITHM TOD
[1] A
[2] A Assume TOD contains the 52-bit clock value
[3] A (ie. the value after shifting) and
[4] A DIV contains our chosen divisor.
[5] A
[6] HI←(TOD÷2×31) A Get high-order half in HI.
[7] LO←(2×31)TOD A Get low-order half in LO.
[8] A
[9] HIQUOT←(2×31)×(HI÷DIV) A Divide HI. The multiplication
[10] HIREM←(2×31)×(HI÷DIV) A by 2×31 reinstates significance.
[11] A
[12] HIQUOT2←(HIQUOT÷DIV) A Divide remainder, giving more
[13] HIREM←(HIREM÷DIV) A quotient and proper remainder.
[14] A
[15] LOQUOT←(LO÷DIV) A Divide LO, giving more quotient
[16] LOREM←(LO÷DIV) A and more remainder.
[17] A
[18] QUOT←(HIQUOT+HIQUOT2+LOQUOT) A Sum all three quotients.
[19] REM←(HIREM+LOREM) A Sum both remainders.
[20] A
[21] →(DIV>REM)/END A If sum of remainders exceeds
[22] REM←REM-DIV A divisor, subtract divisor from
[23] QUOT←QUOT+1 A it and increment quotients' sum.
[24] A
[25] END:R←QUOT,REM
V

```

Now that the principle has been demonstrated, it's time to put it to work. The following Assembler code does so. Afterwards, we can look at its simulation in APL.

The code given below replaces just four instructions in the previous article's example. The original instructions (with their comments) were:

```

      LM      6,7,0(3)      . R6-R7 CONTAIN INPUT DWORD DATE.
*
*-----SPLIT SECONDS OFF-----
      D      6,=F'1000000' . R6=MICROSECONDS OVER. R7=SECONDS.
      LR      2,6           . PUT MICROSECS FRACTION ASIDE IN R2.
*
*-----SPLIT DAYS OFF-----
      SR      6,6

```

The replacement instructions are, ahem, a little more extensive:

```

      LM      2,3,0(3)      . R2-R3 CONTAIN INPUT DWORD DATE.
      SRDL    2,12         . CONVERT TOD VALUE TO MICROSECONDS.
*
*-----SPLIT SECONDS OFF-----
*
* AFTER THE ABOVE SHIFT, THE MICROSECS COUNT IS STILL A 52-BIT NUMBER.
* DIVIDING BY 1,000,000 WOULD PRODUCE A RESULT IN EXCESS OF 31 BITS
* AND SO WOULD YIELD A FIXED POINT DIVIDE EXCEPTION.
* THE PROBLEM IS SOLVED BY BREAKING THE MICROSECS COUNT INTO TWO
* HALVES, DIVIDING EACH SEPARATELY, FURTHER DIVIDING THE HIGH ORDER
* HALF'S REMAINDER AND THEN SUMMING ALL THE RESULTS.
*
      SLDL    2,1           . PUT LO-ORDER 31 BITS INTO R3 WITH
      SRL     3,1           . ZERO SIGN-BIT.
      LR      7,2           . PUT HIGH ORDER HALF IN R7.
      SR      6,6           . CLEAR THE ASSOCIATED EVEN REGISTERS
      SR      2,2           . READY FOR DIVIDE OPERATIONS.
*
      D      2,=F'1000000' . R2=LO MICROSECS R3=LO SECS.
      D      6,=F'1000000' . R6=HI MICROSECS R7=HI SECS. (/2*31)
      LR      9,6           . R8-9 GET HI MICROSECS/2*31 READY
      SR      8,8           . FOR FURTHER DIVISION.
      SR      6,6           . R6-7 ARE HI SECS/2*31.
      SLDL    6,31         . MULTIPLY HI SECS AND MICROSECS BY
      SLDL    8,31         . 2*31 READY FOR ADDITIONS. NOTE
*                          . HI MICROSECS NEEDS ANOTHER DIVISION.
      ALR     7,3           . R7 GETS LO SECS + HI SECS.
      BC      12,*+8        . IF CARRY OCCURRED OUT OF R7, ADD IT
      AH      6,=H'1'       . INTO R6.
*
*
      D      8,=F'1000000' . RE-DIVIDE HI MICROSECS:
*                          . R8=TRUE HI MICROSECS R9=SECS-ADJUST.
      ALR     7,9           . R7=LO SECS + HI SECS + SECS ADJUST.
      BC      12,*+8        . IF CARRY OCCURRED OUT OF R7, ADD IT
      AH      6,=H'1'       . INTO R6.
      AR      2,8           . R2=LO MICROSECS + TRUE HI MICROSECS.
      C      2,=F'1000000' . IF THE SUMMED MICROSECS COUNT
      BL      GETDAYS       . EXCEEDS 1,000,000 REDUCE IT BY
      S      2,=F'1000000' . 1,000,000 AND INCREMENT THE SUMMED

```

```

AL      7,=F'1'          SECONDS COUNT.
BC      12,*+8          . IF CARRY OCCURRED OUT OF R7, ADD IT
AH      6,=H'1'          INTO R6.

*
*-----SPLIT DAYS OFF-----
GETDAYS EQU *
```

The AL and ALR instructions may require a word of explanation. The usual System/370 fixed point binary Add instructions operate on signed values. The high order bit of a 32-bit register is used to carry a sign: an off-bit denotes a positive value and an on-bit denotes a negative value.

AL (Add Logical) instructions view all 32 bits as data and set the condition code to indicate whether a carry-out occurred. When, as here, we operate on a value which straddles two adjacent registers, we can use the AL and ALR instructions to add values into the low-order register of the pair and, if a carry-out occurs, detect it and increment the high-order register.

The APL function below simulates the Assembler code and uses quite a few dependent functions to simulate System/370 instructions. We'll take a look at those in a moment.

```

V DIVIDEΔBYΔΔMILLION INPUT;CK;CC;REGS;TR;X
[1] A
[2] A INPUT is 16-element vector of base-16 integers representing
[3] A a recent 64-bit TOD-clock value.
[4] A
[5] A A typical value is: 9 13 5 11 13 2 12 13 5 14 1 9 10 0 0 0
[6] A or, in hex:          9D5BD2CD5E19A000
[7] A denoting the date: 1987 09 22 07 34 28 145 562
[8] A
[9] CK←1 A                      Turn on argument-checking.
[10] TR←0 A                     Turn off tracing.
[11] CC← 0 0 0 0 0 A           Start condition-code.
[12] REGS← 16 32 p1 A          Start registers with garbage.
[13] A
[14] X←64 BINARY 16ΔINPUT A
[15] REGS[2 3 ;]← 2 32 pX A
[16] TRACE 'ON'
[17] SRDL 2,12 A
[18] SLDL 2,1 A
[19] SRL 3,1 A
[20] LR 7,2 A
[21] SR 6,6 A
[22] SR 2,2 A
[23] D 2,1000000 A
[24] D 6,1000000 A
[25] LR 9,6 A
[26] SR 8,8 A
[27] SR 6,6 A
[28] SLDL 6,31 A
[29] SLDL 8,31 A

-----ASSEMBLER INSTRUCTIONS-----
LM      R2,R3,0(R3)

SRDL    R2,12
SLDL    R2,1
SRL     R3,1
LR      R7,R2
SR      R6,R6
SR      R2,R2
D       R2,=F'1000000'
D       R6,=F'1000000'
LR      R9,R6
SR      R8,R8
SR      R6,R6
SLDL    R6,31
SLDL    R8,31
```

```

[30] ALR 7,3 A
[31] →BC 12,0LC[0IO]+2 A
[32] AH 6,1 A
[33] D 8,1000000 A
[34] ALR 7,9 A
[35] →BC 12,0LC[0IO]+2 A
[36] AH 6,1 A
[37] AR 2,8 A
[38] C 2,1000000 A
[39] →BL GETDAYS A
[40] S 2,1000000 A
[41] AL 7,1 A
[42] →BC 12,0LC[0IO]+2 A
[43] AH 6,1 A
[44] GETDAYS: A
[45] TRACE 'OFF'
[46] A
[47] ' COMPUTED SECS= ',*DECIMAL,REGS[6 7 ;]
[48] ' COMPUTED MSECs= ',*DECIMAL REGS[2;]
[49] ''
[50] ' SECS SHOULD BE ',*(DECIMAL -12465 BINARY 161INPUT)+1000000
[51] ' MSECs SHOULD BE ',*10000001DECIMAL -12465 BINARY 161INPUT

```

GETDAYS EQU *

As I said, the function uses quite a few dependent functions. Before we look at them in detail, a general prospectus is perhaps in order.

Simulated System/370 instructions:

A	Add	AH	Add Halfword
AL	Add Logical	ALR	Add Logical Register
AR	Add Register	BC	Branch on Condition
BCT	Branch on Count	BCTR	Branch on Count Register
BH	Branch on High	BL	Branch on Low
BNO	Branch if Not Ones	C	Compare
CH	Compare Halfword	D	Divide
LA	Load Address	LR	Load Register
MH	Multiply Halfword	S	Subtract
SH	Subtract Halfword	SLA	Shift Left Single
SLDL	Shift Left Double Logical	SR	Subtract Register
SRDA	Shift Right Double	SRDL	Shift Right Dble Logical
SRL	Shift Right Logical		

General services used by the simulated instructions:

ADDL	performs logical addition
ADDR	checks/converts an integer to a 24-bit Boolean
ADDS	performs signed addition
BIN2HEX	converts a Boolean to hexadecimal representation
BINARY	converts a decimal integer (either sign) to Boolean
DECIMAL	converts a Boolean to a signed decimal integer
FULLWORD	checks/converts a decimal integer to a 32-bit Boolean
HALFWORD	checks/converts a decimal integer to a 16-bit Boolean
SUBS	performs signed subtraction
TRACE	displays regs, concode and current instruction image

Listings of all these functions follow. Assembler buffs will confirm that the foregoing represents only a small fraction of the System/370 repertoire. These are easy ones to simulate because they all do things with registers. I have yet to devise a way of simulating mainstorage accesses, although of course it is easy enough to supply literal values directly, as the simulation indeed does.

Not wishing to get caught again by fixed point overflow situations, I have elected to run each simulated register as a row of REGS (a 16 by 32-bit Boolean), program length checks throughout and simulate the System/370 condition code, too.

Many of the dependent functions rely on the existence of REGS and update CC, the condition code. Wherever possible, I've programmed checks for System/370 abends. My old friend, the fixed point divide exception, is in there for starters.

You'll notice that a TRACE function is called all over the place. A glance at TRACE's code shows it to be a pretty ugly looking customer. However, when Boolean TR is on, it produces very pretty hexadecimal displays of the simulated registers. These proved very useful for debugging both the functions and the Assembler code, besides enhancing the entertainment value of the simulation.

You'll also notice that a CHECK function is called everywhere. When Boolean CK is on, it checks that function arguments are correctly-formed. If not, it notifies an error. This error might be considered the equivalent of an Assembly error, but I wouldn't like to strain the metaphor too far.

Before anybody asks: no, this code is not going to do the mile in under four minutes. I take the view that, when I'm having fun, I want it to last as long as possible.

Functions simulating System/370 instructions:

▽ A RV

```
[1] A System/370 Add instruction
[2] A RV[010] is the target register.
[3] A RV[010+1] is the value to be added to it.
[4] A
[5] 'RV' CHECK RV
[6] REGS[RV[010];]+(REGS[RV[010];])ADDS FULLWORD RV[010+1]
[7] 'A' TRACE RV
[8] →(~CC[010+3])/0
[9] 'A ',(RV[010]),',',(RV[010+1]),' ---FIXED POINT OVERFLOW!'
[10] °
```

▽

▽ AH RV

```
[1] A System/370 Add Halfword instruction
[2] A RV[010] is the target register.
[3] A RV[010+1] is the value to be added to it.
[4] A
[5] 'RV' CHECK RV
[6] REGS[RV[010];]+(REGS[RV[010];])ADDS ^32^HALFWORD RV[010+1]
[7] 'AH' TRACE RV
[8] →(~CC[010+3])/0
[9] 'AH ',(RV[010]),',',(RV[010+1]),' ---FIXED POINT OVERFLOW!'
[10] °
```

▽

▽ AL RV

```
[1] A System/370 Add Logical instruction
[2] A RV[010] is the target register.
[3] A RV[010+1] is the value to be added to it.
[4] A
[5] 'RV' CHECK RV
[6] REGS[RV[010];]+(REGS[RV[010];])ADDL FULLWORD RV[010+1]
[7] 'AL' TRACE RV
```

▽

▽ ALR RR

```
[1] A System/370 Add Logical Register instruction
[2] A RR[010] is the target register.
[3] A RR[010+1] is the register containing the value to be added.
[4] A
[5] 'RR' CHECK RR
[6] REGS[RR[010];]+(REGS[RR[010];])ADDL REGS[RR[010+1];]
[7] 'ALR' TRACE RR
```

▽

```

    ▽ AR RR;A
[1]  A System/370 Add Register instruction
[2]  A RR[IO] is the target register.
[3]  A RR[IO+1] is the register containing the value to be added.
[4]  A
[5]  'RR' CHECK RR
[6]  REGS[RR[IO];]+(REGS[RR[IO];])ADDS REGS[RR[IO+1];]
[7]  'AR' TRACE RR
[8]  →(~CC[IO+3])/0
[9]  'AR ',(RR[IO]),',',(RR[IO+1]),' ---FIXED POINT OVERFLOW!'
[10] 0

```

▽

```

    ▽ R←BC VA
[1]  A System/370 Branch on Condition instruction
[2]  A VA[IO] is the condition code value.
[3]  A VA[IO+1] is the branch address.
[4]  A
[5]  'RA' CHECK VA
[6]  R←(V/CC^ 2 2 2 2 +VA[IO])/VA[IO+1]
[7]  'BC' TRACE VA

```

▽

```

    ▽ R←BCT RV
[1]  A System/370 Branch on Count instruction
[2]  A RV[IO] is the register.
[3]  A RV[IO+1] is the branch address.
[4]  A
[5]  'RA' CHECK RV
[6]  REGS[RV[IO];]+32 BINARY(2+REGS[RV[IO];])-1
[7]  R←(V/REGS[RV[IO];])/RV[IO+1]
[8]  'BCT' TRACE RV

```

▽

```

    ▽ R←BCTR RR
[1]  A System/370 Branch on Count Register instruction
[2]  A RR[IO] is the register to be decremented.
[3]  A RR[IO+1] is the register containing the branch address.
[4]  A
[5]  'RR' CHECK RR
[6]  REGS[RR[IO];]+32 BINARY(2+REGS[RR[IO];])-1
[7]  R←(V/REGS[RR[IO];])/DECIMAL 0.8+REGS[RR[IO+1];]
[8]  'BCTR' TRACE RR
[9]  →(RR[IO+1]≠0)/0
[10] R←0

```

▽

```

    ▽ R←BH A
[1]  A System/370 Branch on High instruction
[2]  A A is the branch address.
[3]  A
[4]  R←BC 2,A

```

▽

```

    ▽ R←BL A
[1]  A System/370 Branch on Low instruction
[2]  A A is the branch address.
[3]  A
[4]  R←BC 4,A

```

▽

▽ R←BNO A

```
[1] A System/370 Branch if Not Ones instruction
[2] A A is the branch address.
[3] A
[4] R←BC 14,A
```

▽

▽ C RV;A

```
[1] A System/370 Compare instruction
[2] A RV[0IO] is the register.
[3] A RV[0IO+1] is the value of comparand.
[4] A
[5] 'RV' CHECK RV
[6] A←DECIMAL REGS[RV[0IO];]
[7] CC← 2 2 2 2 T 4 8 2 [0p0IO+1+xA-DECIMAL FULLWORD RV[0IO+1]]
[8] 'C' TRACE RV
```

▽

▽ CH RV;A

```
[1] A System/370 Compare Halfword instruction
[2] A RV[0IO] is the register.
[3] A RV[0IO+1] is the value of comparand.
[4] A
[5] 'RV' CHECK RV
[6] A←DECIMAL REGS[RV[0IO];]
[7] CC← 2 2 2 2 T 4 8 2 [0p0IO+1+xA-DECIMAL HALFWORD RV[0IO+1]]
[8] 'CH' TRACE RV
```

▽

▽ D RV;DIVIDEND;X

```
[1] A System/370 Divide instruction
[2] A RV[0IO] is the even-numbered register of the register pair
[3] A containing the dividend.
[4] A RV[0IO+1] is the divisor.
[5] A
[6] 'RV' CHECK RV
[7] →(1=2|RV[0IO])/XSPEC
[8] DIVIDEND←DECIMAL REGS[RV[0IO];],REGS[RV[0IO+1];]
[9] A
[10] →(X≠DECIMAL 32 BINARY X←RV[0IO+1]|DIVIDEND)/XDIV
[11] →(X≠DECIMAL 32 BINARY X←(DIVIDEND←DECIMAL FULLWORD RV[0IO+1])/XDIV
[12] A
[13] REGS[RV[0IO];]+32 BINARY RV[0IO+1]|DIVIDEND
[14] REGS[RV[0IO+1];]+32 BINARY(DIVIDEND←RV[0IO+1])
[15] 'D' TRACE RV
[16] →0
[17] A
[18] XDIV:
[19] 'D ',(RV[0IO]),',',(RV[0IO+1]),' ---FIXED POINT DIVIDE EXCEPTION'
[20] °
[21] A
[22] XSPEC:
[23] 'D ',(RV[0IO]),',',(RV[0IO+1]),' ---SPECIFICATION EXCEPTION'
[24] °
```

▽

## ▽ LA RA

```

[11] A System/370 Load Address instruction
[21] A RA[010] is the target register.
[31] A RA[010+1] is the address.
[41] A
[51] 'RA' CHECK RA
[61] REGS[RA[010];]←32↑ADDR RA[010+1]
[71] 'LA' TRACE RA

```

▽

## ▽ LR RR

```

[11] A System/370 Load Register instruction
[21] A RR[010] is the target register.
[31] A RR[010+1] is the register containing the value to be copied.
[41] A
[51] 'RR' CHECK RR
[61] REGS[RR[010];]←REGS[RR[010+1];]
[71] 'LR' TRACE RR

```

▽

## ▽ MH RV;X

```

[11] A System/370 Multiply Halfword instruction
[21] A RV[010] is the target register.
[31] A RV[010+1] is the value of the multiplier.
[41] A
[51] 'RV' CHECK RV
[61] X←(DECIMAL REGS[RV[010];])×DECIMAL HALFWORD RV[010+1]
[71] REGS[RV[010];]←32 BINARY X
[81] 'MH' TRACE RV

```

▽

## ▽ S RV;A

```

[11] A System/370 Subtract instruction
[21] A RV[010] is the target register.
[31] A RV[010+1] is the value to be subtracted from it.
[41] A
[51] 'RV' CHECK RV
[61] REGS[RV[010];]←REGS[RV[010];]SUBS FULLWORD RV[010+1]
[71] 'S' TRACE RV
[81] →(~CC[010+3])/0
[91] 'S ',(RV[010]),',',',',(RV[010+1]),' ---FIXED POINT OVERFLOW!'
[101] °

```

▽

## ▽ SH RV;A

```

[11] A System/370 Subtract Halfword instruction
[21] A RV[010] is the target register.
[31] A RV[010+1] is the value to be subtracted from it.
[41] A
[51] 'RV' CHECK RV
[61] REGS[RV[010];]←REGS[RV[010];]SUBS 32↑HALFWORD RV[010+1]
[71] 'SH' TRACE RV
[81] →(~1↑CC)/0
[91] 'SH ',(RV[010]),',',',',(RV[010+1]),' ---FIXED POINT OVERFLOW!'
[101] °

```

▽

## ▽ SLA RV;A

```

[1] A System/370 Shift Left Single instruction
[2] A RV[IO] is the register containing the value to be shifted.
[3] A RV[IO+1] is the number of bit-positions through which the
[4] A value is to be shifted.
[5] A
[6] 'RV' CHECK RV
[7] CC← 0 0 0 0
[8] CC[IO+3]←v/RV[IO+1]↑,REGS[RV[IO];]
[9] REGS[RV[IO];1+32↑RV[IO+1]↓,REGS[RV[IO];]
[10] →CC[IO+3]/OVER
[11] A←DECIMAL REGS[RV[IO];]
[12] CC← 2 2 2 2 T 4 8 2 [0pIO+1×A-DECIMAL FULLWORD RV[IO+1]]
[13] 'SLA' TRACE RV
[14] →0
[15] OVER:'SLA' TRACE RV
[16] 'SLA ',(RV[IO]),',',(RV[IO+1]),' ---FIXED POINT OVERFLOW!'
[17] 0

```

▽

## ▽ SLDL RV

```

[1] A System/370 Shift Left Double Logical instruction
[2] A RV[IO] is the even-numbered register of the register pair
[3] A containing the value to be shifted.
[4] A RV[IO+1] is the number of bit-positions through which the
[5] A value is to be shifted.
[6] A
[7] 'RV' CHECK RV
[8] →(1=2↑RV[IO])/XSPEC
[9] REGS[RV[IO]+ 0 1 ;1← 2 32 p64↑RV[IO+1]↓,REGS[RV[IO]+ 0 1 ;]
[10] 'SLDL' TRACE RV
[11] →0
[12] A
[13] XSPEC:
[14] 0←'SLDL ',(RV[IO]),',',(RV[IO+1]),' ---SPECIFICATION EXCEPTION
[15] 0

```

▽

## ▽ SR RR

```

[1] A System/370 Subtract Register instruction
[2] A RR[IO] is the target register.
[3] A RR[IO+1] is the register containing the value to be subtracted.
[4] A
[5] 'RR' CHECK RR
[6] REGS[RR[IO];]+REGS[RR[IO];]SUBS REGS[RR[IO+1];]
[7] 'SR' TRACE RR
[8] →(~CC[IO+3])/0
[9] 'SR ',(RR[IO]),',',(RR[IO+1]),' ---FIXED POINT OVERFLOW!'
[10] 0

```

▽

## ▽ SRDA RV;A

```

[11] A System/370 Shift Right Double instruction
[12] A RV[ $\Pi$ IO] is the even-numbered register of the register pair
[13] A containing the value to be shifted.
[14] A RV[ $\Pi$ IO+1] is the number of bit-positions through which the
[15] A value is to be shifted.
[16] A
[17] 'RV' CHECK RV
[18]  $\rightarrow (1=2 \mid RV[\Pi IO]) / XSPEC$ 
[19] REGS[RV[ $\Pi$ IO]+ 0 1 ;]  $\leftarrow 2 \mid 32p64 \uparrow (-64 + RV[\Pi IO + 1]) \uparrow$ , REGS[RV[ $\Pi$ IO]+ 0 1 ;]
[20] A  $\leftarrow$  DECIMAL, REGS[RV[ $\Pi$ IO]+ 0 1 ;]
[21] CC  $\leftarrow$  2 2 2 2  $\tau$  4 8 2 [ $\oplus p \Pi IO + 1 \times A$ ]
[22] 'SRDA' TRACE RV
[23]  $\rightarrow 0$ 
[24] A
[25] XSPEC:
[26]  $\square \leftarrow$  'SRDA ', ( $\&RV[\Pi IO]$ ), ', ', ( $\&RV[\Pi IO + 1]$ ), ' ---SPECIFICATION EXCEPTION'
[27]  $\circ$ 

```

▽

## ▽ SRDL RV

```

[11] A System/370 Shift Right Double Logical instruction
[12] A RV[ $\Pi$ IO] is the even-numbered register of the register pair
[13] A containing the value to be shifted.
[14] A RV[ $\Pi$ IO+1] is the number of bit-positions through which the
[15] A value is to be shifted.
[16] A
[17] 'RV' CHECK RV
[18]  $\rightarrow (1=2 \mid RV[\Pi IO]) / XSPEC$ 
[19] REGS[RV[ $\Pi$ IO]+ 0 1 ;]  $\leftarrow 2 \mid 32p64 \uparrow (-64 + RV[\Pi IO + 1]) \uparrow$ , REGS[RV[ $\Pi$ IO]+ 0 1 ;]
[20] 'SRDL' TRACE RV
[21]  $\rightarrow 0$ 
[22] A
[23] XSPEC:
[24]  $\square \leftarrow$  'SRDL ', ( $\&RV[\Pi IO]$ ), ', ', ( $\&RV[\Pi IO + 1]$ ), ' ---SPECIFICATION EXCEPTION'
[25]  $\circ$ 

```

▽

## ▽ SRL RV

```

[11] A System/370 Shift Right Single Logical instruction
[12] A RV[ $\Pi$ IO] is the register containing the value to be shifted.
[13] A RV[ $\Pi$ IO+1] is the number of bit-positions through which the
[14] A value is to be shifted.
[15] A
[16] 'RV' CHECK RV
[17] REGS[RV[ $\Pi$ IO];]  $\leftarrow 1 + 32 \uparrow (-32 + RV[\Pi IO + 1]) \uparrow$ , REGS[RV[ $\Pi$ IO];]
[18] 'SRL' TRACE RV

```

▽

General services used by the simulated instructions:

```

    ▽ R←A ADDL B
[11] A Logical addition of Booleans A and B.
[2]  A
[3]  R←(1+ρA)BINARY(21A)+21B A      Extra high-order bit
[4]  CC←(2×R[IO])φ4↑(Λ/~1↓R)φ 0 1 A detects overflow.
[5]  R←1↓R
    ▽
    ▽ R←ADDR DEC
[11] A Convert DEC decimal address to 24-bit Boolean.
[2]  R←(24ρ2)↑DEC
[3]  →(DEC=DECIMAL 0,R)/0
[4]  →(X#DEC),' exceeds 24-bit address capacity.'
[5]  0
    ▽
    ▽ R←A ADDS B;X
[11] A Signed addition of Booleans A and B.
[2]  CC← 0 0 0 1 A      Assume overflow.
[3]  X←(DECIMAL A)+DECIMAL B A      Add the values.
[4]  →(X#DECIMAL R+(ρA)BINARY X)/0 A If precision lost, overflow.
[5]  CC← 2 2 2 2 ↑ 4 8 2 [0ρIO+1+×X] A Else set condition code.
    ▽
    ▽ R←DIGITS BINARY DEC
[11] A DIGITS declares the number of bits required in result R.
[2]  A
[3]  R←(DIGITSρ2)↑DEC A      Convert DEC decimal integer to Boolean.
    ▽
    ▽ R←BIN2HEX BIN
[11] A Give hex representation of BIN Boolean
[2]  A
[3]  R←'0123456789ABCDEF'[IO+((f(ρBIN)+4)ρ16)↑21BIN]
    ▽
    ▽ REQ CHECK ARG
[11] A Check ARG data according to REQ request code.
[2]  A
[3]  →(0=UNC 'CK')/0
[4]  →(1#CK)/0
[5]  A
[6]  →(1#ρARG)/ERR
[7]  →(0#1↑0ρARG)/ERR
[8]  →(v/0#1|ARG)/ERR
[9]  →(2#ρARG)/ERR
[10] A
[11] →((3 2 ρ'RRRVRA')Λ.=2↑,REQ)/RR,RV,RA
[12] →0
[13] A
[14] RR:→(v/15<ARG)/ERR
[15] →(v/0>ARG)/ERR
[16] →0
[17] A
[18] RV:→(15<1↑ARG)/ERR
[19] →(0>1↑ARG)/ERR
[20] →0

```

```

[21] A
[22] RA:→(15<1↑ARG)/ERR
[23] →(V/O>ARG)/ERR
[24] →((~1+2*24)<1↑ARG)/ERR
[25] →0
[26] A
[27] ERR:[]+'ASSEMBLY ERROR!'
[28] °
▽

▽ R←DECIMAL BIN
[1] R←0p211↑BIN A Convert BIN Boolean to decimal integer.
[2] →(0=1↑BIN)/0 A If BIN sign bit is on,
[3] R←0p-(2*~1+0BIN)-R A convert R to negative number.
▽

▽ R←FULLWORD DEC
[1] A Convert DEC decimal integer to 32-bit Boolean.
[2] R←(32p2)↑DEC
[3] →(DEC=DECIMAL R)/0
[4] []+(0DEC),' exceeds the capacity of a fullword'
[5] °
▽

▽ R←HALFWORD DEC
[1] A Convert DEC decimal integer to 16-bit Boolean.
[2] R←(16p2)↑DEC
[3] →(DEC=DECIMAL R)/0
[4] []+(0DEC),' exceeds the capacity of a halfword'
[5] °
▽

▽ R←A SUBL B;X
[1] A Logical subtraction of Booleans A and B.
[2] A
[3] R←A ADDL(2*1+0B)-B
[4] →(~CC[0IO])/0
[5] CC←20CC
▽

▽ R←A SUBS B;X
[1] A Signed subtraction of Booleans A and B.
[2] CC← 0 0 0 1 A Assume overflow.
[3] X←(DECIMAL A)-DECIMAL B A Subtract the values.
[4] →(X≠DECIMAL R+(0A)BINARY X)/0 A If precision lost, overflow.
[5] CC← 2 2 2 2 ↑ 4 8 2 [0p0IO+1*XX] A Else set condition code.
▽

▽ OPCODE TRACE OPERANDS;I;OPS;R;RR
[1] A Display status quo for calling instruction
[2] A and then await user keystroke.
[3] A
[4] →(A/'ON '=5↑,OPERANDS)/ON
[5] →(0=UNC 'TR')/0
[6] →(~TR)/0
[7] →(A/'OFF '=5↑,OPERANDS)/OFF
[8] R←'CC: ',(0CC),(35p'~'),'INSTRUCTION:
[9] OPS+0,OPERANDS
[10] OPSI(OPS=' ')/1pOPSI+', '
[11] R←R,(6↑OPCODE),OPS,UTCNL

```

```

[12] GO:I←ΠIO-1
[13] RR←(-ΠIO)⊙REGS
[14] LOOP:→((1↑ρRR)<(~ΠIO)+I←I+1)/END
[15] R←R,9↑BIN2HEX RR[I;]
[16] A R←R,(⊗DECIMAL RR[I;]),' '
[17] →(0#8|I+~ΠIO)/LOOP
[18] R←R,ΠTCNL
[19] →LOOP
[20] A
[21] OFF:Π←'TRACE TERMINATED'
[22] TR←0
[23] →0
[24] ON:R←'CC: ',(⊗CC),(44ρ'-'),'COMMENCING TRACE',ΠTCNL
[25] TR←1
[26] →GO
[27] A
[28] END:Π←R
[29] I←ΠINKEY
[30] →0

```

▽

At this stage, the simulation has served its purpose and a sensible person would draw a line under it and move on to something else. Unfortunately, I am increasingly tantalised by the vistas which are starting to open up with this thing. After all, when you simulate the instruction set, you simulate the machine itself. If the editors of VECTOR don't take the precaution of changing their identities and moving to another town you may well be hearing more on this topic.

*****

*** FREELANCE APLs ***

*****

**Requirements**

VSAPL and APL2 background - minimum 4 years experience.  
VM/CMS and TSO experience

**Package**

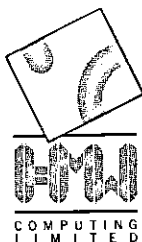
Good Rates  
Possible Overseas Contracts

send CVs to:

Logicom Systems AB (UK) Ltd  
130 Wapping High Street  
9 Gun Wharf  
London E1

**OR**

ring **HARRY** on 01-481-4183



## EXCITING POSITIONS AT THE LEADING EDGE



With top City institutions and  
leading Blue chip companies  
amongst our client list we have a lot to  
offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED  
Hamilton House 1 Temple Avenue  
London EC4Y 0HA Telephone 01-353 4212

**British APL Association Software Library**  
**Short Catalogue**

**DISK 1**

Title: The game of LIFE SIZE = 113K Donor: Paul Chapman

Description: Fast, large universe version of Conway's LIFE Hardware required: PC with IBM mono, IBM colour or Herc mono board. July 1986

**DISK 2**

Title: Fluctuation analysis SIZE = 9K Donor: Claude Bastian

Description: Fluctuation analysis for multichannel spectra. PC Software required: APL*PLUS PC July 1986

**DISK 3**

Title: Free software from APL385 SIZE = 282K Donor: Adrian Smith

Description: DOS file lister, expert system shell, screen design, Epson downloads, digital clock, APL utils, etc. PC Software required: APL*PLUS PC July 1986

**DISK 4**

Title: Piano SIZE = 4K Donor: Gerard Langlet

Description: Simple piano-playing in APL Hardware required: PC Software required: APL*PLUS/PC July 1986

**DISK 5**

Title: Pianoman SIZE = 355K Donor: Stuart Yarus *SHAREWARE*

Description: Very sophisticated piano-playing July 1986

**DISK 6**

Title: FRAME SIZE = 69K Donor: Robert Pullman

Description: Multidimensional numeric array editor. PC Software required: Sharp APL/PC July 1986

**DISK 7**

Title: Get/Put files SIZE = 21K Donor: M Kent

Description: Access filed data by name, not component number. Software required: APL*PLUS/PC July 1986

**DISK 8**

Title: Binary Combinations SIZE = 3K Donor: Otway Pardee

Description: Fast way to get binary image of combinations. Software required: APL*PLUS/PC July 1986

## DISK 9

Title: Workspace Lister SIZE = 16K Donor: Roger Harper

Description: List vars and fns with index and pagination. Hardware required: PC, Epson LQ1500 printer. Software required: APL*PLUS/PC July 1986

## DISK 10

Title: TEXT SIZE = 343K Donor: Ian Feldberg

Description: Typesetting for the IBM PC (needs DISK 11). Software required: APL*PLUS/PC July 1986

## DISK 11

Title: PCPAK SIZE = 33K Donor: Ian Feldberg

Description: GRAPHPAK for the IBM PC. Software required: APL*PLUS/PC June 1986

## DISK 12

Title: Functions for Linear Algebra SIZE = 57K Donor: Garry Helzer

Description: Functions from Helzer's book Applied Linear Algebra. Software required: APL*PLUS/PC July 1986

## DISK 13

Title: DOS utilities SIZE = 57K Donor: Timo Kiravi

Description: View text files, mass copy, DISK directory, command line editor. July 1986

## DISK 14

Title: Translate APL SIZE = 311K Donor: Wayland Products

Description: APL to Reverse Polish Translation. Software required: DOS 2.0, APL*PLUS, IBM, SHARP or PortaAPL. July 1986, April 1987

## DISK 15

Title: RL2 Model 2, Version 3 SIZE = 359K Donor: Wayland Products

Description: Reverse Polish APL Model written in APL. April 1987

## DISK 16

Title: F83VEC Demo SIZE = 320K Donor: Wayland Products

Description: FORTH extended to use APL symbols. Software required: DOS 2.0 or higher. July 1986, April 1987

## DISK 17

Title: Migrate APL SIZE = 268K Donor: Wayland Products

Description: Migrates APL Workspaces to and from DOS files and can print them on NEC Spinwriter printers. Software required: Sharp, IBM, Porta or APL*PLUS. February 1987, April 1987

## DISK 18

Title: FFILES SIZE = 48K Donor: A N Wiggins

Description: Component files for APL2 using AP121. Hardware required: MAINFRAME, PC Software required: APL2, APL*PLUS/PC March 1987

## DISK 19

Title: CAVEDOS SIZE = 83K Donor: Mark Bassett

Description: Adventure front-end to DOS. Software reqd: APL*PLUS/PC 6.0 March 1987

## DISK 20

Title: VecRL Command File SIZE = 210K Donor: Wayland Products

Description: APL style Reverse Polish vector language Hardware required: IBM PC, NEC Spinwriter, optional STSC ROM. Software required: DOS 2.0 April 1987

## DISK 21

Title: Christmas 1986 SIZE = 346K Donor: Wayland Products

Description: 16 Christmas Carols played by PortaAPL or VecRL. April 1987

## DISK 22

Title: User-Supported APL Software DemoDISK *DEMODISK* SIZE = 275K

Donor: Dick Holt

Description: Sampler/demo of six user-supported workspaces. Software required: STSC APL*PLUS PC v3.0+ May 1987

## DISK 23

Title: RAMFONT SIZE = 63K Donor: Adrian Smith

Description: APL fonts and font editors for Herc+ and EGA cards. Hardware required: IBM PC with ECM/EGA or Herc+ mono. Software required: APL*PLUS/PC June 1987

## DISK 24

Title: Uniware Demonstration DISK SIZE = 360K Donor: Uniware

Description: Demonstration of Unitab and The APL Debugger. July 1987

## DISK 25

Title: APL*PLUS/PC Demonstration DISK SIZE = 309K Donor: Cocking & Drury

Description: Demo of the dominant APL interpreter for the PC Hardware required: IBM PC and compatibles, PS/2. 384K minimum memory. August 1987

## DISK 26

Title: STATGRAPHICS Demonstration DISK SIZE = 306K Donor: Cocking & Drury

Description: Demonstration of the most comprehensive statistical & graphics package for PCs Hardware required: IBM PC & compatibles, PS/2. August 1987

**British APL Association  
Software Library****Software Submission (1)**

Please copy and fill in this form for EACH disk you submit.

Details corresponding to items flagged (*) will NOT be made publically available, but are for our records only. Please Use BLOCK CAPITALS for all items except number 18.

0. Submission date: _____
1. Name of donor: _____ 2. Daytime phone(*): _____
3. FULL address(*): _____  
_____
4. Electronic mail addresses(*): _____
5. Short disk title: _____
6. Brief description of disk : _____  
_____  
_____
7. List target machine: _____
8. List additional software required: _____
9. Indicate special hardware requirements: _____
10. Is user documentation provided on the disk?(Y/N): _____
11. List titles of any paper documentation included with your submission  
_____  
_____
12. Is this documentation, or any other, available to users upon application to you?(Y/N): _____ Please give details: _____  
_____
13. Does the disk include any form of payment request from users of the software?(Y/N): _____ Please give details: _____  
_____
14. Does your submission constitute a 'demonstration disk' in that it demonstrates software that is available for purchase?(Y/N): _____
15. If the TARGET machine is NOT a DOS-based PC, does the disk include instructions for transfer to the target machine?(Y/N): _____

## Software Submission (2)

- For APL workspaces, please use the space below to document functions

[illegible]

- Signature: _____

125

# British APL Association Software Library

## Order Form

To: Dave Ziemann  
Flat 3, 63 Queens Crescent  
London NW5 4ES, ENGLAND

PLEASE SUPPLY THE DISKS CIRCLED BELOW:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99	100							

TOTAL NUMBER OF DISKS ORDERED (please double-check): _____

at 2 Pounds per disk : _____ BAA Member's rate

OR at 3 Pounds per disk : _____ Non-member's rate

I want to join the BAA, and enclose  
the 10 Pounds annual membership fee : _____

Postage and handling : _____ 1.00

Add 2 Pounds for orders outside the UK: _____

Total order - remittance included: _____

Your signature: _____

Your name: _____

Full address: _____

Please make cheques payable to the British APL Association.

Payment can be made in US Dollars - pay 1.5 US Dollars for each Pound.

All orders must include payment - allow 30 days for delivery.

BAA membership year runs from 1 May - 30 April. People joining part way through the year will receive appropriate back numbers of Vector.

Software is accepted by the British APL Association on good faith and we do not vouch for or make any claims regarding donated software. The BAA shall not be held responsible for damage caused by the use of library software, and shall not be liable in any way in the event that submitted material proves to be the subject of third party copyright. Your signature above accordingly indemnifies the BAA from all liability.

## Index to Advertisers

APL People Ltd	14,16
Cocking and Drury Ltd	6,80,81
Dyadic Systems Ltd	78
HMW Computing	53,120
Logicom	14,120
MicroAPL Ltd	2
APL Booklist (Ed Shaw)	54

All queries regarding advertising in VECTOR should be made to the advertising editor, Cathy Dargue, at the following address:

Cathy Dargue,  
60 Downhall Ley,  
Buntingford,  
Herts SG9 9TL

Tel: 0895-31313 x3038

## BRITISH APL ASSOCIATION Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____  
 Department: _____  
 Organisation: _____  
 Address Line 1: _____  
 Address Line 2: _____  
 Address Line 3: _____  
 Post or zip code: _____  
 Country: _____  
 Telephone Number: _____

Membership category (please ring): .....88/89    89/90

Non-voting student membership (UK only) .....	£5	£5
UK private membership .....	£10	£10
Overseas private membership .....	£18	£18
Airmail supplement (not needed for Europe) .....	£8	£8
Corporate membership .....	£85	£85
Corporate membership overseas .....	£140	£140
Sustaining membership .....	£360	£360

For student applicants:

Name of course: _____  
 Name and title of supervisor: _____  
 Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

### PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,  
 Alison Chatterton, 9 Oak Grove,  
 HERTFORD SG13 8AT  
 England

## The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

### 1988/89 Committee.

Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Graham Parkhouse 0483-571281 (x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH.
Treasurer:	Philip Goacher 03727-21282	27 Downs Way, EPSOM, Surrey
Activities:	Jonathan Martin 01 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Journal Editor:	Adrian Smith 0904-653071 (x3382)	Brock House, Gilling East, YORK.
Education:	Dr Alan Sykes 0792-295296	Dept of Management Science University College of Swansea, Singleton Park, SWANSEA SA2 8PP
Publicity:	Cathy Dargue 0707-325161 (x2418)	60 Downhall Ley, BUNTINGFORD, Herts SG9 9TL
Technical:	Vacant post	
Recruitment:	Val Lusmore 0225-448501	17, Barton Street, BATH, Avon
Projects:	David Eastwood 01-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG9 8AT

### Journal Working Group

Jonathan Barman	0374-588835
Anthony Camacho	0727-60130
Cathy Dargue	0707-325161
Val Lusmore	0225-448501
Adrian Smith	04393-385
David Ziemann	01-436 9481

### Activities Working Group

Phil Goacher	03727-21282
Maurice Jordan	01-562 3090
David Parker	01-834 2333

Typeset using MS Word 4.0 on Xerox 3700 (Covers on Apple Laserwriter)

Printed in England by Short-Run Press Ltd, Exeter

## VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

## SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People  
The Old Malthouse  
Clarence St, BATH, BA1 5NS  
Tel: 0225 462602

Cocking & Drury Ltd  
180 Tottenham Court Rd  
LONDON, W1P 9LE  
Tel: 01-436 9481

HMW Computing Ltd  
Hamilton House,  
1 Temple Avenue,  
LONDON EC4Y 0HA  
Tel: 01-353 4212  
Fax: 01-353 3325

Inner Product Ltd  
Eagle House  
73 Clapham Common Southside  
LONDON SW4 9DG  
Tel: 01-673 3354

MicroAPL Ltd  
South Bank Technopark  
90 London Road  
LONDON SE1 6LN  
Tel: 01-922 8866

Peter Cyriax Systems  
213 Goldhurst Terrace  
LONDON NW6 3ER  
Tel: 01-624-7013  
Mobile: 0860-377963

APL Software Tech  
14 Rosewood Avenue  
Alveston, BRISTOL BS12 2PP  
Tel: 0454-415737

Dyadic Systems Ltd  
Park House, The High Street  
ALTON, Hants  
Tel: 0420-87024

APL Impetus Ltd  
Rusper, Sandy Lane  
Ivy Hatch, SEVENOAKS  
Kent TN15 0PD  
Tel: 0732-885126

MetaTechnics Systems Ltd  
356 West Barnes Lane  
Motspur Park, Surrey  
Tel:

I.P. Sharp Associates  
10 Dean Farrar St.  
LONDON SW1H 0DX  
Tel: 01-222-7033



The British Computer Society, 13 Mansfield Street, London W1M 0BD.